

[양식 5] 자작자율차 기술보고서

대학생 창작 모빌리티 경진대회

자작자율차 기술보고서

- 자동차번호 : 102
- 참가팀명 : Team K.A.I.
- 참가대학명 : 건국대학교
- 주요내용 (제2장 자작자율차 규정에 따라 작성)

제2장 자작자율차 규정	
2.1 일반 조건	2.10 전방시계
2.2 설계 요구 조건 (전장, 전폭, 축거, 윤거, 무게, 최저지상고 등)	2.11 머리지지대
2.3 차체 구조 및 형상	2.12 운전자 보호 장구
2.4 시동장치	2.13 범퍼
2.5 구동모터 및 동력발생장치(모터)	2.14 자동차 번호판 등
2.6 배터리 규격	2.15 비상 전원차단 스위치
2.7 가상 엔진음 발생시스템	2.16 자율주행시스템 (센서 및 액츄에이터 등 구성장치, 자율주행 알고리즘 등)
2.8 제동장치	
2.9 후사경	2.17 자동차 검사 및 승인

- ※ 1. 작성양식은 제2장 자작자율차 규정에 따라 작성하여 규정에 만족여부를 확인할 수 있도록 하여야 한다.
2. 주요장치에 대해서는 이전년도 출전자동차와의 차이점에 대하여 기술되어 있어야 한다.
(휴먼명조체로 크기는 12포인트, 줄간격은 160%로 작성)

■ 첨부

- 자동차 제원표 1부 (양식 6)

2025. 09. 27.

Team K.A.I. 팀장 허원석 

대학생 창작 모빌리티 경진대회 조직위원장 귀하

[양식 6] 자작자율차 제원표

대학생 창작 모빌리티 경진대회

자작자율차 제원표

- 자동차번호 : 102
- 참가팀명 : Team K.A.I.
- 참가대학명 : 건국대학교

▶ 일반제원

구 분	제 원	보 기
차량 중량	203 (kgf)	
차량 최대 길이	2500 (mm)	
차량 최대 폭	1205 (mm)	
차량 최대 높이	1300 (mm)	
Wheel Base	1300(mm)	
조향륜	전 2륜	
Steering 형식	Ackermann 방식	
최소회전반경	2.4 (m)	
Brake 형식	Caliper type	
Brake 작동륜	4륜	
Suspension 형식	더블위시본	
Frame 설계상 특징	안정성, 높은 강성	
Frame 재질	Steel	
기어비(스프로킷)	60/13	
기타		

▶ 자동차 제원

구 분	제 원	보 기
모터	정격 출력 : 10kW 최대 전류 : 358.5A / 54.55sec (최대토크 기준) 효율 : 91.3%	예) 정격출력, 효율 등
배터리	사용 전압 : 39.2 ~ 58.8V 정격 전압 : 51.8V 최대 전류 : 350A / 10sec 용량 : 78Ah	예) 사용전압, 최대전류, 용량
회생제동유무	유	
기술적 특징	릴레이를 통한 하드웨어 제어	
기타	<보조 배터리> 사용 전압 : 12.8V 최대 전류 : 50A 용량 : 640Wh	

▶ 자율주행 시스템 제원

구 분	제 원	보 기
센서	<카메라> 해상도/FPS : 640x360@30fps x 2 시야각 : 78도 연결 : USB 2.0	
	<LIDAR> 채널 : 32 채널 감지 거리 : 최대 120 m 정확도 : $\pm 1.5 \sim 5$ cm 데이터량 : 655,360 points/sec 보호 등급 : IP68/IP69K	
	<IMU> 센서 구성 : 9축 (가속도, 자이로, 지자기) 출력 데이터 : 자세, 각속도, 가속도 출력 속도 : 최대 100 Hz 연결 : USB, UART	
액추에이터	<스티어링 모터> 정격 전압 : 12V 정격 출력 : 50W 정격 토크 : 7N·m 최대 토크 : 13N·m 정격 회전 속도 : 80rpm 입출력 신호 : CAN_H, CAN_L	
	<브레이크 서보모터> 전압 : 12V-24V 정격 토크 : 140kg·cm 최대 토크 : 400kg·cm 입력 신호 : PWM (logic level 5V) 작동 각도 범위 : 30°~150° 무부하 작동 속도 : 0.74s/60°(12V), 0.37s/60°(24V)	
기타 (GPS 등)	<GNSS> 수신 채널 : 184 채널, 멀티밴드 정확도 : 센티미터급 (RTK) 갱신 주기 : 최대 20 Hz 연결 : USB, UART	

2025. 09. 27.

Team K.A.I. 팀장 허원석 )

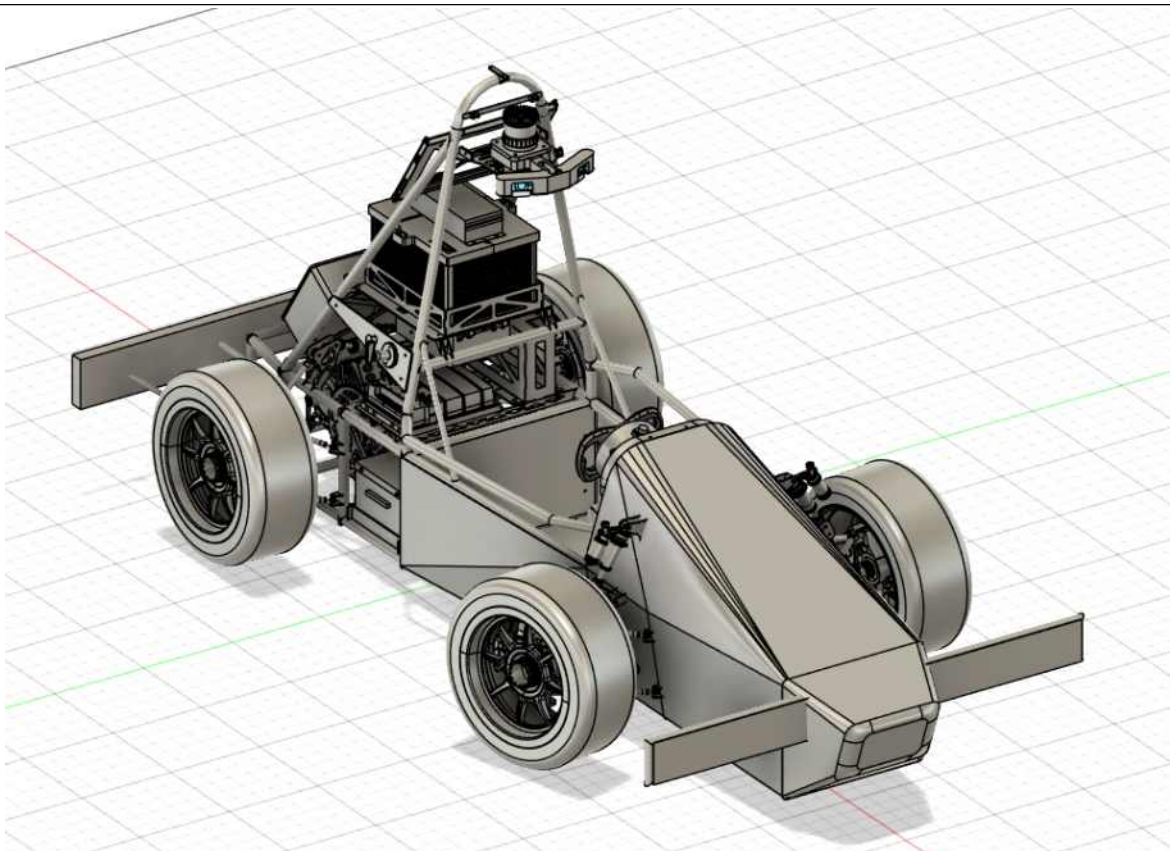
대학생 창작 모빌리티 경진대회 조직위원장 귀하

1. 서론

자율주행 자작자동차를 개발하는 데 있어 하드웨어와 소프트웨어의 통합은 필수적이다. 본 기술보고서는 건국대학교 Team K.A.I.가 2025년 대학생 창작 모빌리티 경진대회 자작자율차 부문에 출전하며 개발한 차량의 전체 기술적 과정을 소개한다. 하드웨어 및 소프트웨어 설계부터 실제 적용까지의 내용을 중심으로 기술하였다.

본 팀은 2024년에도 동일한 대회에 참가하였으나 당시 차량의 하드웨어적 안정성 부족과 소프트웨어의 완성도 문제로 인해 수상에 실패했다. 이러한 결과를 심도 있게 분석한 결과, 차량 구조의 경량화 및 강성 확보, 그리고 인지-판단-제어 전 과정의 신뢰성과 실시간성 강화가 필요하다는 결론에 도달하였다.

2025년 차량은 이러한 반성과 분석을 바탕으로 플랫폼 전면 개선, 센서 하드웨어 재설계, 소프트웨어 아키텍처 최적화, 그리고 ROS 2 기반 모듈화 전략을 통해 새롭게 제작되었다. 본 보고서에서는 해당 자율주행 차량의 구조, 개발 철학, 적용 기술 및 문제 해결 과정을 기술하며, 직면했던 문제와 그에 대한 해결 방안을 중심으로 서술하고자 한다.



2025년 Team K.A.I. 자작자율차 'KAIEV_25' 최종 설계 모델

2. 차량 설계

2.1. 설계 목표

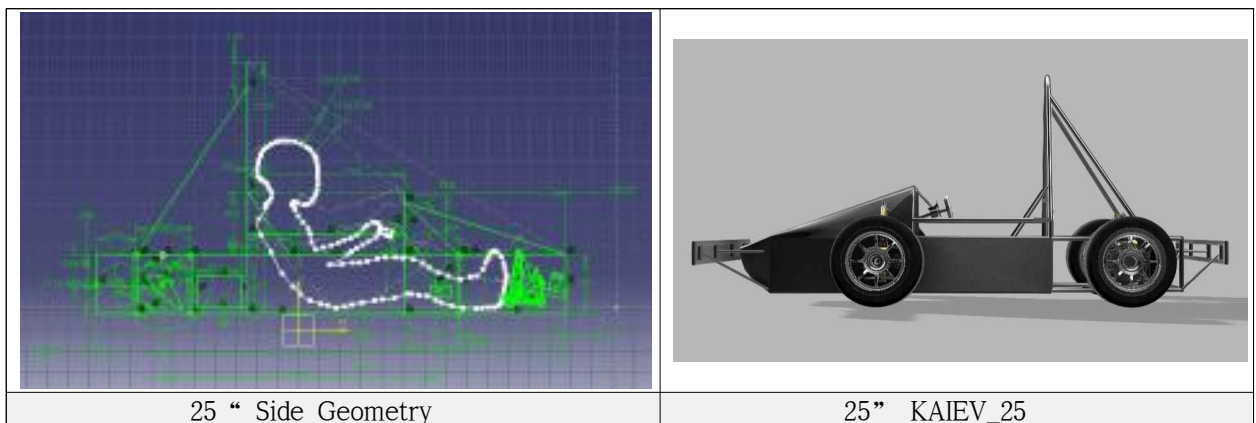
올해 2025년도 Team K.A.I.는 경량화, 단순성, 안정성, 정비성 이 4가지를 목표로 잡고 차량을 설계하였다. 또한 이전 차량에서 발생한 문제점들을 분석하여 수정, 차량의 성능을 전년도 대비 더욱 발전시켰다.

2.2. 프레임 설계

2.2.1. Side Geometry

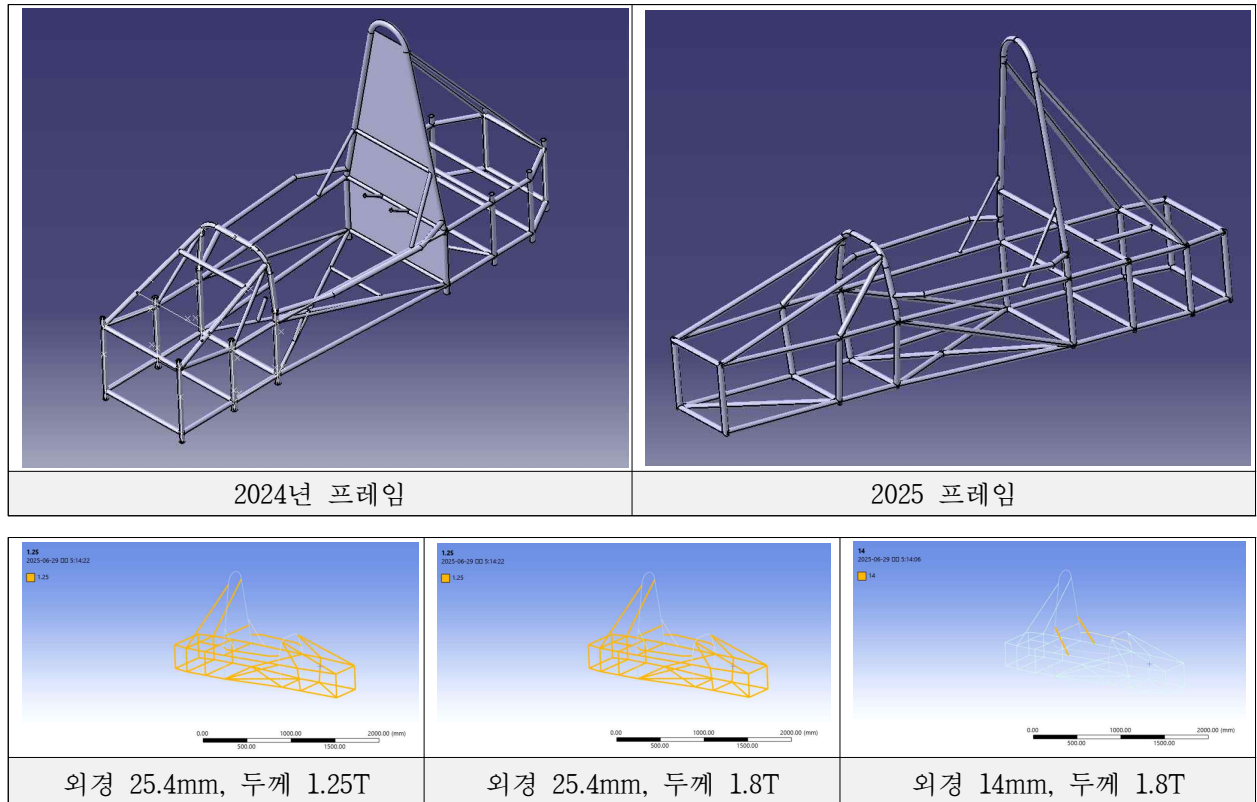
24년도 차량은 설계 시 Driver Position의 부재로 인해 운전 중 스티어링 휠이 드라이버의 다리에 부딪히는 문제가 발생하였다. 또한, 시트와 Pedal Box 간의 거리가 멀어 드라이버의 다리가 브레이크를 강하게 밟지 못하였다. 이를 해결하기 위해 이번 차량은 Driver Position을 반영한 Side Geometry를 설계하여 Driver가 탑승 중 다른 부분에 불편함을 느끼지 않도록 하였다. 또한, 브레이크 페달, 랙앤피니언, 인클로저 박스 등과 드라이버와의 간섭을 고려하여 최적의 Side Geometry를 설계하였다.

또한, 전방시계 기준을 충족하도록 프레임 구조를 설계하였고, 주행 중 후방 상황을 명확히 인지할 수 있도록 후사경을 장착하였다. 운전자 보호를 위해 설치된 머리 지지대(헤드레스트)는 폭 200mm, 높이 130mm, 두께 50mm의 크기로 제작되었으며, 지상으로부터 960mm 높이에 위치시켜 추돌 상황에서 운전자의 머리와 목을 효과적으로 보호할 수 있도록 하였다.



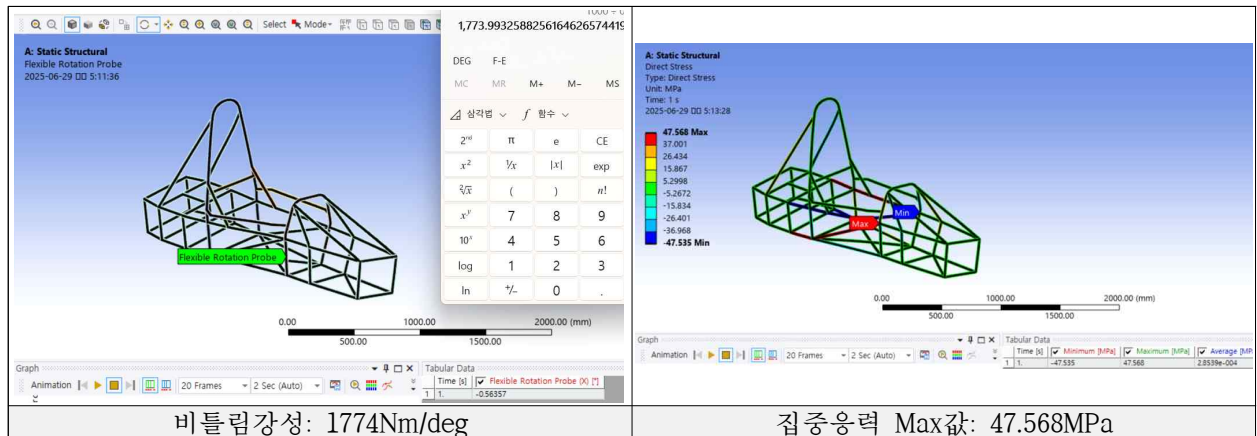
2.2.2. Main Frame

이번 차량 Frame의 핵심은 최소 무게, 최대 강성이다. 프레임은 20kg 대의 무게와 1000~2000 Nm/deg의 비틀림 강성을 목표로 설계하였다. 24년도 차량의 비틀림 강성이 낮았기 때문에 24년에 주로 사용했던 외경 22.2mm 파이프가 아닌 외경 25.4mm 파이프를 주로 사용하였고 엔시스(ANSYS)로 비틀림 강성과 변형률을 확인해 보며 프레임의 안정성을 확인하였다.



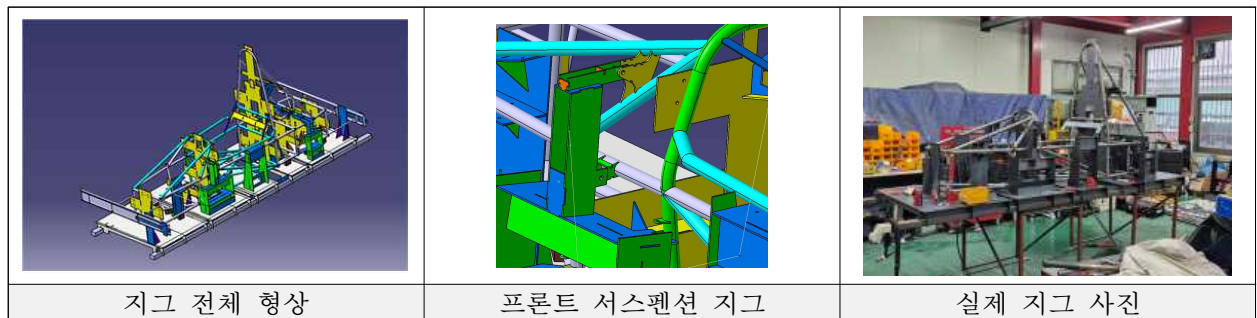
i. Analysis

엔시스(ANSYS)를 사용하여 차량의 무게와 비틀림 강성, 집중응력을 확인하였다. 2024년 차량과 이번 차량의 비틀림 강성과 무게를 비교했을 때, 비틀림 강성은 632Nm/deg에서 1774Nm/deg, 무게는 18.8kg에서 25.3kg로 측정되었다. 비록 무거워졌지만 비틀림 강성을 전보다 크게 확보하였으므로 차량의 운동성능이 좋아질 것이다.



ii. Jig

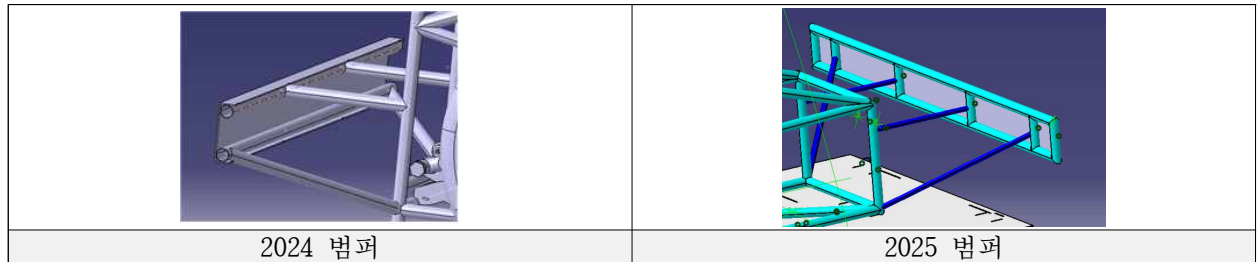
25년도 차량의 Frame JIG는 비용 절감 및 제작 오차의 최소화를 목적으로 설계되었다. 이를 위해 24년도 차량에 사용되었던 Base Plate를 삭제하고, 사각 파이프를 바닥 부재로 사용하여 바닥의 휨으로 인한 제작 오차를 최소화하였다.



또한 사각 파이프를 사용함으로써 기존 Base Plate와 같은 대형판을 삭제, 파이프 단위로 분할 후 조립하여 바닥 부재를 제작하였으며, 이에 따라 대형판의 운반과 가공에 필요한 비용을 절감하였다.

iii. 범퍼

24년도 범퍼 외경을 22.2mm를 사용했지만 25년도에는 25.4mm를 사용하고 범퍼의 세로 부재를 추가함으로써 범퍼의 안정성을 높이고 에너지 흡수 능력을 높여 운전자 보호와 차량 파손을 최소화하려 했다. 범퍼 가로 길이는 해당하는 바퀴 끝단의 길이로 결정하였다. 차체와 범퍼 사이의 간격은 외장 크기의 자유도를 증가시키기 위해 222.2mm에서 292.2mm로 늘리게 되었다.



iv. 운전자 보호 장구

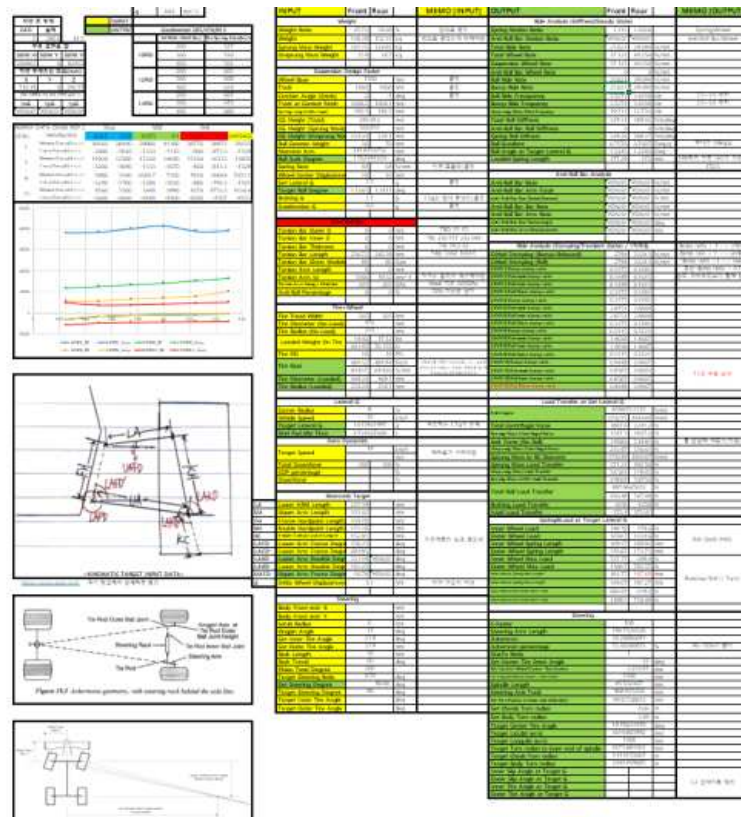
프레임 설계 시 강성 확보와 더불어 운전자 보호 장구의 체결 안정성을 최우선으로 고려하였다. 운전자를 견고하게 지지하기 위해 6점식 안전벨트를 채택하였으며, 벨트 간 거리는 200mm, 폭은 790mm로 설계하여 신체를 효과적으로 고정하도록 하였다. 비상 상황 시 신속한 탈출이 가능하도록 퀵 릴리스 타입의 버클을 적용하는 등 대회 규정을 철저히 준수하였다.

2.3. 샤시 설계

Chassis Part의 목표는 운전자의 자세를 고려한 지오메트리 설계를 통해 안정적인 차량 거동과 현가하질량 감소를 통한 운동 성능 최적화, 그리고 타이어 데이터를 기반으로 한 Steering Geometry 설계를 완성하는 것이다.

2.3.1. 지오메트리 결정

차량의 무게 배분은 가속 주행, 중합 주행을 포함한 모든 주행 상황에서 타이어 접지를 최대한 확보하기 위해 전, 후륜을 50:50을 목표로 부품 배치를 최적화하였다. Upright 패키징을 위해 13인치 휠을 선정하였고, 지름이 작은 Continental C19 205/470 R13 타이어를 선택하여 10psi 기준 약 85N/mm의 타이어 레이트를 목표로 설정하였다. 코너링 시 최대 하중 이동 조건을 횡 가속도 1.5G로 설정하여 현실적인 주행 상황을 반영하였다. 최저지상고는 50mm로 설정하여 노면 간섭을 방지하면서 무게중심 상승을 최소화했고, 윤거는 1000mm로 설계하여 코너링시 안정성과 접지력을 확보하였다. 하중 이동이 최대로 작용할 때 (전륜 기준 코너링 및 제동, 후륜 기준 코너링 및 가속), 횡 가속도를 1.5G로 가정하여 지오메트리를 설계했다.



Target Ride Frequency는 3Hz 중반으로 가정하여 롤 상황 시 전륜은 3.53Hz, 후륜은 3.43Hz로 범프 상황 시 전륜은 3.53Hz, 후륜은 3.43Hz로 도출하였다. 이를 위해 Ride Rate의 변수값인 Wheel Rate를 조정하였고, Hard Point와의 제약조건을 만족하는 범위에서 휠 센터 변위(최대 변위 10mm)를 설정하여 최대 Body Roll을 계산했다.

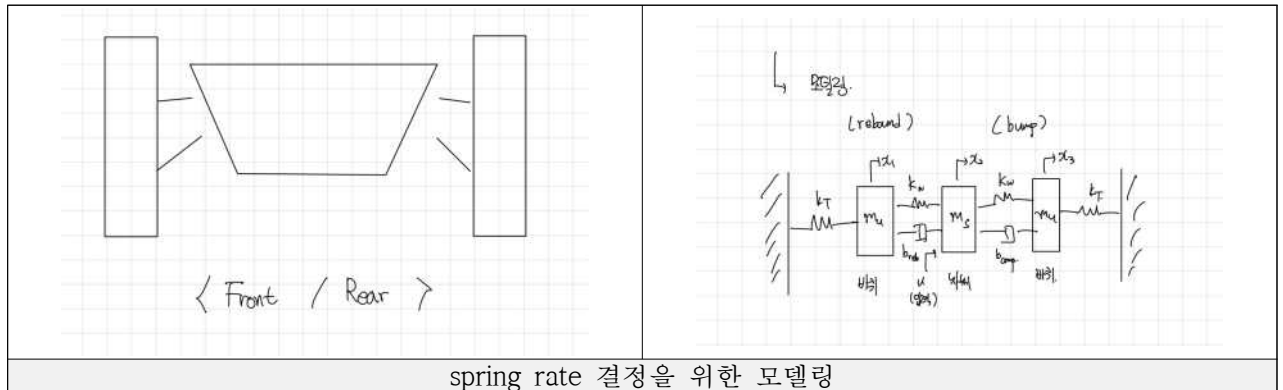
하중 이동량 [N]	430.4836
Wheel Rate [N/mm]	37.12139
Ride Rate [N/mm]	25.82061
Roll Ride Frequency [Hz]	3.527071
Bump Ride Frequency [Hz]	3.527071

<Geometry Design 주요 수치 (전륜)>

하중 이동량 [N]	567.4832
Wheel Rate [N/mm]	44.25584
Ride Rate [N/mm]	29.09438
Roll Ride Frequency [Hz]	3.435935
Bump Ride Frequency [Hz]	3.435935

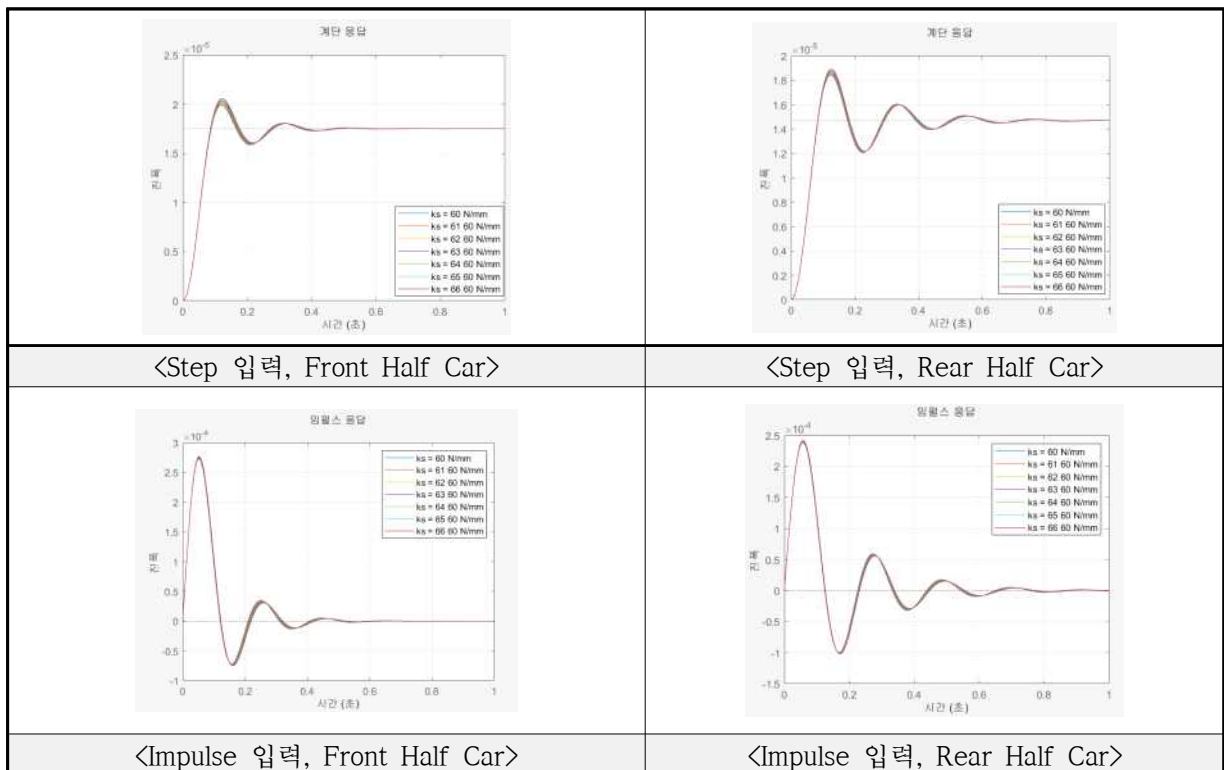
<Geometry Design 주요 수치 (후륜)>

2.3.2. 스프링 레이트 결정



Matlab/Simulink를 통해 Quarter car 및 Half car의 수학적 모델링을 통해 spring rate를 결정하였다. Quarter car 모델링에서는 변위의 입력(step 함수)을 통한 경향성을, Half car 모델링에서는 코너링 시 하중 이동을 고려한 단위 힘 입력(step 함수)을 통해 경향성을 파악하였다. 급격한 스티어링 후 중립 위치로 돌아올 때의 경향성은 임펄스 입력을 통해 확인하였다.

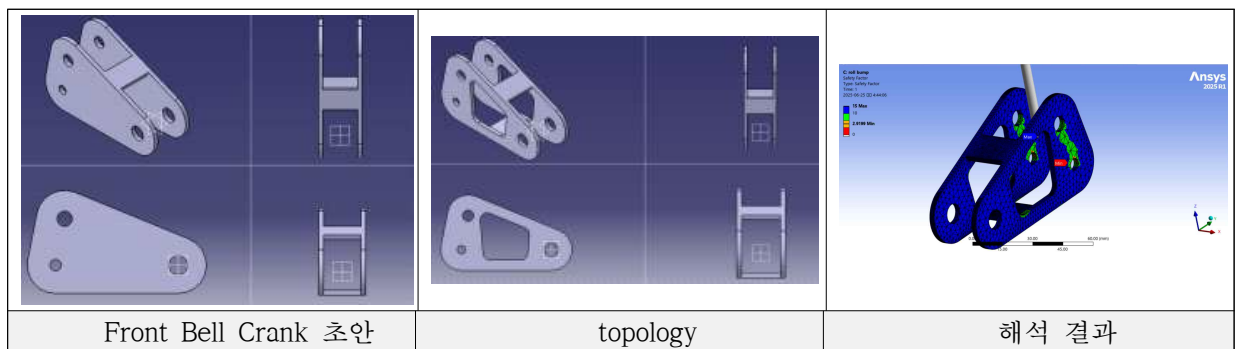
그래프 해석 시 모션비 및 정확한 강성을 위한 제작 가능한 스프링을 고려하여 설정하였다. 전륜 spring rate와 후륜 spring rate를 모두 60N/mm를 목표하여 경향성을 확인하였고, 스프링 강성 변화에 따른 결과가 비슷하여 작년 제작 차량의 스프링 강성인 64N/mm로 결정하였다.



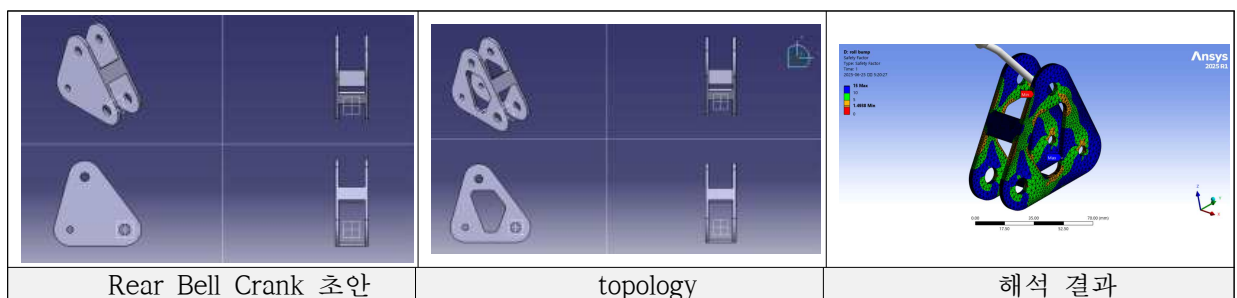
2.3.3. 벨크랭크

최대 코너링 시 횡 가속도를 1.5G로 설정하여 더 현실적이고 안정적인 설계 기준을 설정하였다. 또한, 24년도 차량의 벨크랭크 하중 계산 시, 스프링의 정적 하중 (Static preload)은 고려하지 않고 계산한 오류를 발견하여 정적 및 동적 하중을 모두 고려하여 해석의 신뢰성을 높였다.

벨크랭크의 해석 조건으로 전륜은 최대 코너링에서의 횡 가속도로 인한 spring 압축과 푸쉬로드의 반력을 적용하여 topology optimization을 진행하였다. 예상외의 동적 하중을 대비하여 동적 계수는 2.0을 적용하였다.

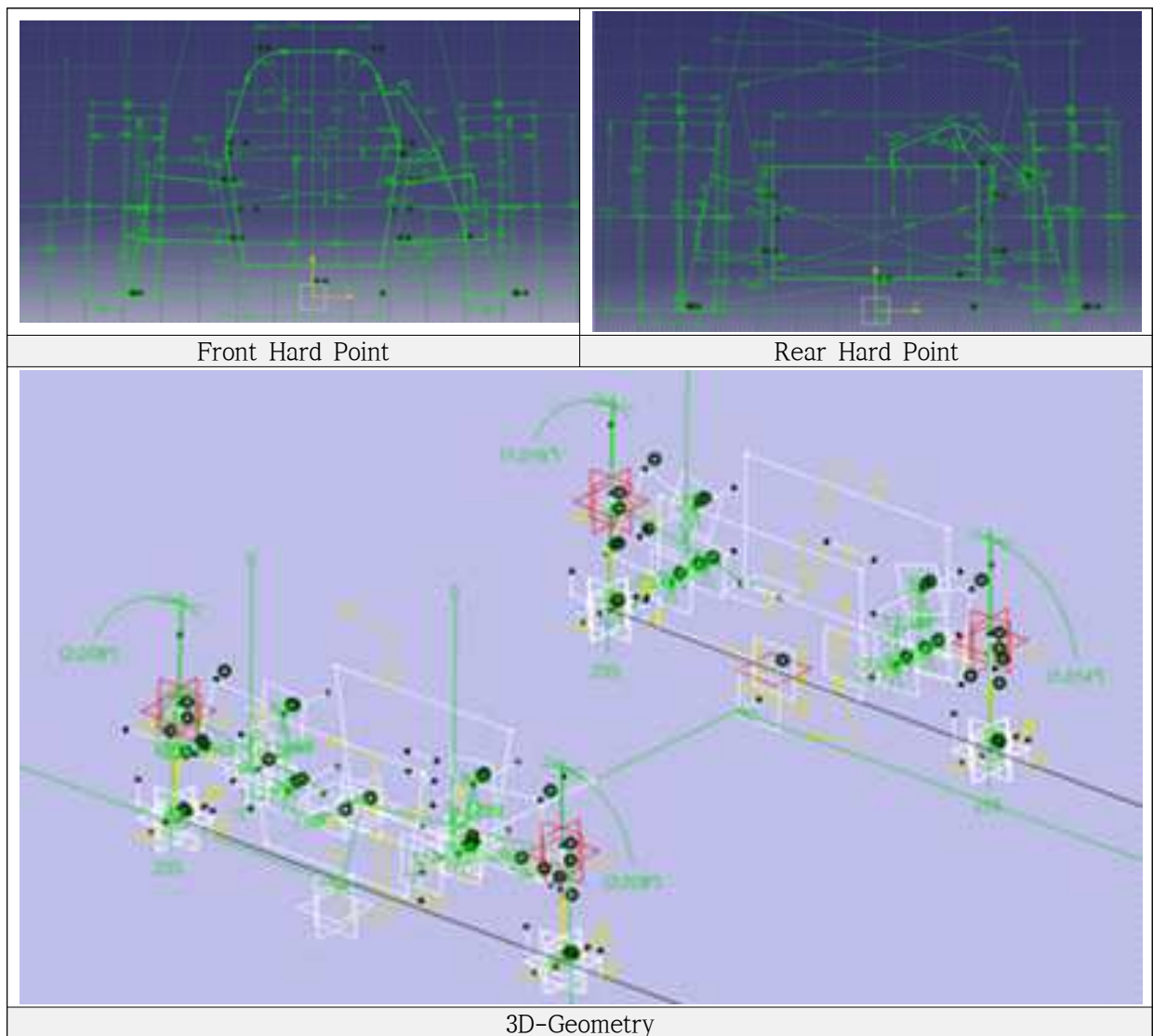


후륜 역시 최대 코너링 시 최대 코너링에서의 횡 가속도로 인한 spring 압축과 푸쉬로드의 반력을 적용하여 topology optimization을 진행하였다.



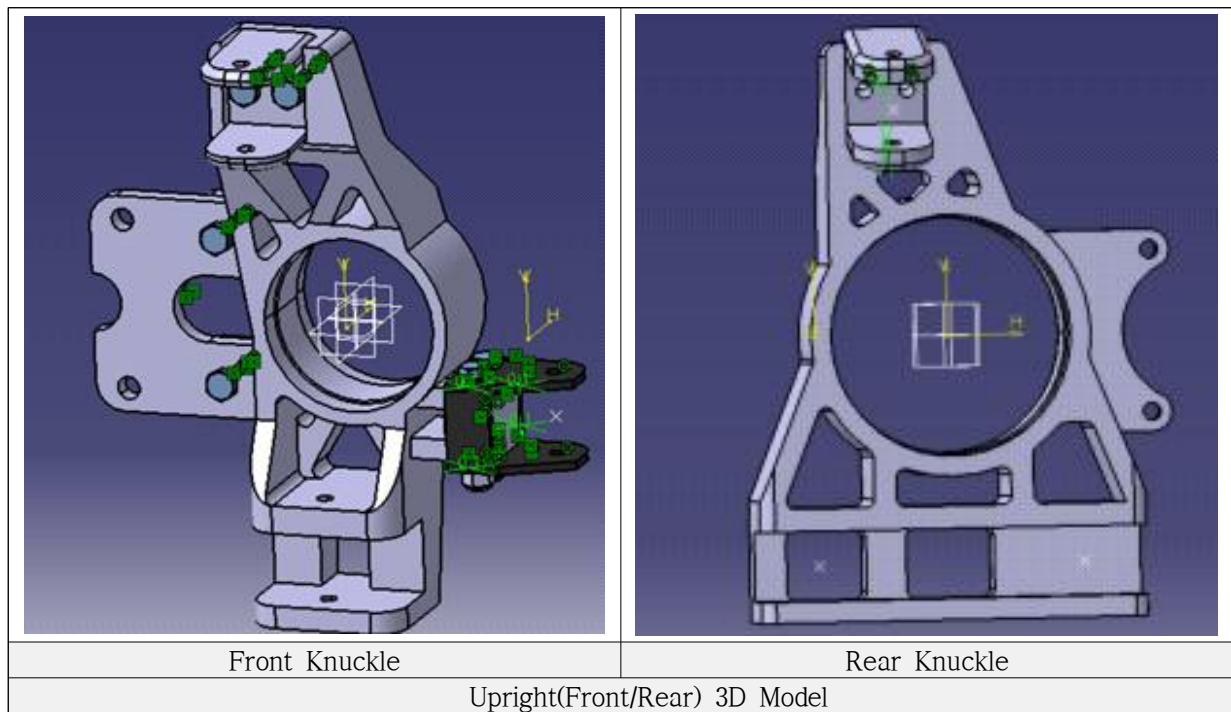
2.3.4. 하드포인트

Hard Point는 최대 코너링 상황에서 롤이 발생할 때 차량의 외측 타이어가 -1~0도 범위 내 캠버 값이 도출되도록 모든 주행 상황 시 양의 캠버 발생을 피하게끔 설계했다. 차량 정면도 기준 좌측 바퀴로부터 스윙 암 길이 변화를 주어 우측 코너링 시 롤 상황에 따른 캠버 변화를 확인했다. 최종적으로 선정된 HardPoint는 다음과 같다.



2.3.5. 업라이트

Hard Point를 기반으로 Compact Packaging과 현가하중량 감소를 목표하여 설계하였다. 재료는 AL-7075를 선정했고, 구조적 안정성과 최대 경량화를 추구하는 것을 목표로 하였다. 하드 포인트 상의 수직 길이 170mm를 업라이트의 크기로 설정하여 작년 차량과 비교한 결과 22.73% downsizing 했다. 후륜의 경우 Toe Link에 작용할 힘에 대비해 Toe Base를 100mm로 설정했고, 전륜과 달리 KPI를 23°로 높여 업라이트 형상을 다르게 설계했다.



차량 하중을 전달받는 푸시로드가 Upper Arm에 장착되어 있어 최대 응력 발생 지점에 보강 설계를 진행했다. 하중 계산 결과를 바탕으로 위상 최적화를 진행했으며 작년 차량과 비교하여 전륜, 후륜 각각 52.35%, 41.18% 경량화를 달성했다.

	24" Front	25" Front	24" Rear	25" Rear
Weight [kg]	1.7	0.81	1.7	1.0
최소 안전율	1.5	1.95	1.9	2.3
경량화	-52.35%		-41.18%	

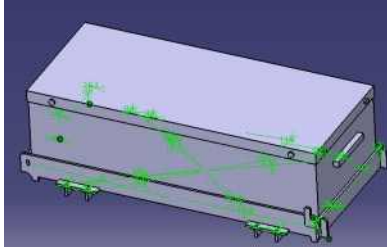
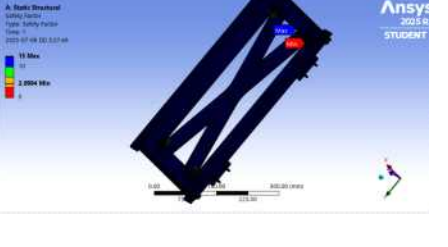
< 24" 차량 / 25" 차량 Upright 비교 >

2.3.6. HV 배터리 마운트.

배터리 마운트는 단단하게 고정하기 위해 사면을 막고 그 중 한 면은 탈부착이 가능하도록 설계하였다. 따라서 마운트 하부와 세 옆면은 용접, 나머지 한 면은 자유롭게 뺄 수 있도록 조립식으로 설계하고 손잡이용 브라켓을 부착하였다.

배터리를 받치는 바닥면에는 트러스 구조와 필렛을 적용함으로써 경량화 및 하중 분산을 의도하였다. 또한, 파이프와 마운트 간 간극을 메우는 형상의 브라켓을 추가하여 하중 전달을 균일화 하였다.

Ansys 해석 결과, 약 26kg 배터리에 두 배 하중을 적용한 조건에서 마운트 바닥면 트러스의 모서리 부분에서 최소 안전계수는 2.1, 나머지 부분에서 약 15의 안전계수를 확보하였다.

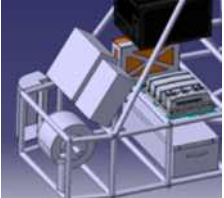
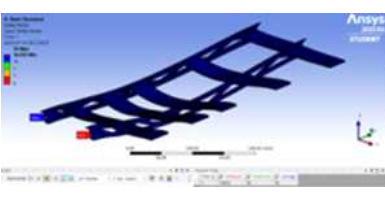
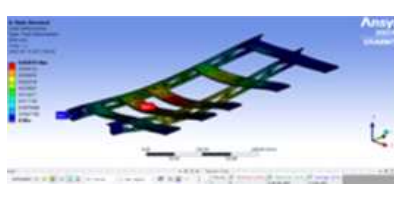
		
배터리 마운트 설계	제작된 배터리 마운트	배터리 마운트 구조 해석 - 전체

2.3.7. 컨트롤러 및 LV 배터리 마운트

컨트롤러는 라디에이터와 연결되는 냉각 호스의 길이를 최소화하기 위해 방화벽 뒤 상부 프레임에 배치하여 쿨링 라인을 단순화하고 냉각 효율을 높였다.

또한, 구조적 강성을 확보하기 위해 프레임 상단에는 수직 판형 보강 브릿지를 용접하고 그 위에 두께 3mm 철제 플레이트를 올렸다. 이를 통해 휨 저항성이 큰 단면 2차 모멘트를 확보하였으며, 냉각 호스와의 간섭을 피할 공간을 마련함으로써 제작성을 향상시켰다.

모터 컨트롤러와 LV 배터리의 하중은 플레이트 전면면에 분포하중으로 작용하며 이를 압력 형태로 적용하여 해석을 수행하였다. Ansys를 통해 수직 브릿지에 토폴로지 최적화 형상을 적용한 결과, 응력 집중이 완화된 최대 변위 0.03mm, 최소 안전계수 14.6을 확보함으로써 경량화와 구조적 안정성 모두 달성하였다.

		
모터 컨트롤러 및 LV 배터리 마운트의 어셈블리 형상	구조 해석 결과: 안전계수 분포	구조 해석 결과: 전체 변위 분포

2.4. 드라이브트레인

2.4.1. 종감속비 선정

금년도 차량은 전년도 주행 테스트를 통해 얻은 로깅 데이터를 기반으로 Optimum Lap 시뮬레이션 툴을 이용해 최적의 종감속비를 선정하였다. 대학생 창작 모빌리티 경진대회 자작자율차 부문의 경기 규정을 반영하여 가속성능, 조향성능, 종합주행성능 코스를 기준으로 시뮬레이션을 진행하였다.

전년도 차량에서는 가속 시뮬레이션을 통해 종감속비 5.46에서 최적의 가속 성능을 발휘할 수 있음을 확인하였다. 그러나 감속비가 지나치게 높을 경우, 모터의 고속 운전 시간 증가 및 열적 부하 증가로 인해 출력 효율이 저하되고, 내구성에도 악영향을 줄 수 있다는 분석 결과를 도출하였다. 이에 따라 동일한 대기어(60T)를 사용하는 조건에서, 종감속비 5.0 이상 구간은 배제하는 것으로 설계 기준을 설정하였다.

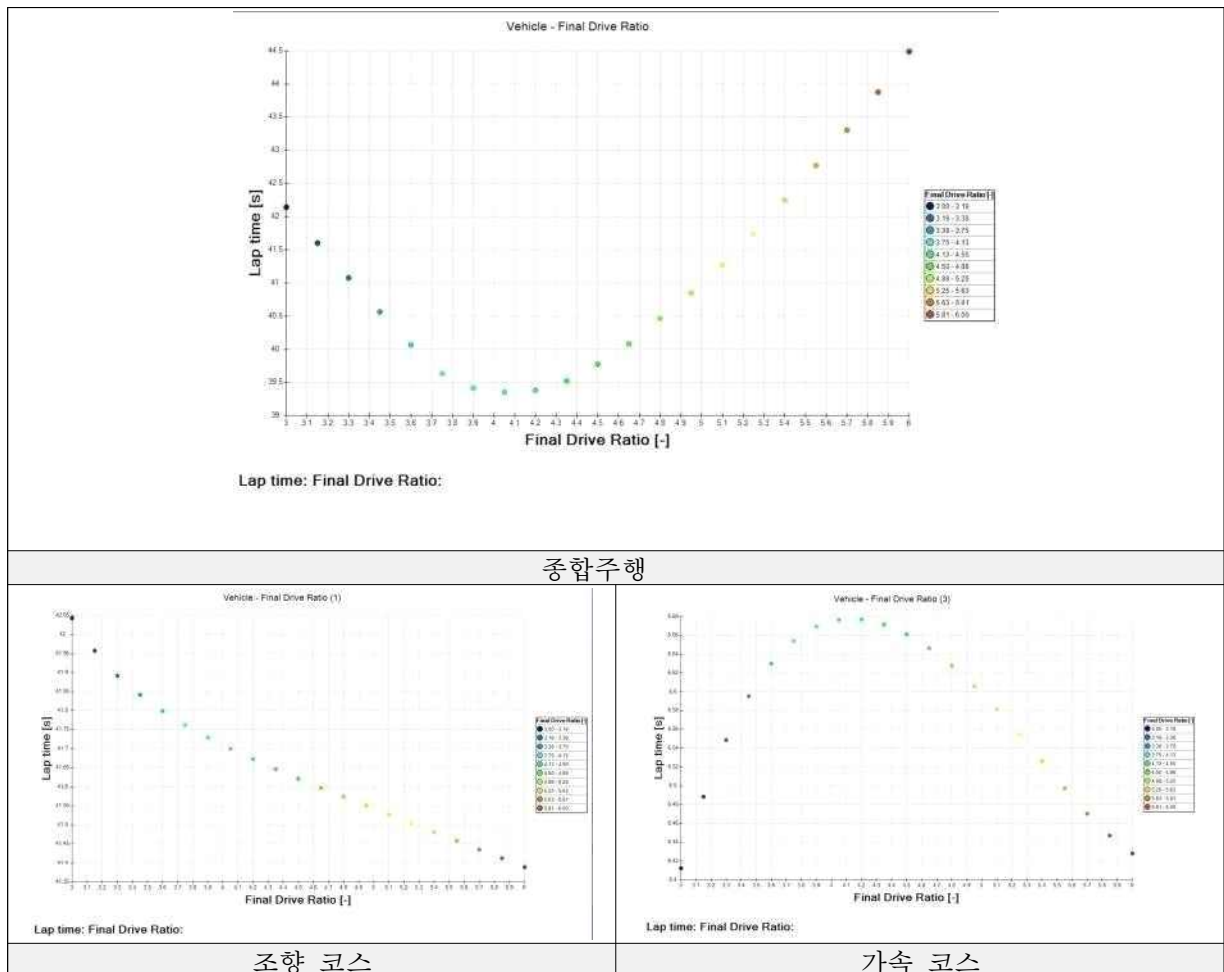
이후 3.5~4.7 사이의 종감속비에 대해 대회 경기별 배점(가속성능 200점, 조향성능 300점, 종합주행성능 500점)을 반영하여 시뮬레이션 가중치를 설정하였다. 특히, 총점 1000점 중 가장 높은 배점인 종합주행성능에 가장 높은 가중치를 부여하고, 가속성능에 가장 낮은 가중치를 두어 전체적인 성능 균형을 최적화하고자 하였다.

종감속비	종합주행	조향	가속	합계
3.5	40.4	41.83	6.61	27135.5
3.6	40.007	41.8	6.63	26936.5
3.7	39.75	41.77	6.65	26620.5
3.8	39.54	41.75	6.66	26698.5
3.9	39.42	41.73	6.67	26636.5
4.0	39.36	41.71	6.66	26602.5
4.1	39.35	41.69	6.68	26596.5
4.2	39.38	41.67	6.68	26608.5
4.3	39.46	41.66	6.67	26646
4.4	39.6	41.64	6.67	26713
4.5	39.77	41.62	6.66	26794
4.6	39.97	41.61	6.65	26891.5
4.7	40.21	41.59	6.64	27007.5

원가 절감을 목적으로 전년도와 동일한 대기어(60t)를 유지하고, 소기어의 잇수를 조절하여 목표 종합속비를 만족시키는 방향으로 최적화를 진행하였다.

소기어 잇수	종합속비
12	5
13	4.62
14	4.29
15	4
16	3.75

이 중, 12, 14, 15, 16t이면, 톱니 수가 짝수이거나 대기어의 잇수의 약수이므로 스포로킷의 같은 톱니와 반복적으로 접촉하므로 특정 부분이 더 빨리 마모되는 문제가 발생한다. 이 문제점을 해결하기 위해 13t를 선정하여 스포로킷과 체인의 마모를 최소화하여 내구성을 향상했다. 계산 및 시뮬레이션 결과 종합속비 4.6은 가속성능, 조향성능, 종합주행성능 등 주요 주행 조건에서의 성능 균형과 차량의 전체적인 내구성 측면에서 적절한 값으로 판단되었다.



2.4.2. 체인 가드

본 차량은 고속 회전하는 구동계 요소인 체인 및 스프로킷의 이물질 접촉 및 파편 비산을 방지하고, 드라이버의 안전을 확보하기 위해 체인 가드를 자체 설계 및 제작하였다.

체인 가드는 체인과 보호판 간 거리를 20mm 이내로 유지하여 규정을 충족시켰다.

또한, 기어 중심 간 거리를 200mm로 설정하였을 때, 대기어와 소기어가 체인과 맞물리는 각도는 각각 236도, 123도로 측정되었으며, 이를 통해 스프로킷과 체인이 120° 이상 맞물리는 영역을 완전히 덮도록 설계하였다. 설계된 체인 가드는 주행 중 체인 이탈 및 이물질 유입을 효과적으로 차단하여 실제 테스트에서의 안정성 확보는 물론, 향후 내구 주행에서도 지속적인 보호 성능을 발휘할 것으로 기대된다.

전년도 차량에는 428 규격의 일반 체인을 사용하였으나, 고속 회전 및 고토크 운용 조건에서의 내구성 확보와 체결 신뢰성 향상을 위해 금년도 차량에는 Heavy-duty 사양인 428H 체인으로 변경하여 적용하였다.

428H 체인은 동일한 피치(12.7mm)와 롤러 치수를 유지하면서도, 측판(Plate)의 두께가 증가된 구조로 설계되어 인장 강도와 피로 수명이 향상되었으며, 고출력 BLDC 모터의 출력 조건에서도 체인 이탈 및 장력 불안정 현상 없이 안정적인 동력 전달 성능을 확보할 수 있었다. 그 결과 추가적인 중량 증가에도 불구하고 신뢰성과 내구성 향상 측면에서 실질적인 이점을 확보한 것으로 판단된다.

The image shows a technical drawing of a DID 428H roller chain. The top view is a side elevation showing the chain's profile with dimensions: G (overall height), g (height to pin center), d (pin diameter), F (roller width), T (plate thickness), W (roller width), and E (height to roller center). The bottom view is a front elevation showing the chain's link structure with dimensions: h (link height), HJ (pin end), P (pitch), and RJ (roller end).

Dimensions

Unit (mm)

Chain No.	Connecting link	Pitch	Roller link width	Roller dia.	Pin					Plate				Avg. tensile strength		Max. allowable load		Approx. weight (kg/m)	
		P	W	D	d	E	F	f	G	g	T	t	H	h	kN	kgf	kN	kgf	
DID 428H	RJ	12.70	7.94	8.50	4.51	18.9	20.1	10.88			2.0	2.0	12.0	10.4	23.3	2,380	4.90	500	0.76

25년도 차량 사용 체인: 428

25년도 차량 사용 체인: 428

2.4.3. 스프로킷

i. 대기어



대기어의 경우 inventor의 웨이프 생성 기능을 사용하여 토크에 따른 하중을 고려하여 형상을 만들고 ansys의 정적해석을 수행하여 안전성을 검토하였다.

ii. 소기어

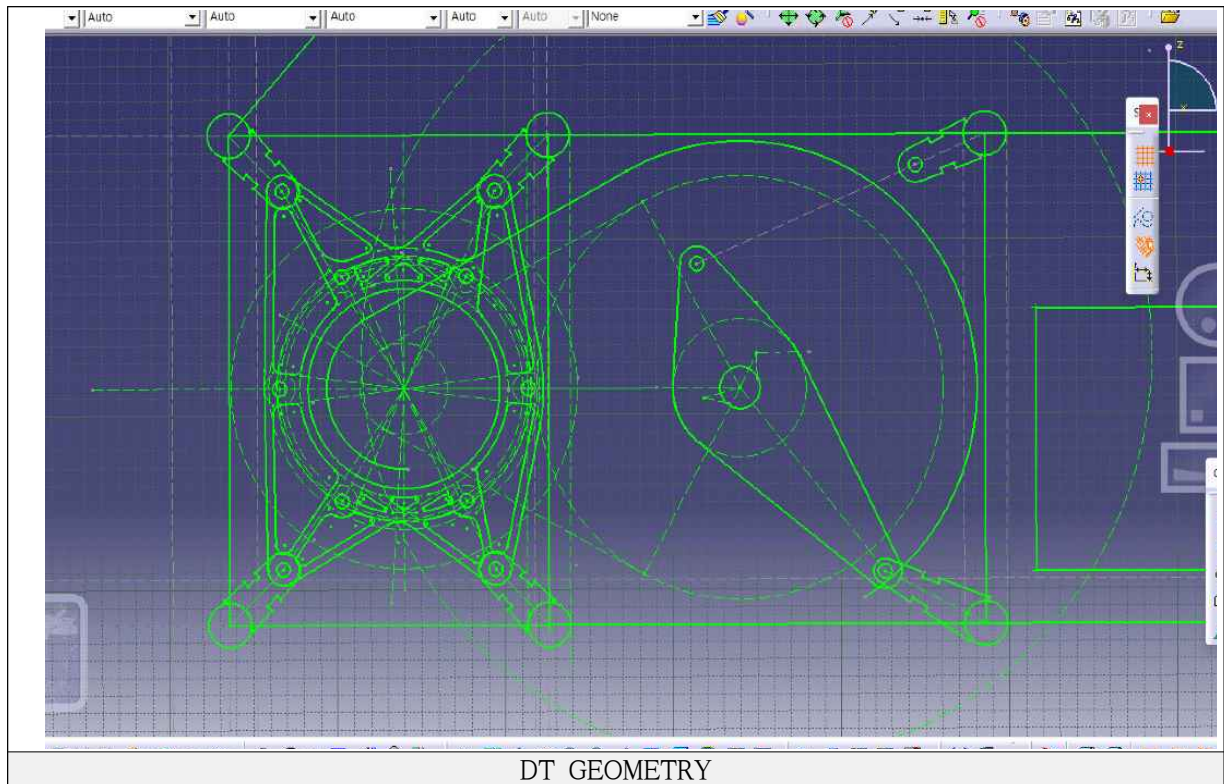


설계 초기 단계에서부터 체결 안정성, 체인 라인 정렬, 가공성 등을 종합적으로 고려한 결과, 중심 허브가 돌출된 B타입 기어 형식을 채택하였다. B타입 기어는 세트 스크류 및 키홈 삽입 공간이 충분하여 체결 구조물의 배치가 용이하며, 특히 본 차량에서는 6mm 키홈과 120° 간격으로 배치된 M6 세트 스크류 3점을 병행 적용함으로써 회전력 전달 시의 고정 강성을 확보하였다.

이러한 구조는 차량의 공간적 제약과 고속 회전 환경, 체결 신뢰성 요구를 모두 만족시키며, 실차 조립 및 주행 테스트 결과에서도 진동, 이탈, 간섭 없이 안정적인 구동 성능을 달성하였다. 설계된 소기어는 체인과의 간섭 없이 안정적인 동력 전달이 가능함이 기대된다. 본 형상은 내구 주행 환경에서도 충분한 내구성과 신뢰성을 갖춘 구성으로 판단된다.

iii. 마운트

차량의 전체 지오메트리를 기반으로, 모터의 중심 위치 및 차체 프레임과의 위치를 확인하여 브라켓 체결부와 모터 체결부의 위치를 결정하고 마운트의 기초 형상을 결정하였다. 모터의 안전성을 위해 마운트를 프레임에 4점으로 체결하였다.



해석의 신뢰성을 위해 2025 대학생 스마트 e-모빌리티 대회를 기준으로 차량이 급가속하는 극한 상황을 가정하였으며, 이때 작용하는 체인장력과 관성력을 계산하였다. 체인 장력과 관성을 구하는 식은 다음과 같이 산정하였다.

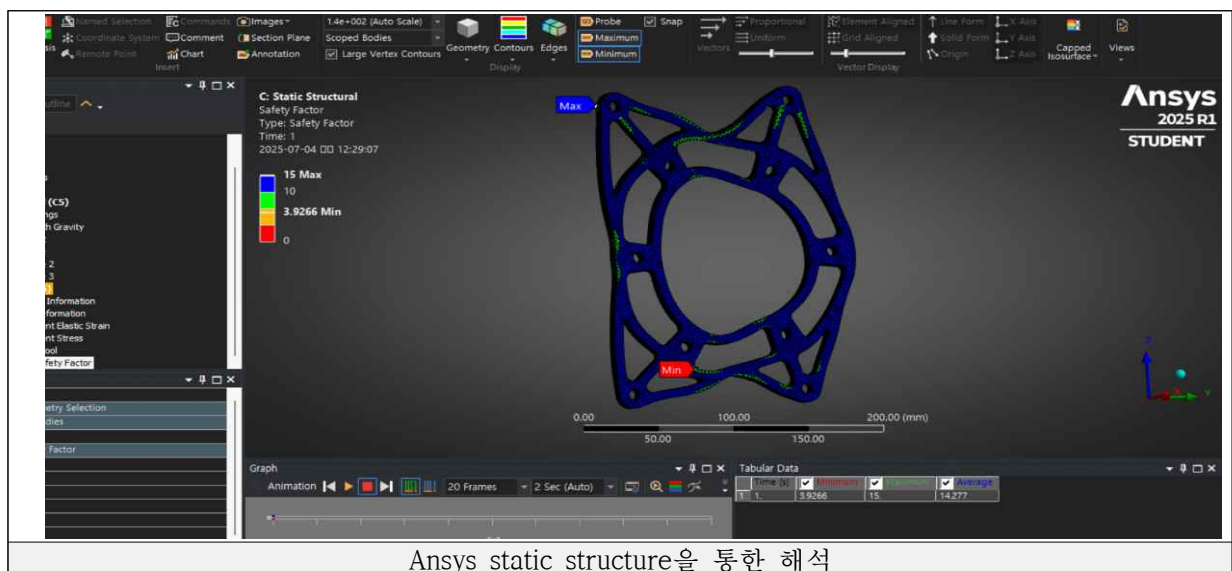
$$T = \frac{\tau}{r} \quad (\tau: \text{모터 } STALL \text{ 토크}, r: \text{작은 스프로킷 반경})$$

$$F_i = ma \quad (m: \text{모터 무게}, a: \text{가속시 최대 가속도})$$

	합력	각도	x방향 힘	y방향 힘	z방향 힘
모터하중	166.77	-90	0	0	-166.77
체인장력	3863.636	-17.61	0	3682.623	-1168.75
관성력	187	180	0	-187	0

Ansys static structure를 통해 해석을 진행했고, 하중 조건은 실제 주행에서 발생 가능한 최대치로 설정하여 보수적으로 접근하였다.

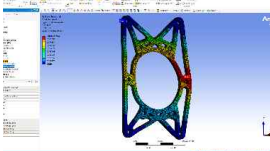
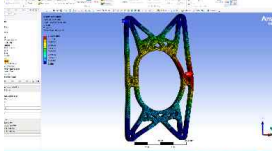
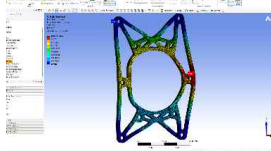
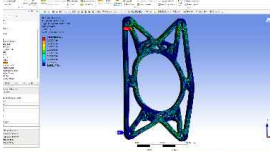
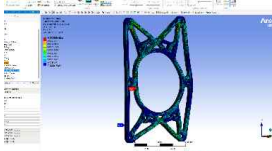
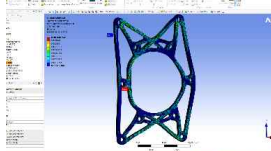
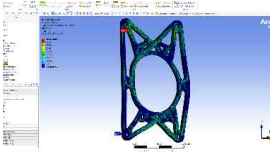
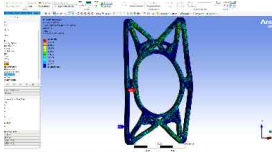
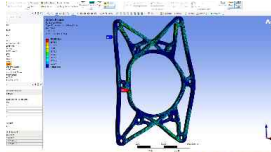
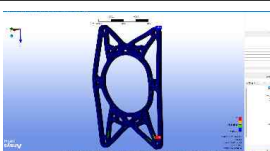
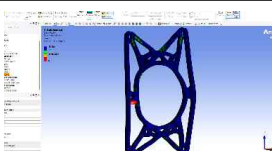
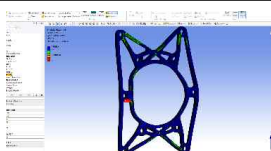
해석 조건으로는 최소 안전 계수는 3.0 이상으로 설정했고, 마운트 두께는 8mm로 고정하였으며 Rib의 두께는 7mm로 하였다. 해석 결과, 마운트의 상부와 하부 일부 영역에서 응력이 집중된다는 것이 확인되었으며, 이에 트러스 구조를 추가하고 응력 분산을 위해 필렛을 적용하여 안전성을 확보하였다. 또한 응력이 집중되지 않은 일부 Rib를 제거하고 형상을 수정하여 재료 절감을 통해 경량화를 시도하였으며, 그 결과 안전계수가 3.9에서 5.2로 크게 향상되었으나 무게는 275.65g에서 281.75g으로 소폭 증가하였다.



Ansys static structure을 통한 해석

이에 따라 모터 마운트의 Rib을 7mm에서 6mm, 5mm로 줄여가며 3가지 모델에 대한 해석을 진행하여 비교하였다. 6mm, 5mm 모델 모두 최소 안전계수를 충족하였고, 형상 수정의 목표인 재료절감과 경량화에 가장 충족하는 5mm 모델을 채택하였다. 특히 이는 24년도 모터 마운트 대비 약 13% 경량화된 수치이며, 최소 안전계수는 다소 감소하였지만 실제 하중 조건 하에서 충분한 강성을 확보한 것으로 판단된다. 기존 마운트 대비 구조적 안정성과 경량화 측면에서 효율적인 최적화 결과를 도출하여 설계 기간 단축과 구조 신뢰성 확보에 유리하다고 판단하였다.

	24년도	25년도	증가율/감소율
Weight	243	211.45	12.98%
Minimum Safety Factor	6.4	3.249	49.23%

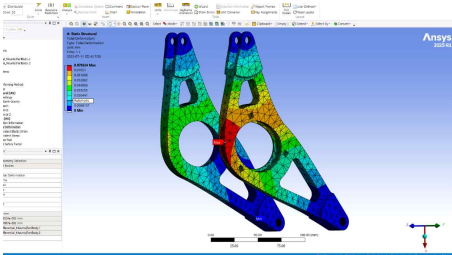
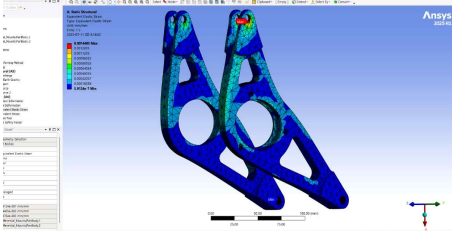
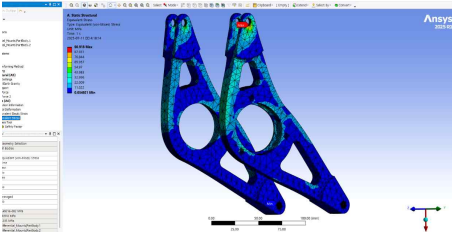
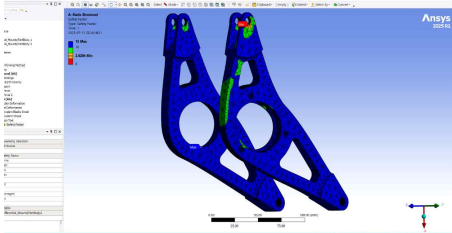
	7mm	6mm	5mm
Weight	281.75	246.88	211.45
Maximum Total Deformation [mm]			
	0.098782	0.12042	0.15311
Maximum Equivalent Elastic Strain [mm/mm]			
	0.00071076	0.00083631	0.0011416
Maximum Equivalent Stress [MPa]			
	48.957	57.62	78.667
Minimum Safety Factor			
	5.2944	4.4985	3.2949

이번 디퍼렌셜 마운트에는 체인 장력 조절을 위해 턴버클 텐서너를 체결하였다. 턴버클은 디퍼렌셜 마운트 상부와 프레임 사이 간격을 조절해 마운트의 전후 위치를 미세하게 이동시킬 수 있으며, 이를 통해 체인 장력을 정밀하게 설정하고, 체인의 수명 연장과 구동 효율 유지에 기여한다. 또한 초기 조립 시간을 단축하고, 세부 세팅 및 유지보수 과정의 편의성과 효율 또한 개선된다.

하중 조건은 차량 급가속 상황을 기준으로 설정하였다. 작용하는 체인 장력은 위의 식을 적용하여 계산하고 작용점은 가장 큰 힘을 받는 대기어 첫번째 이빨의 위치를 계산하여 해석을 진행하였다. 이때 원점은 좌측 디퍼렌셜 마운트의 내측 베어링 홀의 중심점이다.

	합력	위치		힘	
체인장력	3863.636	x	-3.997	x	-3682.623
		y	64.7	y	0
		z	96.688	z	-1168.75
관성력	27.5	x	0	x	-27.5
		y	64.7	y	0
		z	0	z	0

해석 조건으로는 최소 안전계수를 2.0 이상으로 설정하였고, 해석 결과 최소 안전 계수 2.6204를 확보하였다. 좌우 마운트의 무게는 각각 290.77g으로 동일하게 설계되 었으며, 좌우 균형 및 조립 편의성 측면에서도 적절한 결과를 도출하였다. 전체적으 로 주어진 하중 조건에서 충분한 구조적 강성을 확보하였으며, 기존 부품을 현 차량 에 적절히 적용함으로써 기능적 요구를 만족하는 결과를 확인할 수 있었다.

Weight	290.77g
Maximum Total Deformation [mm]	 0.079324
Maximum Equivalent Elastic Strain [mm/mm]	 0.0014405
Maximum Equivalent Stress [MPa]	 98.918
Minimum Safety Factor	 2.6204

2.5. 타이어

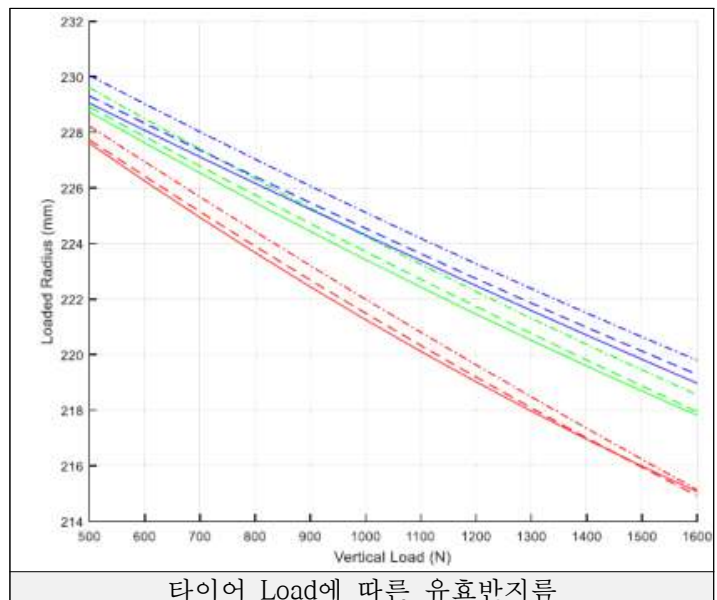
타이어는 Continental(現 Hoosier)사의 205/470 R13 wet 타이어를 사용하였다.

타이어 선정 과정에서 가장 중요하게 고려한 점은 대회 규정의 준수와 자율주행 환경에서의 안정성이었다. 대회 규정 2.3절 '설계요구 조건' 사 항에 따라 슬릭 타이어의 사용은 금지되어 있으므로, 예측 불가능한 기상 변화나 노면 상태에 관계없이 일관된 주행 성능을 확보하여 자율주행 시스템의 안정성을 높이하고자 하였다. 본 대회의 주행 환경은 최고 속도보다 저속(약 20~40km/h)에서의 정밀한 경로 추종이 중요하다. 따라서 타이어의 한계 그립 성능보다는 예측 가능한 조향 반응성과 일관된 거동 특성을 보이는 것이 유리하다고 판단하였으며, 해당 Wett 타이어가 이러한 요구 조건에 부합하다고 판단하였다.

Tire size	Item no.	Tread width in mm **	Diameter in mm **	Recom. Rim in inch	Measured rim in inch	Total width on measured rim in mm**	Compounds	Price in €* (excl. VAT)	Price in €* (incl. 19% VAT)
16.0 x 7.0-10	44070	178	411	6-7	6,0	193	W3	263,87	314,00
18.0 x 6.0-10	44115	157	452	6-8	6,0	198	W3	268,91	320,00
20.0 x 6.5-13	44140	155	521	6-8	6,0	184	W3	323,53	385,00
20.0 x 7.5-13	44150	188	523	7-8	7,0	211	W3	330,25	393,00
20.0 x 9.0-13	44180	248	518	9-10	9,0	271	W3	351,26	418,00
21.0 x 6.5-13	44185	170	538	6-8	6,0	183	W3	323,53	385,00
22.0 x 8.0-13	44196	198	559	7,5-8,5	8,0	226	W3	361,34	430,00
22.0 x 9.0-13	44217	244	574	8,5-10	9,0	269	W3	372,27	443,00
20.5 x 7.0 R 13	44442	183	528	6-8	6,0	198	W3	318,49	379,00
22.5 x 8.0 R 13	44443	n.n.	n.n.	n.n.	n.n.	n.n.	W3	369,75	440,00
185 / 60 R 13	44421	165	551	5-7	5,5	191	W3 ²⁾	272,27	324,00
185 / 60 R 13	46100	173	546	5-7	6,0	196	W2 ²⁾	272,27	324,00
200 / 540 R 13	44330 44335	188	541	8-9	9,0	239	W3 HC W3	333,61	397,00
205 / 470 R13	44345	183	478	7-8	7,0	197	FSAE WET	313,45	373,00

타이어 제조사 카탈로그(Item no. 44345)

타이어 단면폭은 약 197mm, 트레드 폭은 약 183mm이며 새 타이어 기준 무게는 약 4.2kg이다. 냉간 공기압은 12~13psi에서 시작하여 주행 중 약 18psi 근처의 열간 공기압을 목표로 설정한다. 차량 총중량 약 200~300kg에 최적화되어있으며 정적 하중이 각 타이어당 250~400lb 수준에서 충분한 강도를 가진다.



각 타이어가 받는 지면 반력을 계산하기 위해 각 바퀴를 저울에 올려 차량의 무게 중심과 타이어가 받는 힘을 계산하였다. 특히 운전자가 탑승한 상태에서 후륜 양 타이어의 유효반경은 최종 감속비, 모터 rpm과 곱해져 차량의 속도를 추산하는 데 사용되는 중요한 파라미터였고, 실측한 하중 데이터를 바탕으로 차량의 정적 상태와 유사한 조건(공기압 120kPa, 캠버 0도)의 데이터를 기준으로 분석을 진행하였다. 후륜의 각 바퀴에는 운전자가 탑승하였을 때 약 110kgf 이 측정되었고, 타이어의 외경 470mm가 아닌 실제 작동시 450mm 유효지름을 속도 계산식에 삽입하였다.

2.6. 냉각 계통

냉각의 주요 과제는 모터 및 컨트롤러의 열관리를 최적화하여, 시스템의 신뢰성과 성능을 극대화하는 것이다.

전압과 전류는 각각 50V와 200A로 제어하여 사용하였다. 50V의 저전압 제어를 통해 감전 위험을 낮추고, 200A의 고전류 운용으로 높은 출력과 안정적인 구동 성능을 확보하였다. 전력 손실로 발생하는 열에너지는 시스템에 영향을 줌으로, 모든 방출 열을 냉각하는 것을 목표로 하였다.

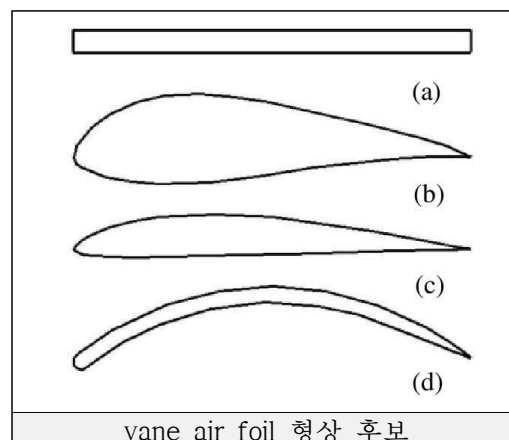
구분	전압(V)	전류(A)	효율(%)	발열량(kW)	목표 냉각량(kW)
모터	50	200	91	0.9	0.9
컨트롤러	50	200	90	1	1

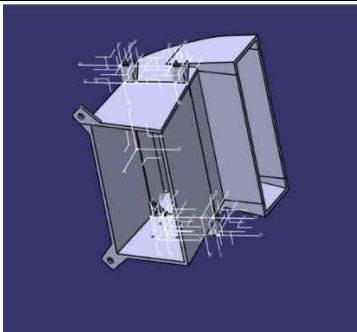
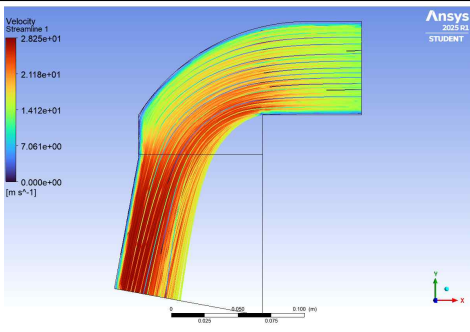
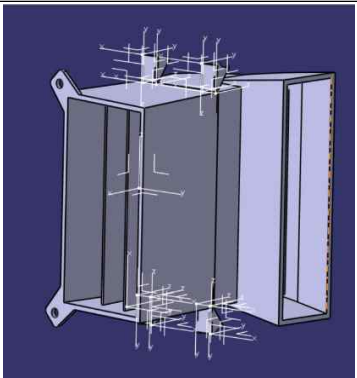
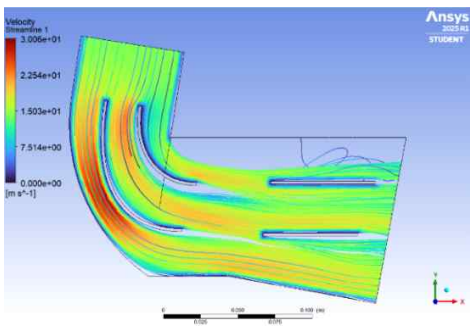
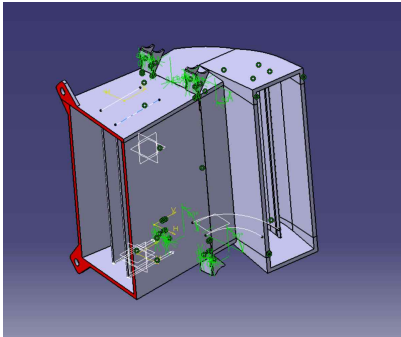
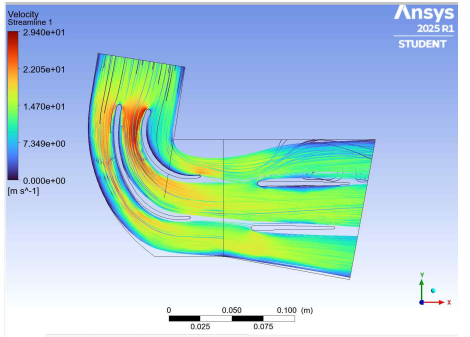
모터는 덕트를 통한 공냉 시스템을 적용하여 냉각하였다. 덕트를 유동 해석한 결과, 곡률의 영향으로 인해 공기 흐름이 외측 벽면으로 집중되어 속도 분포의 불균형 및 유량 분포의 비대칭 현상이 관찰되었다. 특히 곡률 중심에서 벗어난 유선 분포는 후속 시스템의 냉각/송풍 효율 저하 및 소음을 발생시킨다.

유동 불균형을 개선하고, 공기 흐름을 보다 정렬된 방향으로 유도하기 위해 flow guide vane을 곡관부 내에 배치하였다. 이는 중심 이탈을 억제하여 유동의 안정성과 분포의 균일성을 확보한다.

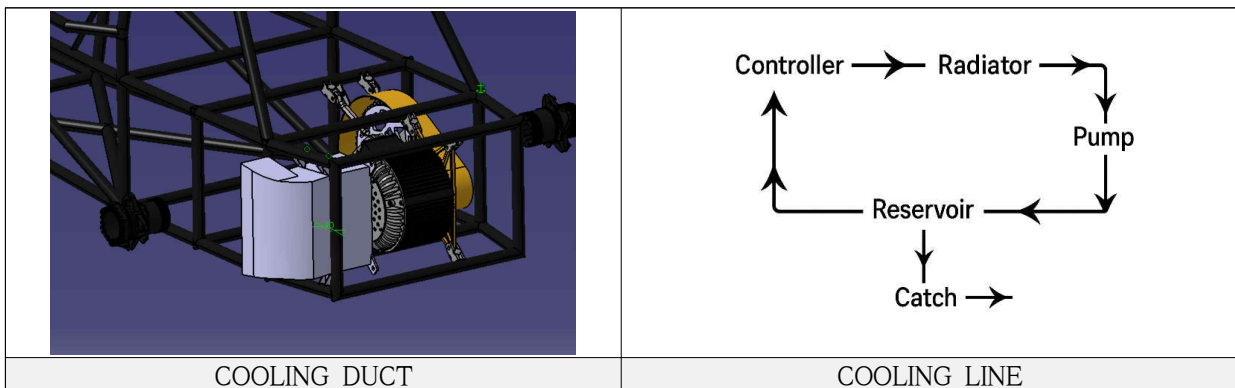
초기에는 단순 직사각형 단면의 베인을 적용하여 유동 분포의 일부 개선을 확인하였으나, 여전히 흐름의 균일성 측면에서 한계가 존재하였다. air foil 중 (d) 형상이 가장 우수한 성능을 보인 하지만, 응력에 의한 파손위험이 존재해 안정성과 내구성을 고려해(c)형상으로 채택하였다.

vane의 형태를 (c)형상의 air foil 형태로 수정하여 분석한 결과 단순 직사각형일 때보다 유량 분포의 비대칭을 완화시켰으며 공기의 속도가 낮아져 vane에 가하는 응력을 줄여 파손의 위험을 감소시켰다.



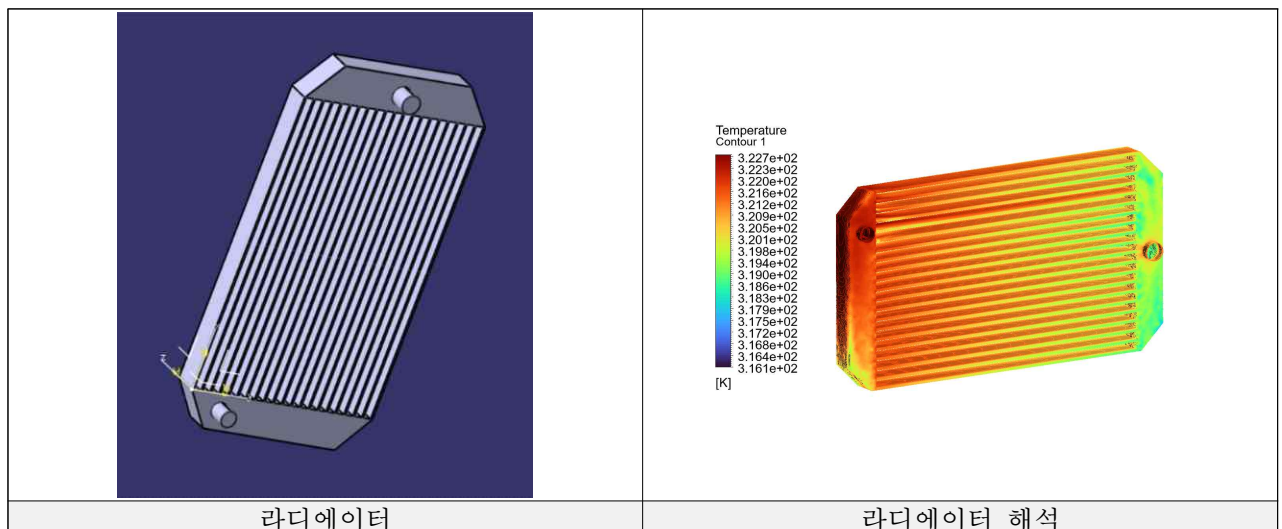
Vane이 없을 때		
Rectangular vane		
Air foil vane		

ANSYS fluent를 이용해 유량을 측정한 결과, inlet에서의 유량은 $0.2352\text{m}^3/\text{s}$ 이고 outlet에서의 유량은 $-0.2352\text{m}^3/\text{s}$ 로 $0.2352\text{m}^3/\text{s}$ 의 유량을 확보해 목표 유량인 $0.2\text{m}^3/\text{s}$ 을 초과 달성하였다. 덕트의 위치는 모터를 효과적으로 냉각할 수 있도록 덕트의 outlet과 모터 사이 거리를 30.68mm 로 정해 모터에 가해지는 유량을 극대화하였다.



컨트롤러는 라디에이터를 이용한 수냉방식으로 냉각했다. 컨트롤러에서 발생한 열이 곧바로 라디에이터로 이동되어 효과적으로 열을 방출할 수 있고, 라디에이터 다음에 펌프가 배치돼 냉각수의 온도가 낮을 때 압송되므로 펌프의 수명 및 효율 측면에서 유리하다. 또한, 한 방향으로 순환하는 간단한 흐름 구조이므로 컨트롤러에서 펌프 제어 및 온도 모니터링이 용이하다.

라디에이터는 ANSYS Fluent를 활용하여 열해석을 수행하였다. 이는 유동 해석을 통해 라디에이터 입구와 출구에서의 냉각수 온도를 정량적으로 평가함으로써, 해당 라디에이터의 열 감소 성능을 검증하고 냉각 효과를 분석한다. 라디에이터 열해석을 수행한 결과, 목표 온도 강하량인 3° C보다 큰 3.3° C의 온도 강하가 일어났으며 라디에이터 코어에도 균일한 온도 강하가 일어나는 것을 입증했다.



2.7. 인클로저

자율주행 시스템을 구성하는 각종 전자 장비는 주행 중 발생하는 진동, 분진, 수분 등 외부 환경으로부터 반드시 보호되어야 한다. 또한, 고전압 부품과 운전자를 물리적으로 격리하여 안전을 확보하는 것이 매우 중요하다. 본 차량의 인클로저 시스템은 이러한 안전성, 신뢰성, 정비 용이성을 목표로 설계하였으며, 크게 차량 후방의 '메인 컨트롤 박스'와 운전자 전방의 '대시보드 패널'로 구성된다.

2.7.1. 메인 컨트롤 박스

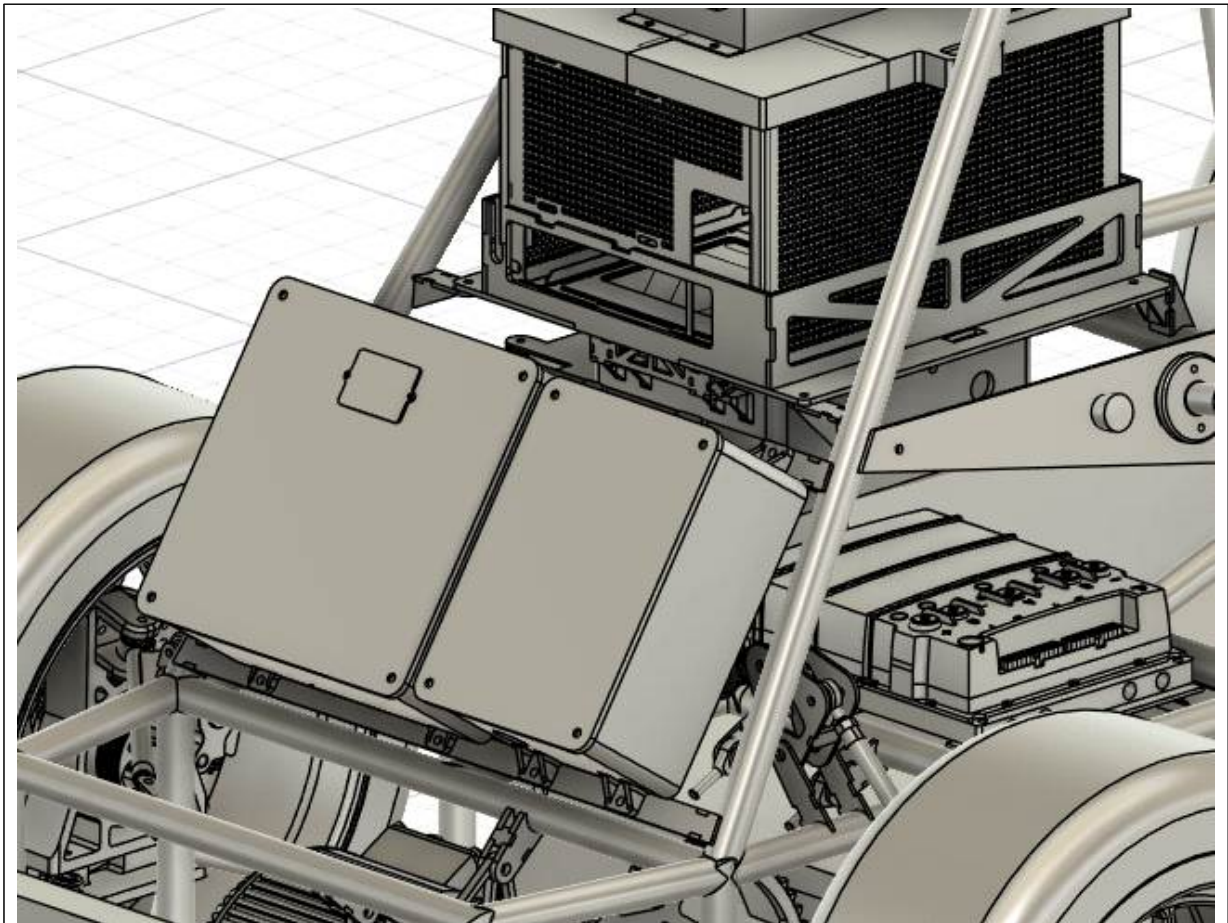
메인 컨트롤 박스는 자율주행의 두뇌 역할을 하는 고성능 컴퓨터(PC)와 모터 컨트롤러, 각종 센서 인터페이스 보드, 전원 분배 모듈 등을 통합하여 수납하는 핵심 인클로저이다. 본 팀은 다음과 같은 목표로 설계를 진행하였다.

첫째, 보호 성능을 강화하였다. 대회 규정 2.3.마의 방수 및 방진 요구사항을 충족하기 위해 IP 등급을 고려하여 설계하였으며, 외부 충격으로부터 내부 부품을 보호할 수 있도록 충분한 강성을 확보하였다.

둘째, 효율적인 열 관리를 구현하였다. 고성능 컴퓨터와 컨트롤러에서 발생하는 열을 효과적으로 배출하기 위해, 인클로저 상단과 측면에 공기 순환을 위한 통풍구를 배치하고 내부에는 냉각 팬을 설치하여 강제 공랭 방식을 채택하였다.

셋째, 정비 편의성을 확보하였다. 대회 현장에서 발생할 수 있는 문제에 신속하게 대응하기 위해, 최소한의 볼트 체결로 상부 커버를 쉽게 개방할 수 있도록 설계하여 내부 부품에 대한 접근성을 높였다.

넷째, 내진동 설계를 적용하였다. 차량 주행 시 발생하는 진동이 민감한 전자 장비에 미치는 영향을 최소화하기 위해, 프레임과 체결되는 부분에 방진고무를 적용하고 내부 부품들을 견고하게 고정하였다.



메인 컨트롤 박스 3D 모델. 주요 전자 장치의 집중 배치와 방열 구조

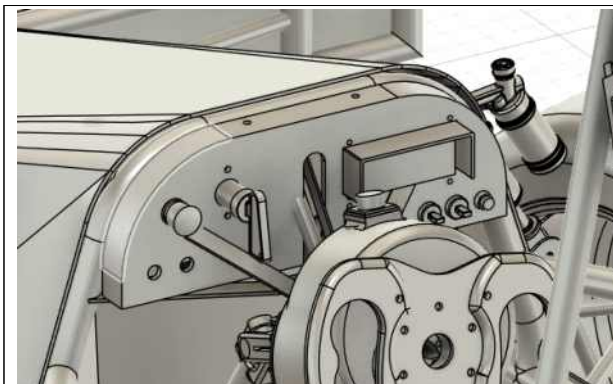
2.7.2. 대시보드 패널

대시보드 패널은 운전자가 차량의 상태를 직관적으로 파악하고, 수동/자율 모드를 안전하게 전환할 수 있도록 설계된 운전자 인터페이스의 핵심이다.

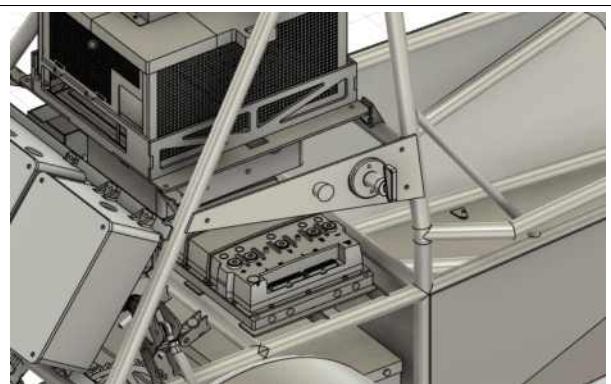
첫째, 인체공학적으로 설계하였다. 운전자가 스티어링 휠을 잡은 상태에서도 주요 스위치에 쉽게 접근할 수 있도록 배치하였으며, 주행 중 시선 분산을 최소화하도록 시인성을 고려하여 디자인하였다.

둘째, 규정을 준수하였다. 대회 규정 2.18.1. 나에 명시된 자율주행 시스템 마스터 스위치(ASMS)를 명확하게 식별하고 조작할 수 있도록 별도의 공간에 배치하였으며, 킬스위치, E-stop 스위치를 포함하여 설계하였다. 후측면에도 동일하게 킬스위치와 E-stop 스위치가 구현되어 있다.

셋째, 시스템 상태 표시 기능을 마련하였다. 차량의 전원 상태, 자율주행 시스템의 활성화 여부 등을 운전자가 즉각적으로 인지할 수 있도록 LED 인디케이터를 장착할 공간을 확보하였다.



운전자 인터페이스를 위한 대시보드 패널.
ASMS, 킬스위치, E-stop 등 배치



리어 측면 스위치 구성. E-stop + 킬스위치

2.8. 제동장치

차량의 제동 성능은 안전과 직결되는 핵심 요소이다. 본 차량은 4륜 모두에 제동력을 배분하는 4륜 제동 방식을 채택하여 어떠한 주행 상황에서도 안정적인 감속이 가능하도록 설계하였다. 고정 캘리퍼 방식을 적용하여 페달 응답성을 높이고 일관된 제동력을 확보하였으며, 이는 운전자가 차량을 정밀하게 제어하는 데 이바지한다.

2.9. 파워트레인

자율주행 차량의 전기 구동 시스템은 크게 구동 모터와 배터리에서 나온 직류를 모터에 맞게 3상 AC 전류로 바꾸어줄 뿐만 아니라 여러 통신 기능 및 모터 제어의 기능을 수행하는 모터 컨트롤러로 구성된다. 아래는 구동 모터와 모터 컨트롤러의 사양 및 선택 이유이다.

2.9.1. 구동 모터

차량의 구동 모터 선택에 있어서 속도 제어의 정밀도, 에너지 효율, 내구성 등이 중요한 요소들로 작용한다. 여러 요소를 표로 정리하면 다음과 같다.

항목	24년도 PMDC 모터	25년도 BLDC 모터
브러시유무	O (브러시 있음)	X (브러시 없음)
속도 제어 방식	전압 제어	전자 회로(PWM 및 벡터 제어)
효율	브러시 마찰로 인해 손실 발생	고효율 (브러시 없음)
내구성	브러시 마모로 수명이 짧음	내구성 높음
소음 및 유지보수	브러시 접촉 소음 발생	저소음, 유지보수 필요 없음
출력 밀도	상대적으로 낮음	더 높은 출력 밀도 제공

<모터 비교>

i. 내구성 및 유지보수

PMDC 모터는 브러시와 정류자가 있어 시간이 지남에 따라 마모가 발생하고, 이에 따른 정기적인 유지보수가 필요하다. 반면, BLDC 모터는 브러시가 없어 마찰에 의한 손실이 적으며, 긴 수명을 제공하여 자율주행 차량의 지속적인 운영에 유리하다.

ii. 효율과 출력 밀도

BLDC 모터는 브러시 손실이 없어 PMDC보다 높은 효율을 가지며, 같은 크기에서 더 높은 출력 밀도를 제공한다. 이는 전기 자율주행 차량의 주행 가능 거리 증가와 배터리 효율 극대화에 기여한다.

iii. 속도 및 토크 제어

PMDC 모터는 단순한 전압 조절을 통해 속도를 제어하지만, 고속에서 효율이 떨어지고 브러시 소손 문제가 발생할 수 있다. 반면, BLDC 모터는 PWM(Pulse Width Modulation) 제어 및 벡터 제어를 활용하여 더 정밀한 속도 및 토크 제어가 가능하다. 이는 다양한 주행 환경에서 안정적인 성능을 보장하는 데 유리하다.

높은 에너지 효율과 정밀한 제어 기능 및 낮은 유지보수 비용 등의 이유를 바탕으로 BLDC 모터가 차량에 더 적합하다고 판단하였다. 위의 내용을 바탕으로 최종 선택한 모터는 Golden motor 사의 HPM-10kw이다.



Model	HPM-10kw motor
Voltage(V)	48
Rated current(A)	245
Rated Power(kW)	10
Rated Torque(Nm)	26
Rated RPM	3700
Weight(kg)	17

< 모터 사양 >

2.9.2. 모터 컨트롤러

작년 사용한 모터 컨트롤러는 Kelly 사의 HPM 72601 controller이다. 다음과 같은 기준으로 비교하여 모터 컨트롤러를 재선정하였다.

i. 통신 방식 및 네트워크 호환성

자율주행 차량에서 모터 컨트롤러는 단순한 모터 제어 장치가 아니라, 차량 전체 시스템과 긴밀하게 연결되어 실시간으로 데이터를 주고받는 핵심 구성 요소다. 따라서 통신 방식의 신뢰성과 확장성이 중요한 고려 요소로 작용한다.

현재 사용되는 주요 통신 방식으로는 RS-232, CAN, UART, SPI 등이 있지만, 그중 CAN(Controller Area Network) 통신이 차량에 가장 적합하다고 생각했다. 해당 부분에 대한 상세 설명은 3.5. 저수준 제어의 3.5.1. 제어기 부분에서 설명하도록 한다.

ii. 전력 효율 및 지속 운행 시간 최적화

모터 컨트롤러의 효율성은 차량의 전체적인 에너지 관리에 직결된다. 높은 Peak Current(최대 전류)와 Continuous Current(연속 전류)를 지원하는 컨트롤러일수록 모터에서 요구하는 출력을 안정적으로 공급할 수 있으며, 회생 제동 기능이 포함되면 에너지 재활용을 통해 주행 가능 시간을 늘릴 수 있다.

iii. GOLDEN MOTOR 모터 컨트롤러 특징

iii.a. FOC (Field-Oriented Control) 지원

Golden Motor 컨트롤러는 FOC(벡터 제어)를 지원하여, 기존의 정현파 제어(Sine Wave) 방식보다 더욱 정밀한 속도 및 토크 제어가 가능하다. 더 부드러운 가속 및 감속과 저속에서도 높은 효율을 유지하는 것, 에너지 소비를 최소화할 수 있도록 하는 것이다. 이는 자율주행 차량에서 모터의 응답성과 효율을 극대화하는 데 중요한 요소다.

iii.b. 회생제동

Golden Motor EZkontrol C481200은 완전한 회생 제동 기능을 지원하여 감속 시 발생하는 에너지를 배터리에 재충전할 수 있다. 이를 통해 배터리 지속 시간을 연장하고, 전체적인 에너지 효율을 향상할 수 있다.

iii.c. 향상된 통신 및 사용자 편의성

Golden Motor 컨트롤러는 CAN 통신뿐만 아니라 Bluetooth 지원이 가능하며, 스마트폰 앱이나 PC를 통해 실시간으로 모터와 차량 상태를 모니터링하고 설정을 변경할 수 있다.

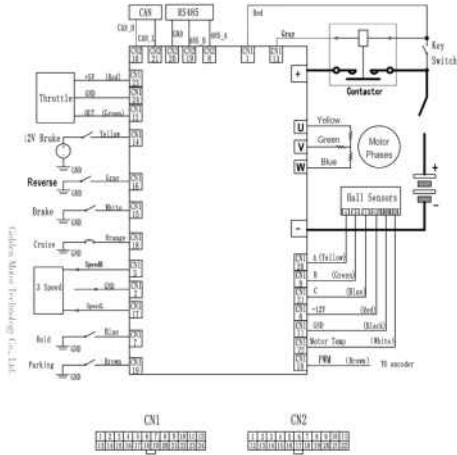
iii.d. 내구성 및 방수 설계

Golden Motor 컨트롤러는 IP등급 방진/방수 설계를 갖추고 있어 외부 환경 변화에도 안정적인 성능을 유지할 수 있다. 반면, Kelly HPM 72601은 일반적인 방진 수준이며, 혹독한 환경에서 성능 저하 가능성이 있다.

iv. 최종 모터 컨트롤러 선정

- CAN 통신 지원→차량내 ECU 및 센서와의 원활한 연동
- FOC(Field-Oriented Control) 적용→더 정밀한 속도 및 토크 제어 가능
- 회생제동기능탑재→배터리 효율을 높여 주행가능 시간 증가
- Peak Current 300A, Continuous Current 200A → 모터 출력 안정성 확보
- Bluetooth 및 PC 소프트웨어 연동 가능→실시간 모니터링 및 설정 편리

위 조건을 충족하는 모터 컨트롤러를 선정하는 것을 목표로 삼았다. 따라서 위의 사항을 만족하는 여러 모터 컨트롤러 중에서, Golden Motor EZkontrol C481200을 최종적으로 선정하였다.

	
EZkontrol C481200	컨트롤러 다이어그램

항목	스펙
Item No.	EZkontrol C481200
Rated Voltage (V)	48
Max. Voltage (V)	72
Max. Phase Current (A)	1200
Rated Phase Current (A)	480
Max. Bus Current (A)	600
Motor Rated Power (KW)	7-12

<컨트롤러 사양>

2.10. 배터리

차량의 배터리는 구동을 위한 48V 고전압(HV) 배터리, 차량의 전장 시스템을 위한 12V 저전압(LV) 배터리로 구성된다.

2.10.1. 고전압 배터리

배터리는 51.8V 78Ah(14S30P) 더블류웍스 전지 모듈을 사용한다.

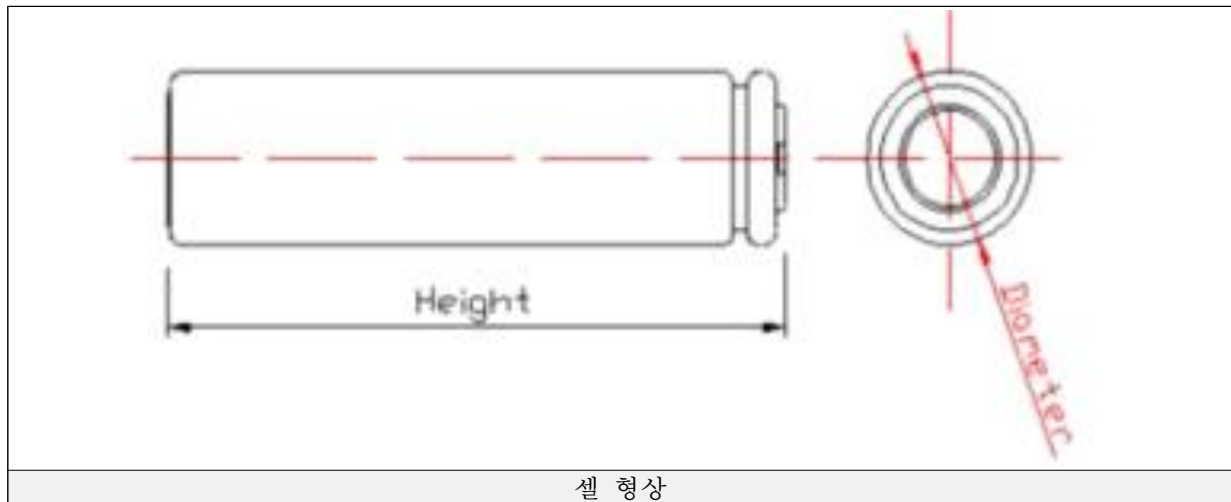


배터리 외관

항목	사양	비고
제품 명칭	51.8V 더블류웍스 배터리팩	14S30P
모델 번호	WWR026PP1430	
전지 구성	30 병렬 14 직렬	INR_18650_2600 CELL 적용
사용 온도	0~45 ℃(충전), -20 ~ 60℃(방전)	
보관 온도	-20 ~ 45℃	
사용 / 보관 습도	10 ~ 85%RH	
외부 연결 단자	M6 Ring terminal	
PACK 관리 기판	PCM 내장	
CELL 간 연결 방법	Ni-Plate by spot welding	
정격 전압	Pack voltage 51.8 VDC	
전력량	3,936 Wh	30P14S 충/방전 중 보호검출 조건을 만족하면 보호기능 절차에 따라 동작함.
정격 용량	78.0Ah	
동작 전압	39.2VDC ~ 58.8VDC	
최대 지속 충전 전류	39 A (0.5C)	
충전 방식	CC-CV 충전	
충전 정지 전압	Pack 전압 58.8[V]	정상 동작 상태
최대 지속 방전 전류	350A/10sec 이하, 300A/30sec 이하 (Continuous = 150A)	
방전 정지 전압	Pack 전압 37.8[V] (cell = 2.7V)	
소비 전류	1 mAh 이하	

<기본 사양>

i. 배터리 셀 사양



셀 형상

Items	Criteria
Nominal Capacity	2.6 Ah
Nominal Voltage	3.7 V
Dimension	Diameter $\leq$ 18.40 mm / Height $\leq$ 65.50 mm
Weight	48 g
Operating Voltage	Continuous 2.75 - 4.2 V
Operating Temperature	Charge 0 ~45 °C
	Discharge -20 ~ 60 °C
Storage Temperature	-20 to 45 °C

<셀 사양>

ii. 보호 기능 사양

보호 기능과 관련한 목록은 아래와 같이 동작한다.

ii.a. 전류 보호

항목	조건	범위	유지시간	비고
과충전 보호 전류	충전	충전전류 100[A] 이상	1[s] 이상	충전기 제거
과방전 보호 전류	방전	방전전류 500[A] 이상	1[s] 이상	Load 차단
short 전류 보호	외부 단락		400[us] 이상	

<전류보호 검출조건>

ii.b. 전압 보호

전압보호 검출 및 해제 조건은 다음과 같다. 하기 표의 전압[V]은 개변 CELL 전압을 나타낸다.

명칭	조건	범위	유지시간	해제
과충전 보호 전압	충전	최대 셀전압 4.25[V] 이상	1[s] 이상	4.15[V] 이하
과방전 보호전압	방전	최저 셀전압 2.70[V] 이하	1[s] 이상	3.00[V] 이상

<전압보호 검출조건>

2.10.2. 저전압 배터리

보조 배터리는 GSP BATTERY 사의 12V 50Ah 리튬 인산철 배터리를 사용한다. 구동을 위한 전력외에 다른 모든 전장품에 전원은 이 보조 배터리로 공급한다.



Items	Criteria
name	LFP 12V 50Ah
Nominal Voltage	12.8V
Dimension	265 * 77 * 170 mm
Weight	5.3 kg
Charging Voltage	14.6V
Operating Temperature	Charge 0 ~45 °C
	Discharge -20 ~ 60 °C
Storage Temperature	-20 to 45 °C
Max Charging Current	50A
Max Discharging Current	50A
Safety Protection	BMS

<LV 사양>

작년에는 LFP 12V 100Ah 배터리를 사용하였으나, 실사용 데이터를 분석한 결과 전력 소모가 예상보다 낮아 올해는 50Ah로 용량을 줄인 배터리를 채택하였다.

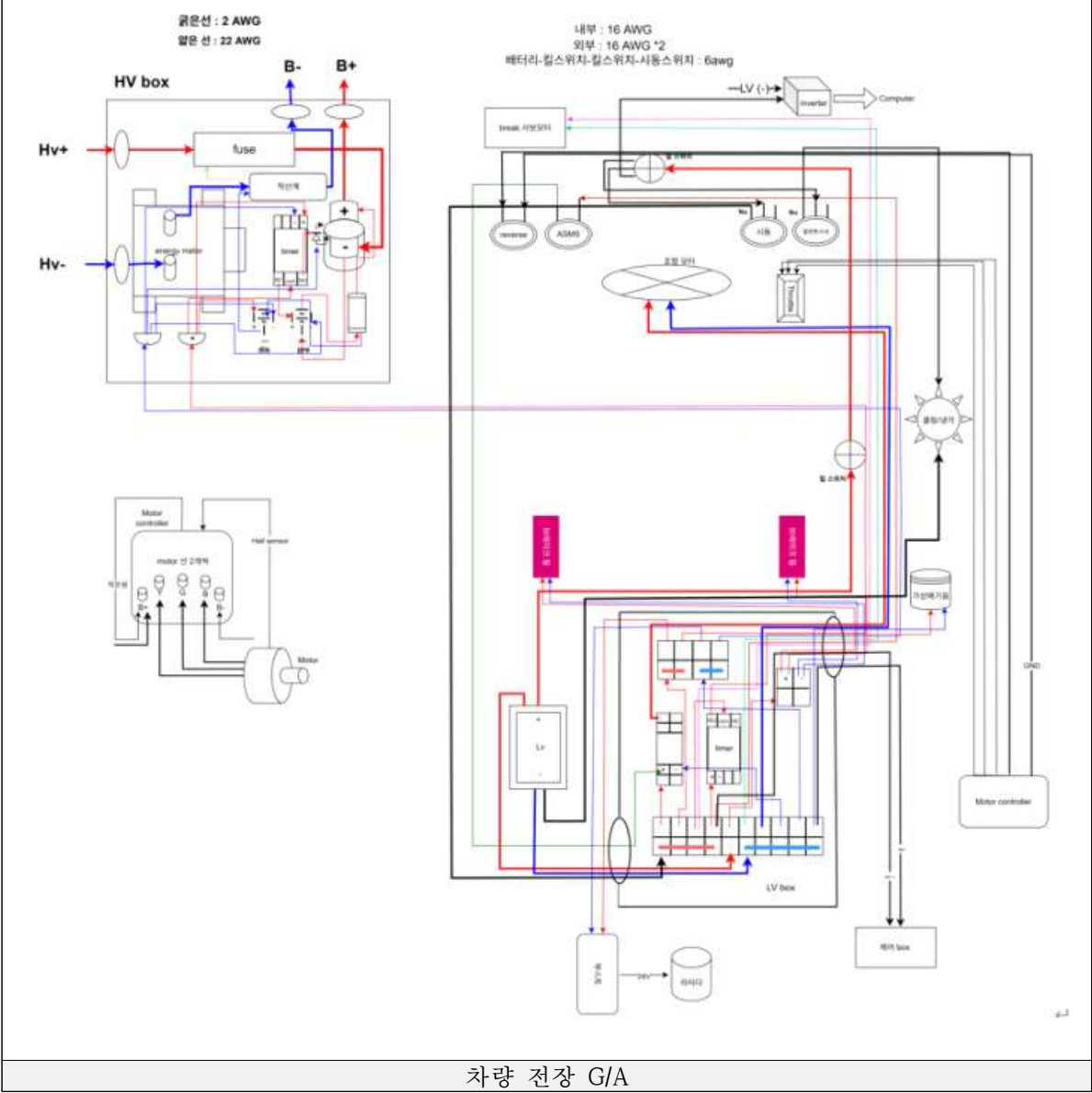
이로써 배터리 무게는 약 7.2kg 감소(-57.6%), 크기는 길이 기준 65mm, 폭 기준 96mm, 높이 기준 46mm 줄어들어 전체 체적이 약 72% 축소되었다.

패키징 유연성 및 차량 경량화 측면에서 이점이 크며 차량 응답성을 대폭 개선할 수 있었다.

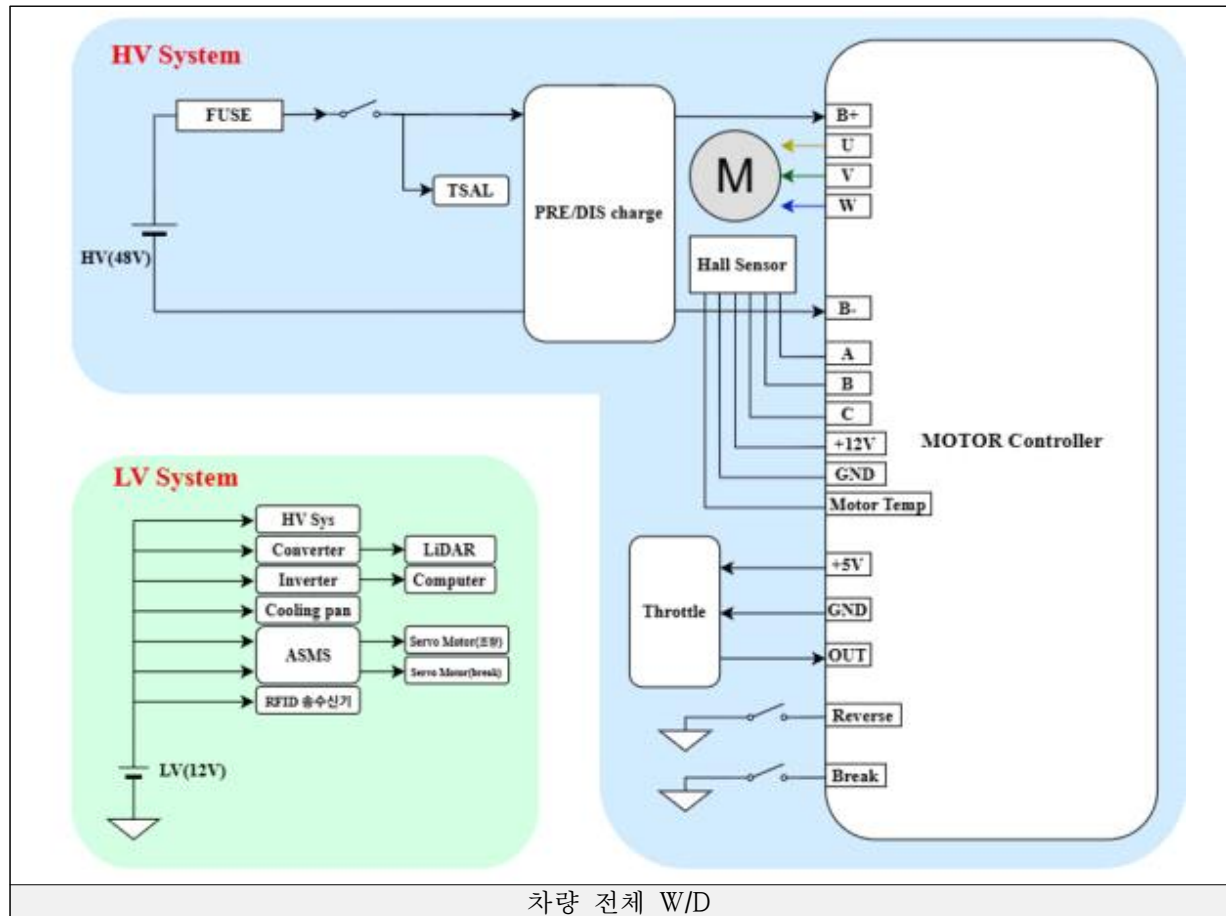
항목	24년도 LV 배터리	25년도 LV 배터리
용량	100Ah	50Ah
전압	12.8V	12.8V
크기	330x173x216 mm	265x77x170 mm
무게	12.5kg	5.3kg

<LV 배터리 비교>

2.11. 차량 전장 시스템



차량 전장 시스템은 차량 구동을 위한 HV 박스와, 배전반의 역할을 하여 전력을 구동계를 제외한 전장품으로 배전해주는 배전반의 역할을 하는 LV박스로 나뉜다.

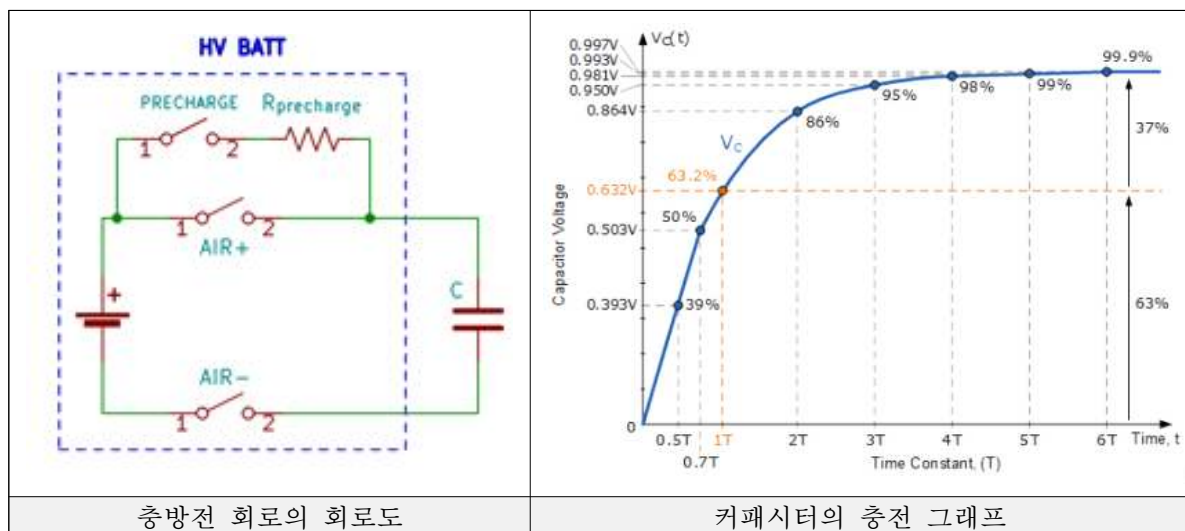


차량 전체 W/D

2.11.1. HV 시스템

i. Pre-Discharge Circuit

초기 충전 시 Motor Controller에 HV를 공급하는 과정에서 갑작스러운 고전압이 인가될 경우 대전류가 흘러 Motor Controller의 capacitor을 포함한 주요 소자들이 이를 견디지 못하고 손상되는 경우가 발생할 수 있다. 이는 방전하는 경우에서도 마찬가지이다. 이에 초기충전-방전 시에는 저항을 연결해 전류의 크기를 제한하여 안정적으로 작동할 수 있도록 한다. 충전이 완료된 이후에는 손실을 방지하기 위해 HV와 Motor Controller 사이에 있던 저항을 제거해준다.



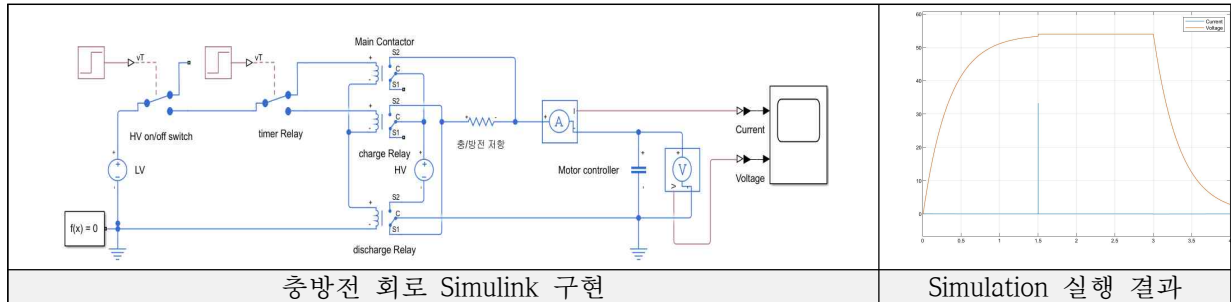
i.a. 초기 충전 시정수

초기 충전을 회로로 생각하면, HV와 연결되어 있는 직렬 RC회로임을 알 수 있다. 스위치를 닫는 순간 Capacitor은 충전되기 시작하고, Motor Controller 내부의 있는 Capacitor에 전류가 흐르기 시작한다. 시간에 따른 전류식은 다음과 같다.

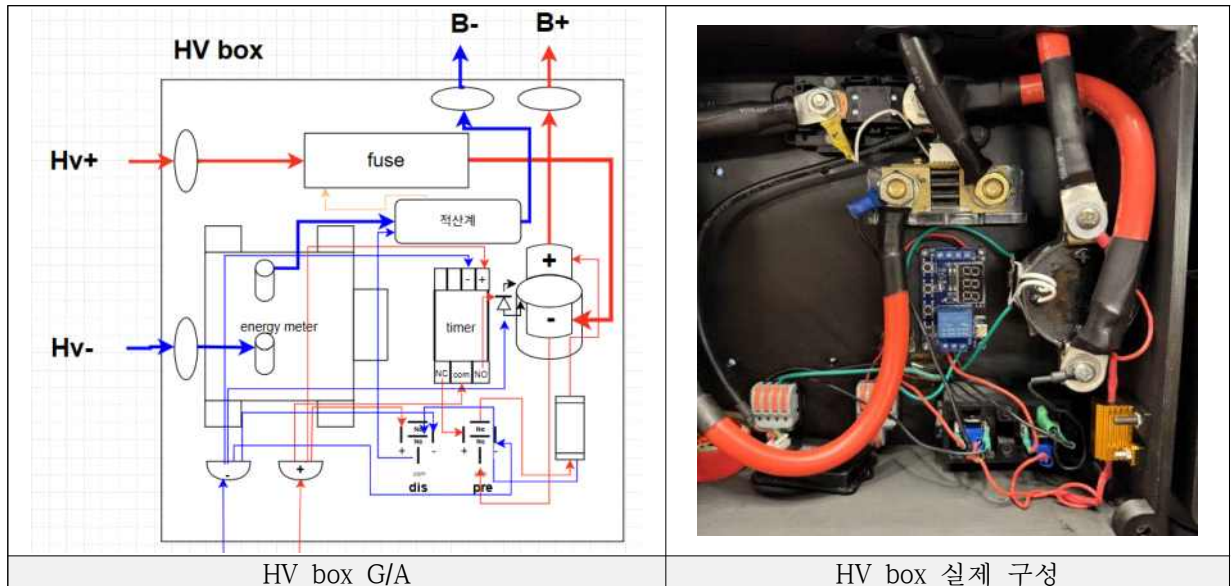
$$i = \frac{V}{R} \times e^{\left(\frac{-t}{RC}\right)}$$

RC 시정수는 τ 이다. 3τ 의 시간 동안 충전 한다면 95%가 충전되어 90% 이상의 충전을 만족할 수 있다.

i.b. 충방전 회로 시뮬레이션 (Simulink)



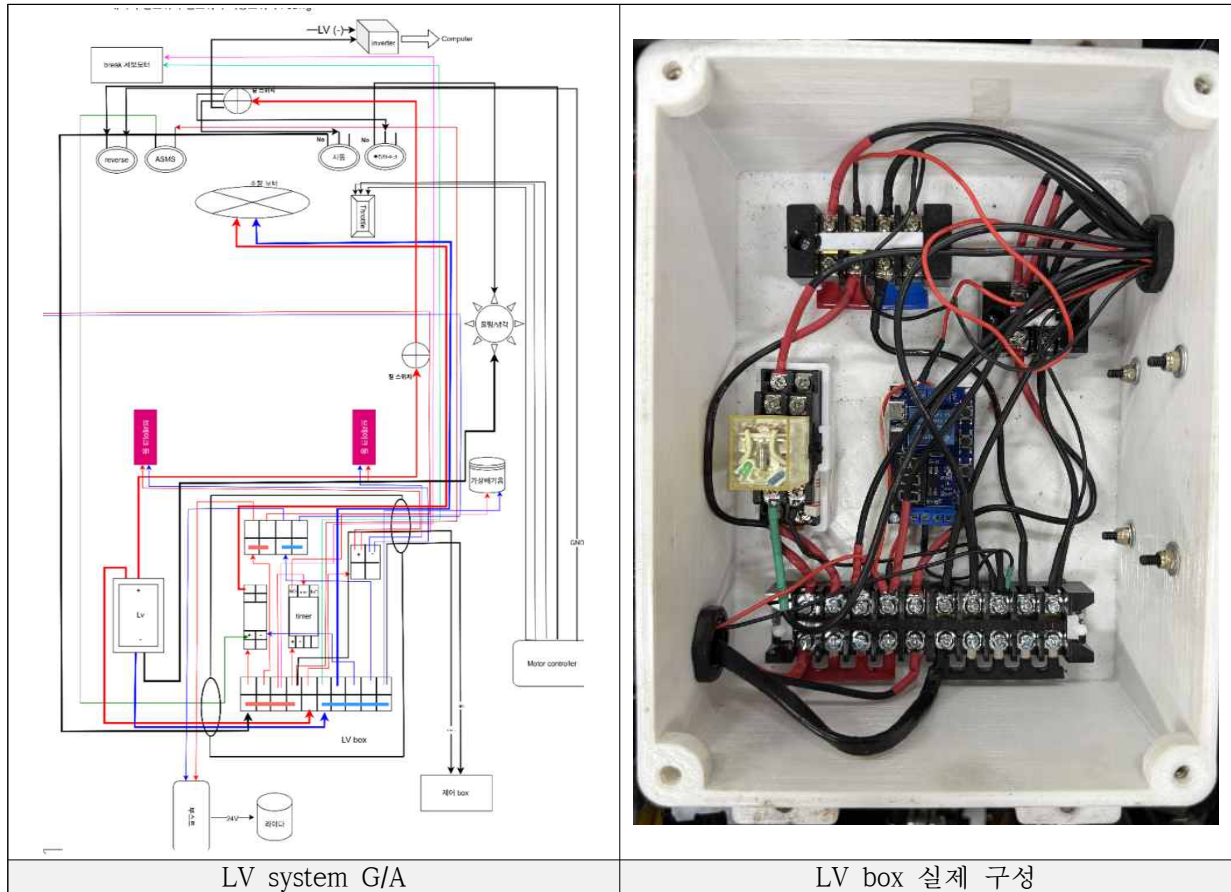
시뮬레이션 구현을 위하여 timer Relay 1.5s, HV on/off switch 3s 이후 초기값 1에서 0이 되도록 하였다. 인덕터에 전압이 생성되면 NC에서 NO로 COM을 연결하는 릴레이 특성을 이용하여 충전-방전 회로를 구현하였다. 0-1.5s 사이에선, charge Relay와 discharge Relay가 켜지며 C와 S2가 연결되게 된다. 이는 HV와 연결되어 있는 RC회로의 모습으로 저항을 통해 전류의 세기를 감소시켜 초기에 안전하게 충전할 수 있도록 한다. 마찬가지로 1.5s-3s 구간에선 Main Contactor, discharge Relay가 켜지며 저항 없이 HV와 Capacitor가 연결되며, 3s 이후에는 모든 Relay가 꺼지며 저항을 통해 Capacitor가 방전하게 된다.



HV 고전압이 인가되는 부분에는 퓨즈를 통해 과전류 보호를 한다. 이후 타이머 릴레이 및 충전 방전 릴레이를 거쳐 모터 컨트롤러가 충전된다. 모터 컨트롤러로 연결되기 직전 Main contactor(Air)를 이용해 보다 안전하게 연결된다.

돌입 전류나 순간적인 과도 전류를 고려하여 저항과 연결되지 않는 부분의 케이블은 낮은 게이지를 사용하였다. 또한 작년 대비, 저항의 갯수를 2개에서 1개로 줄이며 경량화 및 효율을 개선시켰다.

2.11.2. LV 시스템



차량 전장 부품들의 원활한 전원 공급과 수동 / 자율 모드 전환을 위한 LV system을 설계하였다. 기본적으로 LV배터리에서 나오는 12V를 사용하였으며, 라이다와 같이 24V가 필요한 부품은 부스트 컨버터를 연결해 승압하여 사용하였다. 브레이크등은 유압 장치 신호선에 배터리를 바로 연결하여 시동에 상관없이 브레이크를 밟으면 켜질 수 있도록 하였다. 브레이크등을 제외한 나머지 전장 부품들은 중간에 릴스위치를 연결하여 항상 릴스위치가 연결되었을 때만 전원이 공급된다. 브레이크 서보모터는 Estop 작동을 위하여 상시 켜져있도록 하며, 조향 서보모터는 Relay를 사용하여 자율모드에서만 켜지게 된다. 가상배기음은 Timer Relay를 연결해 차량이 출발 전회로를 거쳐 실제 켜지는 시간에 맞추어 가상음이 나올 수 있도록 하였다. 후진모드는 수동 시에는 모터컨트롤러에서 제공하는 Reverse를 GND에 연결되도록 하였고, 자율 시에는 신호를 통해 속도 reference 값을 반전시켰다. 쿨링/냉각 팬은 스위치를 통해 필요시에만 작동하도록 하여 부적절한 전력 소비를 줄였다. GND는 차량 본체가 아닌 단자대를 통한 LV박스 내부에 배치하도록 하여 감전 위험을 줄였고 제어box와의 전위 level를 맞추기 위해 GND를 공유하였다.

2.12. 센서 구성

2.12.1. LIDAR: Ouster OS1-32

주변 환경의 3차원 구조와 거리를 측정하는 핵심 센서로서, 32채널을 지원하는 Ouster OS1-32 모델을 채택하였다. 이 모델은 수직 해상도가 높아 콘과 같은 낮은 장애물의 형상을 정밀하게 식별하는 데 유리하며, 초당 655,360개의 포인트를 생성하여 조밀한 포인트클라우드를 제공한다. 이를 통해 후처리 단계에서 객체를 군집화하고 분할하는 과정의 정확도를 높일 수 있었다. 또한, IP68 등급의 방수방진 구조는 다양한 주행 환경에서의 작동 신뢰성을 보장하였다. LiDAR는 차량의 루프 중앙 전면에 설치하여 전방 시야를 최대한 확보하고, 차량 자체에 의해 데이터가 가려지는 현상을 최소화하도록 배치하였다.

2.12.2. Camera: Logitech C920 (2대)

LiDAR가 제공하기 어려운 색상과 같은 의미론적 정보를 획득하기 위해 두 대의 로지텍 C920 카메라를 구성하여 사용하였다. 이로써 원거리에 있는 콘의 색상도 명확하게 식별할 수 있으며, 30fps의 프레임 속도는 고속 주행 상황에서도 충분한 데이터를 제공할 수 있게 하였다. 특히, 두 대의 카메라를 센서 뭉치 좌우에 일정 간격을 두고 배치함으로써 스테레오 비전과 유사한 효과를 얻어, 한쪽 카메라의 시야가 가려지더라도 다른 쪽 카메라로 정보를 보완할 수 있는 Redundancy와 더 넓은 FOV를 확보하여 코너 구간의 퓨전 안정성을 개선하였다.

2.12.3. GNSS: u-blox ZED-F9P

차량의 전역 위치를 센티미터 수준의 정밀도로 파악하기 위해 RTK(Real-Time Kinematic) 보정을 지원하는 u-blox ZED-F9P 모듈을 사용하였다. ZED-F9P는 GPS, GLONASS 등 여러 위성 시스템의 신호를 동시에 수신하여 가용 위성 수를 극대화하며, RTK 기준국의 보정 신호를 통해 일반적인 GPS의 수 미터급 오차를 수 센티미터 이내로 줄일 수 있다. 이 고정밀 위치 정보는 SLAM 시스템의 초기 위치를 설정하고, 주행 중 발생하는 누적 오차를 보정하는 데 결정적인 기준점으로 활용되었다.

2.12.4. IMU: myAHRs+

차량의 자세(Roll, Pitch, Yaw)와 움직임 변화를 측정하기 위해 3축 가속도계, 3축 자이로스코프, 3축 지자기 센서가 통합된 9축 IMU인 myAHRs+ 모델을 선정하였다. 이 센서는 칼만 필터가 내장된 펌웨어를 통해 센서 노이즈를 자체적으로 보정하고 안정적인 자세 데이터를 출력한다. GNSS 신호가 불안정하거나 수신 불가능한 짧은 구간에서 IMU의 관성 항법 데이터를 통해 차량의 위치를 Dead Reckoning으로

추정하였으며, GNSS 데이터와 융합하여 보다 부드럽고 정확한 차량의 위치 및 자세 정보를 생성하는 데 사용되었다.

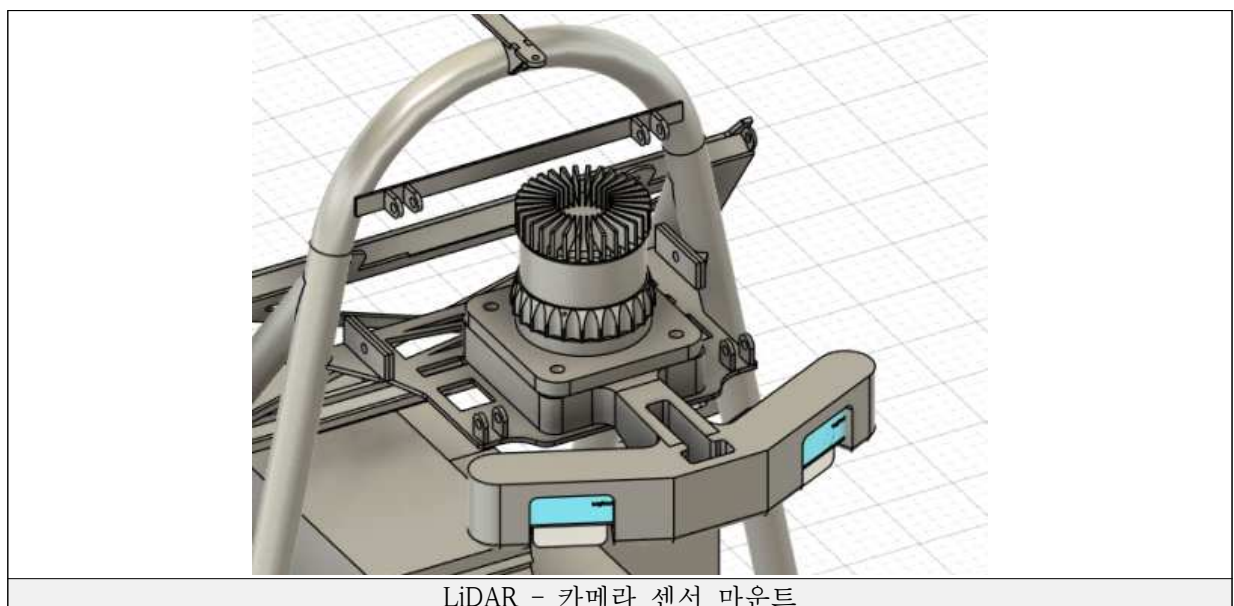
2.12.5. 센서 마운트

자율주행 인지 시스템의 핵심인 LiDAR와 카메라는 정확한 데이터를 취득하기 위해 최적의 위치에 견고하게 장착되어야 한다. 이를 위해 메인 롤 후프 상단에 센서 마운트를 설계 및 제작하여 배치하였다. 센서 마운트 설계 시 다음과 같은 사항들을 중점적으로 고려하였다.

첫째, 최적의 시야각을 확보하였다. 대회 규정 2.18.2.다의 장착 위치 규정을 준수하면서, LiDAR 센서가 주변 환경에 대한 360도 데이터를 가림 없이 수집하고 전방 카메라는 주행 경로 상의 라바콘을 명확히 인지할 수 있도록 차량의 가장 높은 지점에 위치시켰다. 이를 통해 차량 자체 구조물로 인한 데이터 손실(가려짐 현상)을 최소화하였다.

둘째, 진동에 의한 데이터 오차를 억제하였다. 주행 중 발생하는 진동은 포인트 클라우드와 이미지 데이터의 품질을 저하시키는 주요 원인이다. 따라서 프레임과 일체화된 견고한 구조로 설계하여 센서의 떨림을 최소화하고, 이를 통해 인지 시스템의 안정성과 신뢰도를 높였다.

셋째, 센서 간의 상호 위치 관계를 최적화하였다. 2대의 카메라를 일정한 간격을 두고 배치하여 스테레오 비전과 유사한 효과를 얻음으로써 더 넓은 시야를 확보하고 코너 구간에서의 퓨전 안정성을 개선하였다. 또한, GNSS 안테나를 최상단에 배치하여 위성 신호 수신율을 극대화하였다.



3. 소프트웨어

3.1. 미들웨어 마이그레이션

올해 우리 팀은 시스템의 근간을 기존 ROS1에서 ROS2로 전면 전환했다. 이것은 시스템의 실시간 성능과 안정성, 그리고 개발 효율성을 한 단계 끌어올리기 위한 필수적인 과정이었다.

가장 큰 구조적 변화는 ROS2의 컴포넌트 아키텍처를 도입한 것이다. 과거 ROS1 환경에서는 LiDAR 전처리, 카메라 인지, 센서 퓨전 등 각 기능 모듈이 별도의 프로세스로 실행되었다. 이 때문에 포인트클라우드와 같이 용량이 큰 데이터를 노드 간에 주고받을 때마다 데이터 직렬화, 프로세스 간 통신(IPC)으로 인한 오버헤드가 상당했다. ROS2에서는 여러 노드를 단일 프로세스 내에서 효율적으로 실행하는 Composition이 가능해졌다. 우리 팀은 이 기능을 적극 활용하여 인지 파이프라인의 주요 노드들을 하나의 프로세스로 묶어, 메모리상에서 데이터 포인터를 직접 전달하는 제로카피에 가까운 통신을 구현하였다. 이를 통해 데이터 파이프라인에서 발생하는 불필요한 데이터 복사를 제거하여, LiDAR 데이터 수신부터 최종 퓨전까지의 전체 지연 시간을 크게 단축시킬 수 있었다.

또한, ROS2의 라이프사이클 관리 기능을 도입하여 시스템의 안정성을 강화했다. 대회 현장에서 발생할 수 있는 예기치 않은 상황에 대비해, 각 인지 모듈을 'Unconfigured', 'Inactive', 'Active'와 같은 명확한 상태로 체계적으로 관리하도록 설계했다. 이를 통해 특정 노드에서 오류가 발생하더라도 전체 시스템을 중단시키는 대신, 해당 기능만 안전하게 비활성화하거나 재시작하는 것이 가능해졌다. 이는 대회 당일 시스템의 운영 안정성과 디버깅 효율을 높이는 중요한 개선점이었다.

3.2. 인지

자율주행 시스템의 성능은 주변 환경을 얼마나 정확하고 신속하게 인지하는지에 따라 결정된다. 우리는 각기 다른 강점을 가진 센서들을 유기적으로 결합하는 다중 센서 퓨전 방식을 채택하여, 단일 센서만으로는 달성하기 어려운 높은 수준의 인지 강건성과 정확성을 확보하고자 하였다. 전체 인지 시스템은 크게 LiDAR, 카메라, 그리고 위치 및 자세 추정을 위한 GNSS/IMU로 구성된다.

3.2.1. LiDAR 기반 라바콘 인지

LiDAR 센서를 이용한 3차원 인지 파이프라인의 일차적 목표는 주행 환경 내에 존재하는 콘의 정확한 3D 위치와 형태 정보를 강건하게 추출하는 것이었다. 이는 후

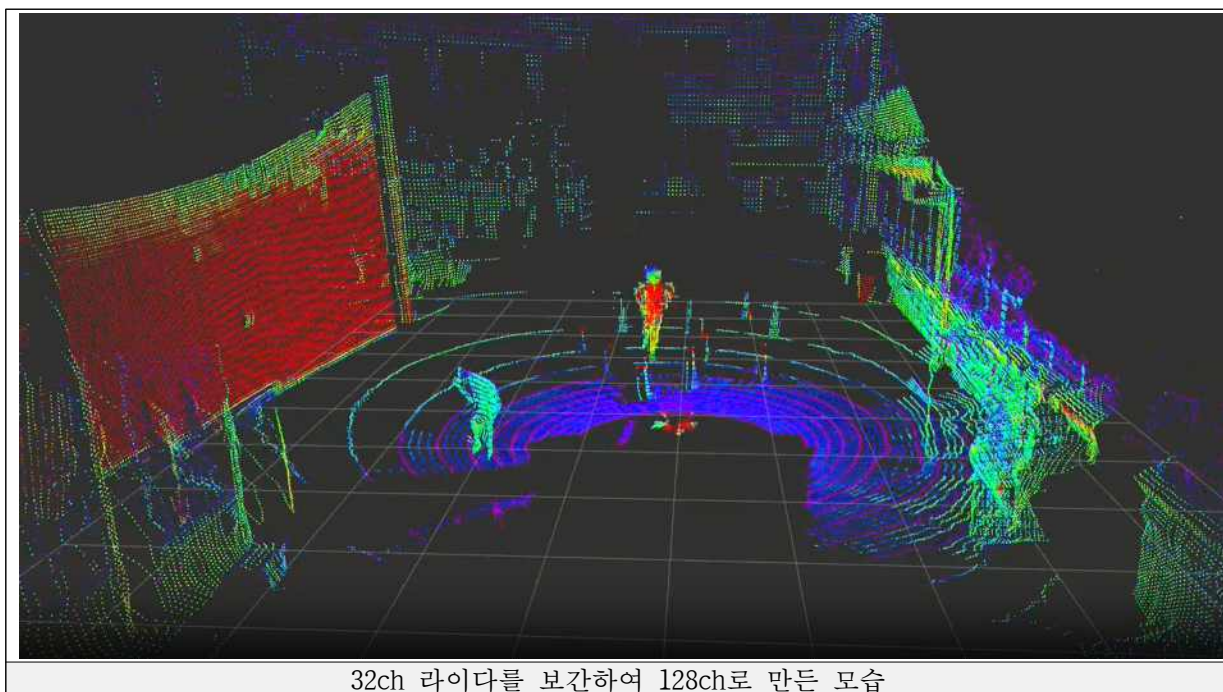
속 단계인 카메라와의 센서 퓨전 및 SLAM을 통한 맵 생성 과정에서 가장 기초적이면서도 핵심적인 데이터로 사용된다. 따라서, 방대한 양의 원본 포인트클라우드 데이터로부터 노이즈와 불필요한 정보를 효과적으로 제거하고, 실제 콘 객체만을 정밀하게 분할 및 추적하기 위한 다단계 파이프라인을 구축하였다. 전체 프로세스는 포인트클라우드 전처리, 객체 클러스터링 및 후보군 생성, 그리고 UKF 기반 객체 추적의 세 단계로 구성된다.

i. 포인트 클라우드 전처리 및 보간

Ouster OS1-32 LiDAR로부터 수신되는 원본 포인트클라우드의 해상도는 수직 방향의 물리적 한계로 인해, 거리의 제곱에 비례해 포인트 밀도가 떨어져 데이터가 희소해지는 근본적인 문제를 안고 있다. 이는 원거리 콘의 형상을 식별하거나 군집화하는 데 어려움을 야기한다. 이 문제를 해결하고 인지 성능을 극대화하기 위해, 우리는 전처리 파이프라인의 가장 첫 단계에서 포인트클라우드 보간을 수행하였다.

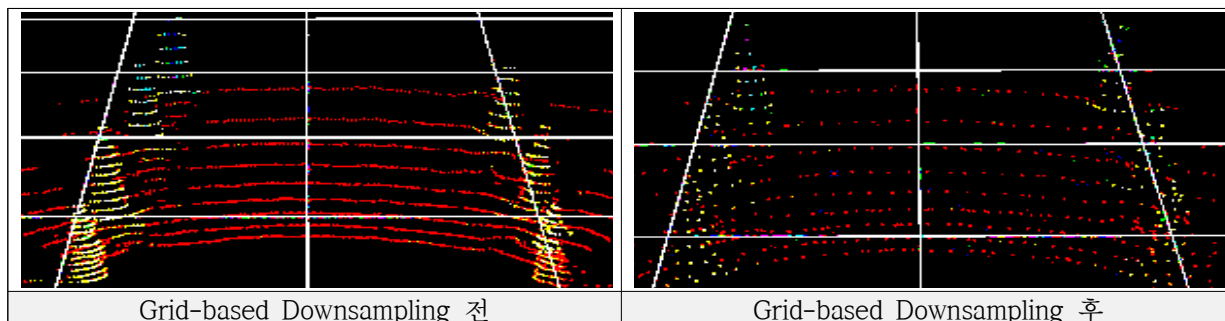
가장 핵심적인 아이디어는 비정형(Unstructured) 데이터인 포인트클라우드를 정형(Structured) 데이터인 이미지와 유사한 형태로 다루는 것이다. Ouster LiDAR는 고정밀 엔코더를 통해 각 포인트의 Azimuth 정보를 정확하게 제공하며, ring 필드를 통해 각 포인트가 어떤 수직 채널에서 측정되었는지 식별할 수 있다. 이 두 정보를 활용하여, 우리는 수신된 포인트클라우드를 1024x32 해상도를 갖는 정형 격자 데이터로 재구성하였다.

데이터가 정형화된 이후, BeamAltitudeManager를 통해 32개의 실제 빔 채널 사이에 목표 해상도(64채널)에 맞는 가상의 보간 빔 채널을 수학적으로 정의하였다. 그 후, 각 방위각 Column에 대해 인접한 두 실제 빔 채널의 포인트들을 찾아, 이들 사이에 선형 보간을 수행하여 가상의 빔 채널에 위치할 새로운 포인트를 생성하였다. 이 과정에서 우리는 객체의 경계면에서 발생할 수 있는 데이터 왜곡을 방지하기 위해 불연속성 감지 로직을 통합하였다. 즉, 보간 대상이 되는 두 포인트 간의 3차원 거리가 사전에 설정된 임계값을 초과할 경우, 이는 객체의 가장자리로 판단하여 보간을 수행하지 않고 더 가까운 쪽의 포인트를 그대로 사용하도록 설계하였다. 이를 통해 허공에 인위적인 포인트가 생성되는 것을 방지하고 맵의 기하학적 무결성을 유지하였다.

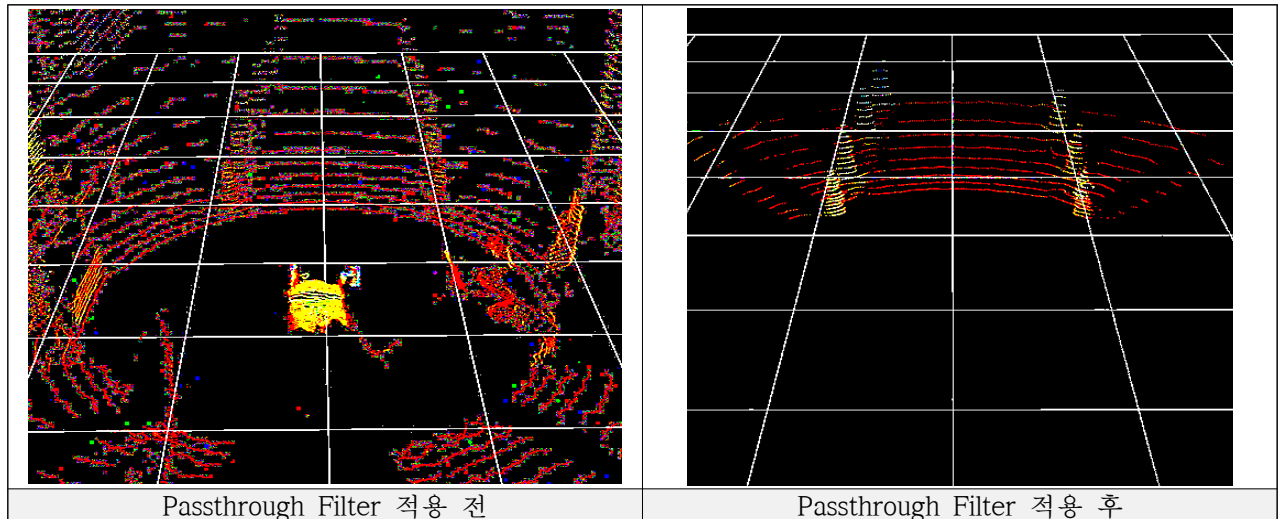


이러한 보간 과정을 통해 우리는 32채널 LiDAR 데이터를 물리적 센서의 교체 없이 1024x64 또는 그 이상의 고밀도 포인트클라우드로 변환할 수 있었다. 이렇게 밀도가 보강된 데이터는 후속 처리 단계의 강건성과 정확성을 향상시키는 기반이 되었다.

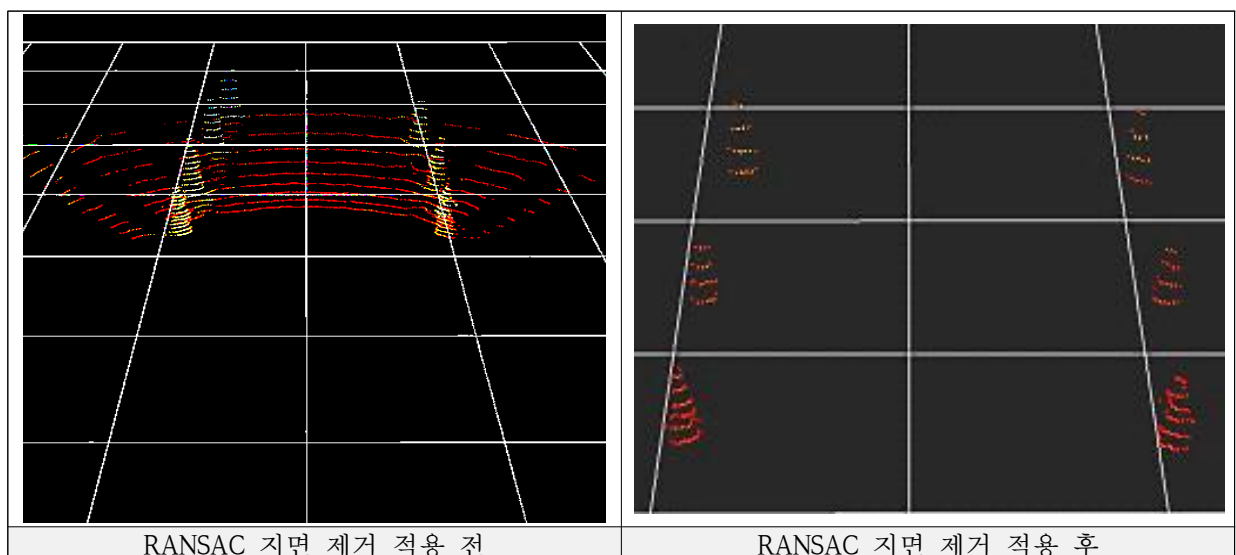
이후, 보간된 고밀도 포인트클라우드의 연산 효율성을 확보하기 위해 VoxelGrid 필터를 적용하여 전체 데이터 밀도를 균일하게 조정하였다. 이는 3차원 공간을 정육면체 형태의 복셀로 분할하고 각 복셀 내의 포인트들을 하나의 무게중심점으로 압축하는 방식으로, 데이터의 양을 줄이면서도 객체의 형상 정보는 최대한 보존하는 역할을 수행했다.



다음으로, 차량의 주행과 직접적인 관련이 없는 데이터를 제거하기 위해 ROI 필터링을 수행하였다. 차량 전방을 기준으로 설정된 거리와 좌우 시야각 내에 존재하는 포인트들만을 후속 처리 단계로 전달하여, 인지 시스템이 핵심 영역에 집중하도록 하였다.



마지막으로, 정확한 객체 분할의 선결 과제인 지면 제거를 수행하였다. 우리는 경사로나 고르지 않은 노면에서도 강건하게 동작하는 RANSAC 알고리즘을 도입하였다. RANSAC은 무작위 표본 합의를 통해 지면 평면 모델을 추정하고, 이 모델에 부합하는 포인트(Inliers)들을 지면으로, 그렇지 않은 포인트(Outliers)들을 잠재적 객체로 분류한다. 이 방식은 대회가 진행되는 트랙 환경을 고려했을 때, 콘의 하단부 소실 없이 온전한 형상을 후속 단계로 전달하는 가장 효율적이면서도 높은 강건성을 확보할 수 있는 방식이라고 판단하였다.

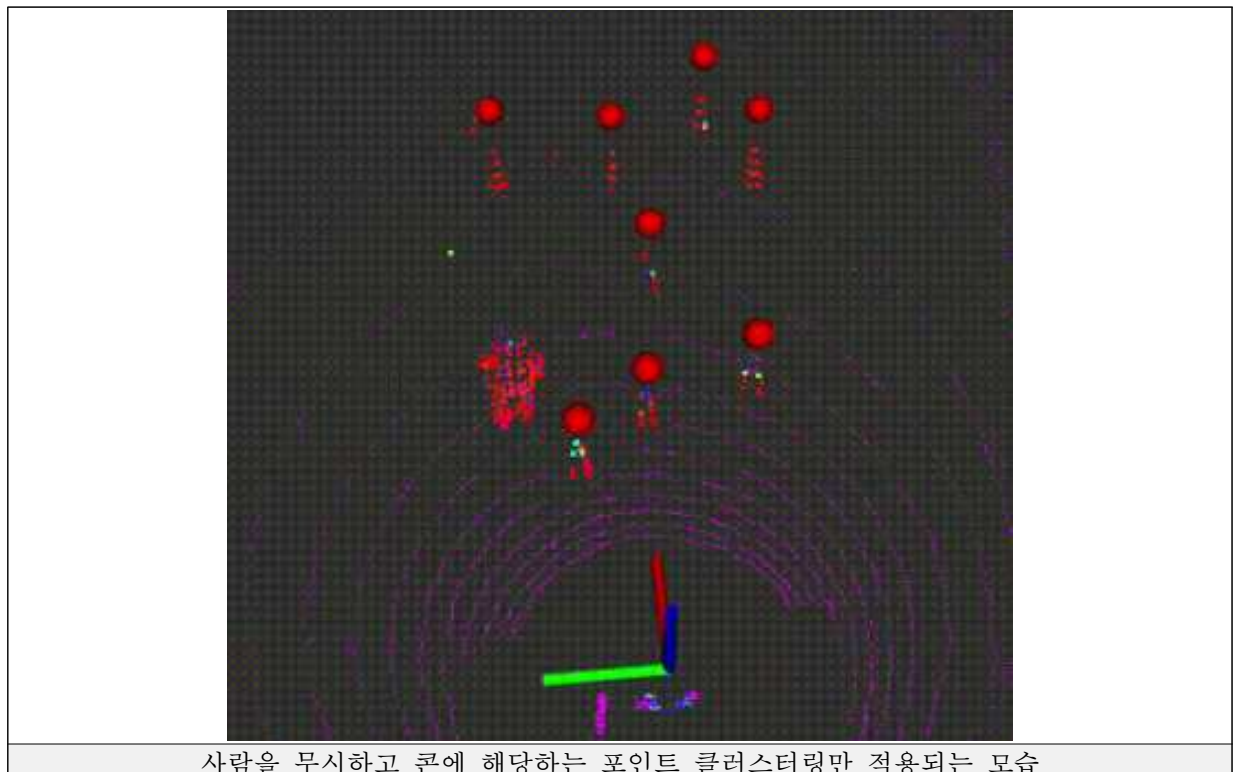


ii. 객체 클러스터링 및 후보군 생성

지면이 제거된 포인트클라우드에는 콘을 비롯한 사람, 가드레일, 외부 장애물 등의 다양한 객체들이 혼재해 있다. 이 중에서 콘으로 추정되는 포인트 그룹만을 정확하게 분리해내기 위해 밀도 기반 클러스터링 알고리즘을 적용하였다.

우리는 사전에 클러스터의 개수를 지정할 필요가 없고 기하학적으로 복잡한 형태의 군집도 효과적으로 찾아내는 DBSCAN 알고리즘을 채택하였다. DBSCAN은 특정 포인트 주변의 밀도를 기준으로 군집을 확장해나가는 방식으로, 두 개의 핵심 파라미터인 탐색 반경(epsilon)과 군집을 형성하기 위한 최소 포인트 수(min_points)를 사용한다. 이 파라미터들을 실제 콘의 물리적 크기와 LiDAR 포인트의 밀도에 맞춰 최적화함으로써, 물리적으로 분리된 각각의 콘을 하나의 독립된 클러스터로 정밀하게 군집화하였다. 이 과정에서 어떤 클러스터에도 속하지 못하는 희소한 포인트들은 노이즈로 간주되어 자연스럽게 필터링되었다.

클러스터링이 완료된 이후, 각 클러스터가 실제 콘일 가능성을 검증하는 필터링 단계를 거쳤다. 클러스터를 구성하는 포인트의 개수, 클러스터의 물리적 높이와 폭, 그리고 가로세로 비율 등을 기준으로 사전에 정의된 제약조건을 설정하였다. 비정상적인 크기나 형태를 가진 클러스터를 이 단계에서 제거함으로써 오탐지를 최소화하고, 가장 유력한 콘 후보군만을 선별하여 다음 추적 단계로 전달하였다.

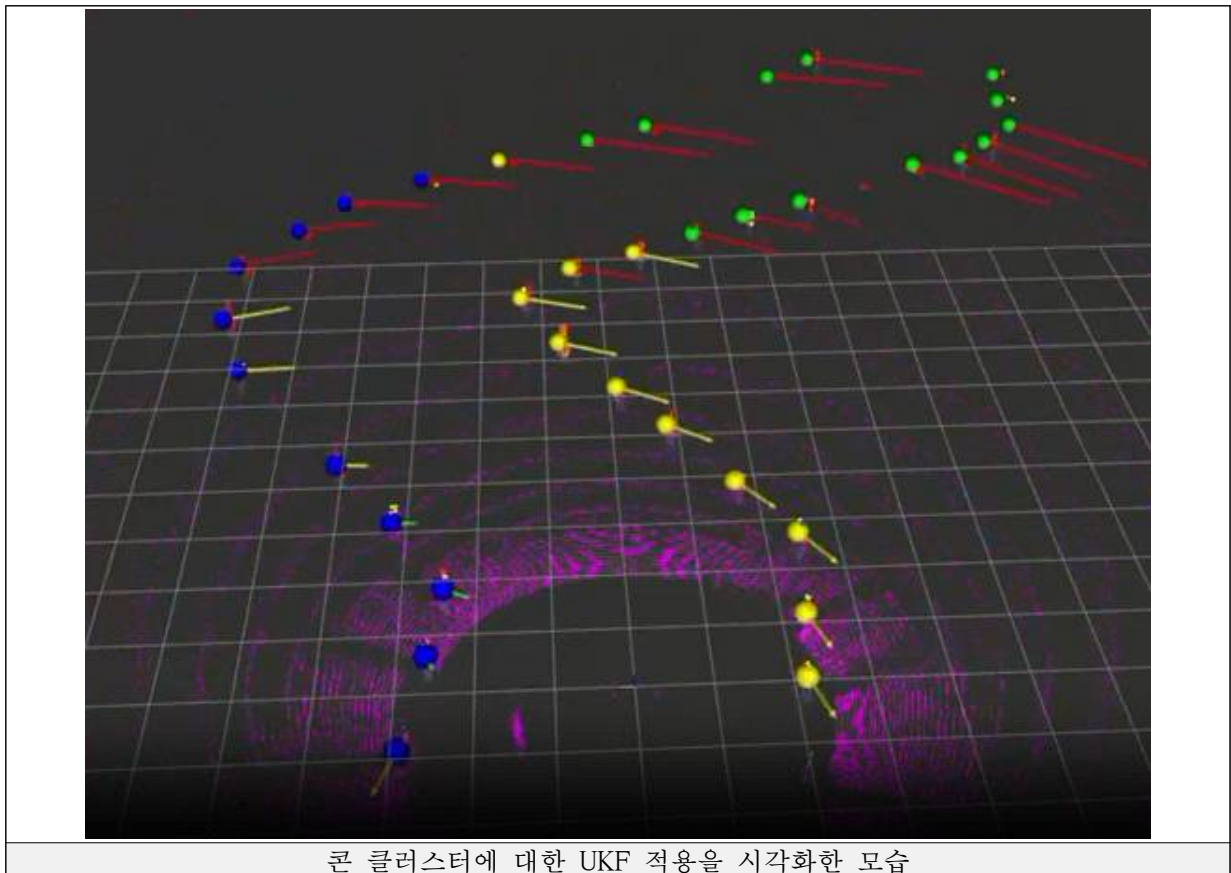


iii. UKF 기반 객체 추적

매 프레임마다 독립적으로 콘을 인지하는 방식은 데이터 연관 문제를 해결할 수 없어 안정적인 주행에 한계가 있다. 따라서 각 콘의 시간적 연속성을 파악하고 동적인 상태를 예측하기 위해 객체 추적 기술을 적용하였다.

차량과 콘의 상대적인 움직임에서 발생하는 비선형성을 효과적으로 처리하기 위해, 우리는 표준 칼만 필터(KF)나 확장 칼만 필터(EKF)보다 비선형성에 더 강건한 무향 칼만 필터(Unscented Kalman Filter, UKF)를 도입하였다. UKF 트래커는 현재 프레임에서 검출된 콘과 이전 프레임에서 추적 중이던 콘의 상태(위치, 속도 등)를 차량에 달린 IMU의 운동정보를 함께 고려하여 예측하고, 새로 관측된 값으로 이를 보정하는 예측-업데이트 과정을 반복한다.

이를 통해 일부 프레임에서 특정 콘이 일시적으로 인지되지 않더라도 연속적인 추적을 유지할 수 있으며, 각 콘에 부여된 고유 ID는 후속 SLAM 단계에서 랜드마크를 일관되게 식별하는 데 결정적인 역할을 수행한다. 이 파이프라인의 최종 출력물인 위치와 추적 ID가 부여된 3D 콘 배열은 다음 단계인 센서 퓨전 파이프라인의 핵심 입력 데이터로 사용된다.



3.2.2. 카메라 기반 인지

카메라 기반 인지 시스템은 LiDAR만으로는 획득하기 어려운 객체의 종류와 색상 같은 의미론적(Semantic) 정보를 식별하는 데 핵심적인 역할을 수행한다. 특히 대회 규정상 중요한 역할을 하는 콘의 색상과 신호등의 상태를 정확하게 판별하기 위해, 우리는 두 대의 로지텍 C920 카메라를 탑재하여 넓은 시야각을 확보하는 듀얼 카메라 시스템을 구축하였다. 이 시스템은 콘 인지와 신호등 인지라는 두 가지 주요 과업을 병렬로 처리하도록 설계되었다.

i. 비전 기반 콘 인지

실시간 객체 탐지 분야에서 높은 성능과 처리 속도를 보이는 YOLO 모델을 채택하여 콘 인지를 수행하였다. 특히, YOLOv8 모델은 경량화 버전에서도 우수한 정확도를 보여주어, 제한된 연산 자원 내에서 실시간성을 확보해야 하는 우리 시스템에 가장 적합하다고 판단하였다.

우리는 사전에 수집한 대량의 주행 이미지 데이터를 바탕으로 파란색, 노란색, 빨간색 콘을 포함하는 커스텀 데이터셋을 구축하였다. 이 데이터셋을 사용하여 YOLOv8 모델을 파인튜닝함으로써, 실제 대회 환경과 유사한 다양한 조도, 거리, 가림(Occlusion) 상황에 강건하게 반응하는 콘 인지 모델을 개발하였다. 각 카메라로부터 입력되는 이미지 스트림은 독립적인 YOLO 프로세서를 통해 처리되며, 검출된 각 콘에 대해 2D 이미지 평면상의 Bounding Box, Confidence Score, 그리고 'blue cone', 'yellow cone'과 같은 클래스 정보를 출력한다. 이 2D 객체 정보는 후속 센서 퓨전 단계에서 LiDAR 기반 3D 정보와 결합하는 데 사용된다.



ii. 비전 기반 신호등 인지

특정 미션 구간 통과를 위해 요구되는 신호등의 상태를 정확하게 판별하는 것은 필수적이다. YOLOv8과 같은 강력한 딥러닝 모델만으로 신호등을 인지할 수도 있으나, 신호등은 명확한 색상 패턴을 가지며 실전 환경에서 비교적 고정된 위치에 나타난다. 이에 본 팀은 딥러닝의 강력한 객체 검출 능력과 전통적인 컴퓨터 비전의 빠른 색상 분석 능력을 결합한 하이브리드 인지 알고리즘을 개발하였다. 이 접근법은 순수 딥러닝 방식보다 가볍고, 다양한 조명 변화 및 부분적 가려짐 상황에서 더 높은 안정성과 신뢰성을 확보할 수 있다고 판단하였다.

신호등 인지 파이프라인의 첫 단계는 전체 이미지에서 신호등이 위치한 ROI를 특정하는 것이다. 이를 위해 사전에 'traffic_light' 클래스로 학습된 YOLOv8 모델을 사용하여 신호등의 2D Bounding Box를 1차적으로 검출한다. 이 방식은 주행 중 차량의 흔들림이나 위치 변화에도 불구하고 항상 신호등 영역을 안정적으로 추적할 수 있게 해준다.

다음으로, YOLOv8에 의해 검출된 ROI는 후속 색상 분석의 정확도를 높이기 위해 조명 변화에 강건한 HSV 색상 공간으로 변환된다. 변환된 HSV 이미지에 대해, 사전에 실험적으로 정의된 빨간색과 초록색의 임계값 범위를 적용하여 각 색상에 해당하는 픽셀을 마스킹한다. 이후 ROI 전체 면적 대비 해당 색상 픽셀이 차지하는 비율을 계산하여 각 색상에 대한 HSV 신뢰도를 산출한다.

최종적인 신호등 상태 판별은 verification_mode 파라미터에 따라 하이브리드 방식을 사용한다. 이 방식은 YOLOv8 모델이 출력한 클래스 신뢰도(예: 'RED'일 확률)와 HSV 색상 분석을 통해 계산된 픽셀 비율(HSV 신뢰도)을 가중 합산하여 결합 신뢰도를 계산한다. 예를 들어, confidence_weight_yolo와 confidence_weight_hsv 파라미터를 각각 0.5로 설정하여 두 신뢰도를 균등하게 반영한다. 만약 이 결합 신뢰도가 사전에 설정된 임계값을 넘고, HSV 신뢰도가 threshold_ratio 이상으로 특정 색상을 명확히 가리킬 경우, 해당 색상을 최종 신호등 상태로 확정한다.

이러한 하이브리드 방식은 YOLO가 형태적으로 신호등을 정확히 검출하고, HSV가 물리적인 색상 정보를 검증하는 상호 보완적인 구조를 통해 다음과 같은 강점을 가진다. 첫째, YOLO가 신호등의 위치를 먼저 특정하므로 배경에 유사한 색상의 물체가 있더라도 오인할 확률이 현저히 줄어든다. 둘째, 딥러닝 모델이 간혹 색상을 오

인하거나 불확실한 결과를 내놓더라도, HSV 기반의 물리적 색상 검증을 통해 한 번 더 확인함으로써 최종 판별의 신뢰도를 높인다. 마지막으로, 전체 이미지가 아닌 작은 ROI에 대해서만 색상 분석을 수행하므로 연산 효율성이 뛰어나다. 이렇게 판별된 신호등 상태 정보는 판단 및 제어 모듈로 전달되어, 정지, 출발 등 미션 수행 전략을 수립하는 데 핵심적인 입력으로 사용된다.



비전 기반으로 신호등의 위치와 종류 인식

3.2.3. IMU 캘리브레이션

자율주행 시스템의 위치 및 자세 추정 성능은 관성측정장치(IMU) 데이터의 품질에 크게 좌우된다. 하지만 IMU 센서는 내재적으로 바이어스(bias)와 노이즈를 포함하고 있어, 원본 데이터를 직접 사용할 경우 시간이 지남에 따라 오차가 누적되어 심각한 드리프트(drift)를 유발한다. 이는 EKF와 같은 상태 추정 필터의 성능을 저하시키는 주된 원인이 된다. 따라서 본 팀은 센서 데이터의 신뢰성을 확보하고 위치 추정의 정확도를 극대화하기 위해 체계적인 IMU 캘리브레이션 프로세스를 개발하고 적용하였다.

i. 데이터 수집 및 정적 바이어스 추정

캘리브레이션의 가장 기본은 센서가 정지 상태일 때의 출력 특성을 파악하는 것이다. 이를 위해 차량을 수평하고 진동이 없는 환경에서 약 5분간 정지 상태를 유지하여 약 30,000개의 IMU 데이터 샘플을 수집하는 improved_calibration.py 스크립트를 자체 제작하였다.

단순히 수집된 데이터의 평균을 사용하는 방식은 순간적인 노이즈나 이상치(outlier)에 취약하다. 이러한 문제에 대응하기 위해, 본 팀은 사분위수 범위(IQR, Interquartile Range)를 이용한 이상치 제거 기법을 적용하여 전체 데이터의 약 1.5%에 해당하는 이상치를 제거하고 통계적 강건성을 확보하였다. 최종적으로 추정된 바이어스 값은 가속도계에서 $[-0.0482, 0.0153, -0.0921] \text{ m/s}^2$, 자이로스코프에서

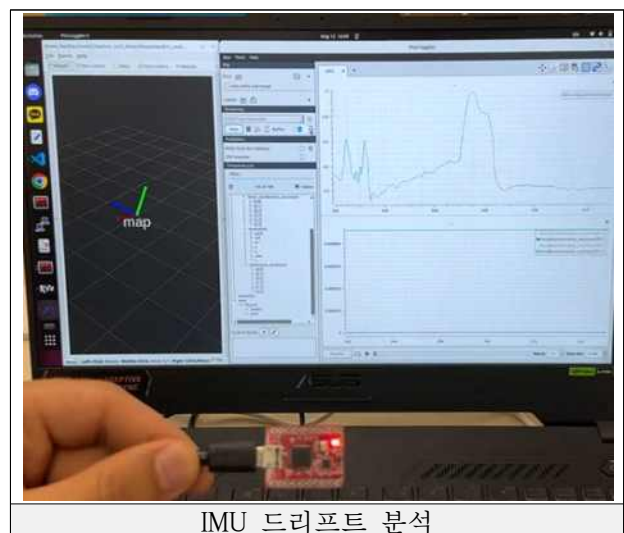
[0.0015, -0.0008, 0.0021] rad/s로 계산되었다. 특히 중력 가속도 성분을 정확히 분리하기 위해, 스크립트는 데이터의 평균 벡터를 계산하여 중력이 작용하는 주축을 자동으로 판별하고 해당 축에 이론적인 중력 가속도(9.81 m/s^2) 값을 적용하여 순수한 바이어스 성분만을 정밀하게 추정하도록 설계하였다.

ii. 노이즈 특성 분석 (Allan Variance)

정적 바이어스 외에도 시간에 따라 변화하는 노이즈 특성을 파악하는 것은 EKF 필터의 성능 튜닝에 매우 중요하다. 본 팀은 IMU 센서 노이즈 분석의 표준 기법인 알란 분산(Allan Variance) 분석을 improved_calibration.py 스크립트에 통합하였다. 수집된 대량의 데이터를 바탕으로 알란 분산을 계산하여, 필터 성능에 직접적인 영향을 미치는 핵심 파라미터인 바이어스 안정성(Bias Stability) 값을 각 축에 대해 도출하였다. 분석 결과, 자이로스코프의 바이어스 안정성은 약 $2.5e-4 \text{ rad/s}$, 가속도계는 약 $5.0e-3 \text{ m/s}^2$ 수준으로 측정되었다. 이 정량적 지표는 EKF의 프로세스 노이즈 공분산 행렬(Q)을 설정하는 핵심 근거로 사용되어 accel_bias_noise와 gyro_bias_noise 파라미터를 최적화하고 필터가 IMU 바이어스의 시간적 변화를 더 정확하게 추정하도록 하였다.

iii. 캘리브레이션 적용 및 검증

위 과정을 통해 도출된 모든 캘리브레이션 파라미터는 JSON 형식의 파일로 저장된다. 실제 주행 시에는 imu_preprocess_node가 이 캘리브레이션 파일을 로드하여 수신되는 원본 IMU 데이터에 대해 실시간으로 바이어스를 보상하고, 주행 중 발생하는 노이즈를 감소시키기 위해 차단 주파수 10.0Hz의 저역 통과 필터(LPF)를 적용한다.

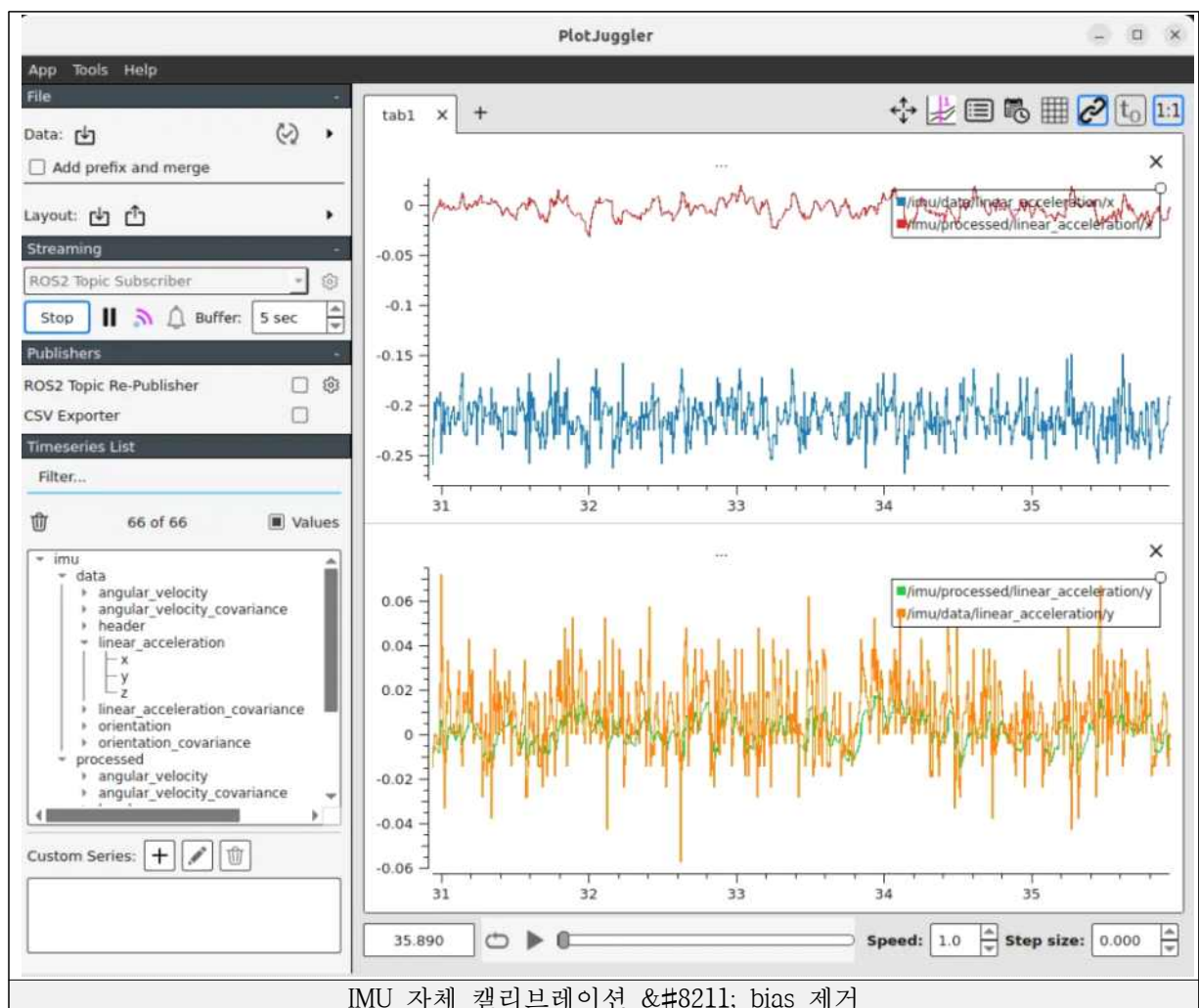


캘리브레이션의 효과를 정량적으로 검증하기 위해, dead_reckoning_node.py를 이용하여 약 180초간의 정적 데이터를 기반으로 드리프트를 비교 분석하였다. 그 결과는 다음과 같다.

자세 드리프트 (Yaw): 원본 데이터는 180초 후 약 15.7도의 누적 오차를 보인 반면, 캘리브레이션된 데이터는 단 0.8도의 오차만을 보여 약 19.6배의 뚜렷한 성능 향상을 확인하였다.

위치 드리프트: 원본 데이터는 약 2.5미터의 드리프트가 발생했으나, 캘리브레이션 후에는 약 1.1미터로 감소하여 약 2.3배의 개선 효과를 보였다.

이 검증을 통해, 특히 자이로 바이어스 보정이 Yaw 드리프트 억제에 매우 효과적임을 입증하였으며, 가속도계 바이어스 보상 역시 위치 추정의 누적 오차를 의미 있는 수준으로 감소시키는 것을 확인하였다.



3.2.4. RTK-GPS 보정

표준 GNSS는 전리층 및 대류층 통과 지연, 위성 시계 오차 등으로 인해 수 미터 수준의 오차를 내재하고 있어 자율주행의 위치 인식 센서로 단독 사용하기에는 정밀도가 부족하다. 이러한 한계를 극복하고 센티미터 수준의 위치 정밀도를 달성하기 위해, 본 팀은 반송파 위상차 보정(RTK, Real-Time Kinematic) 방식을 채택하였다.

RTK는 C/A 코드 기반의 의사거리 오차를 보정하는 DGPS 방식보다 한 단계 발전된 기술로, 파장이 약 19cm에 불과한 L1 반송파의 위상을 직접 측정하여 정밀도를 극대화한다. 이 방식의 핵심 과제는 신호 수신 시점에 위성과 수신기 사이의 정확한 반송파 파장의 개수, 즉 모호 정수(Integer Ambiguity)를 결정하는 것이다. 기준국과 이동국(차량)이 두 개 이상의 동일한 위성을 동시에 관측하고, 관측된 위상 데이터를 서로 차분함으로써 위성과 수신기의 시계 오차, 대기권 오차 등 공통 오차 요소를 제거한다. 이 과정을 통해 모호 정수의 후보군이 수억 개에서 수십 개 이내의 작은 정수 범위로 축소되며, 이를 성공적으로 특정하면 'Float' 상태에서 'Fix' 상태로 전환되어 센티미터급 정밀도를 확보하게 된다.

RTK 보정 데이터 흐름은 다음과 같이 구성된다.

차량의 U-blox 수신기는 위성 신호를 수신하여 고정밀 위치를 `ublox_gps` 노드로 전달하고, 이는 `/ublox_gps_node/fix` 토픽으로 발행된다.

`fix2nmea` 노드는 이 NavSatFix 메시지를 NMEA 0183 표준의 GNGGA 문장으로 변환하여 `/ntrip_client/nmea` 토픽으로 발행한다.



`ntrip_client` 노드는 이 NMEA 문장을 이용해 차량의 대략적인 위치를 NTRIP Caster(서버)로 전송한다.

Caster는 이 위치를 기반으로 가장 가까운 기준국의 RTCM 3.x 버전 보정 데이터를 인터넷을 통해 스트리밍한다. 본 대회에서는 대회장 인근의 국토지리정보원 위성 기준국을 기지국으로 선정하여 기준국-이동국 간 거리를 최소화하고 대기권 오차 상쇄 효과를 극대화하였다.

ntrip_client 노드는 수신한 RTCM 데이터를 /ntrip_client/rtcm 토픽으로 발행한다. ublox_gps 노드는 이 RTCM 데이터를 U-blox 수신기로 전달하며, 수신기 내부 칩셋이 위성 신호와 RTCM 데이터를 융합하여 RTK 연산을 수행하고 고정밀 위치가 포함된 NAV-PVT 메시지를 출력한다. 이 결과는 다시 /ublox_gps_node/fix 토픽으로 발행되어 최종적으로 cm급 정밀도를 갖는 위치 정보를 시스템에 제공한다.

GNSS 안테나는 차량의 전륜 축 중앙에 장착하여 위성 신호 수신율을 극대화하였으며, 차량의 상부에 위치한 LiDAR 및 카메라 센서와의 물리적, 전자기적 간섭을 회피하도록 설계하였다.

3.2.5. 콘의 절대 좌표 변환

LiDAR 및 카메라 기반의 인지 시스템은 센서를 기준으로 한 상대 좌표계 (os_sensor, base_link 등)에서 콘의 위치를 검출한다. 그러나 SLAM을 통한 지도 생성 및 전역 경로 계획을 위해서는 모든 인지 객체가 일관된 하나의 절대 좌표계, 즉 map 프레임으로 통일되어야 한다.

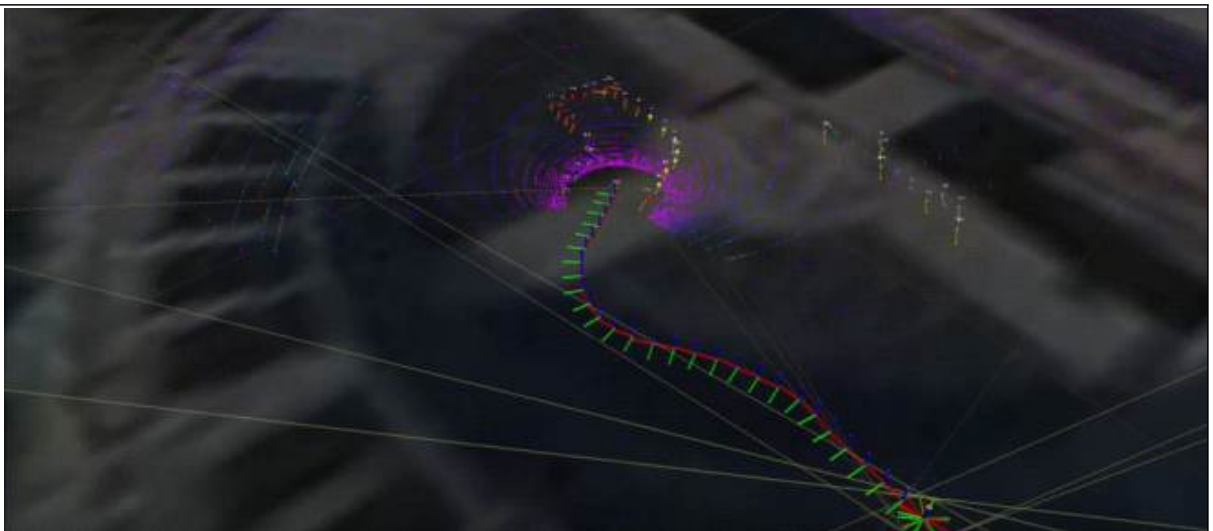
본 시스템은 ROS2의 tf2 라이브러리를 사용하여 이러한 좌표 변환을 실시간으로 수행한다. 인지 파이프라인에서 검출된 콘의 상대 좌표는 다음과 같은 변환 체인을 거쳐 절대 좌표로 변환된다.

[map ← odom ← base_link ← os_sensor]

os_sensor → base_link: LiDAR 센서와 차량의 기준점(base_link) 사이의 기하학적 관계는 고정되어 있으므로, 이는 tf2_ros의 static_transform_publisher를 통해 정적 변환으로 발행된다.

map → odom → base_link: gps_imu_fusion 패키지의 EKF 노드는 IMU와 RTK-GPS 데이터를 융합하여 map 프레임 기준 차량의 현재 위치와 자세를 지속적으로 추정한다. 이 결과는 map에서 base_link로의 동적 변환 정보로 발행된다.

cone_frame_transformer.py 노드는 /cone/fused/ukf 또는 /cone/lidar 토픽에서 base_link 기준의 콘 배열 메시지를 구독한다. 메시지를 수신하는 즉시, TF2 버퍼의 lookup_transform 함수를 이용해 해당 타임스탬프의 map 프레임과 base_link 프레임 간의 변환 행렬을 조회한다. 이후, 배열 내 모든 콘의 위치 벡터에 이 변환 행렬을 적용하여 map 프레임 기준의 절대 좌표로 변환하고, /cone/fused/ukf/map과 같은 새로운 토픽으로 발행한다. 이 과정을 통해 후단의 SLAM 및 경로 계획 모듈은 시간에 따라 변하는 차량의 움직임과 관계없이, 항상 일관된 절대 좌표계 상에서 콘의 위치 정보를 활용할 수 있다.



실제 지도 위에 콘의 절대좌표를 시각화한 모습 (건국대학교 노천극장)

3.2.6. 센서 퓨전 - GNSS, AHRS, Odometry

안정적이고 신뢰도 높은 차량의 위치 및 자세 정보는 SLAM의 기반이 되고 모든 인지 결과의 기준이 되는 핵심 데이터이다. RTK-GPS는 절대 위치의 정확도를 보장하지만 낮은 수신율과 음영 지역의 한계가 명확하고, IMU는 높은 빈도로 상대 움직임을 측정하나 오차 누적으로 인한 드리프트가 필연적이다. 이 두 센서의 장점을 결합하고 단점을 상호 보완하기 위해, 본 팀은 gps_imu_fusion 패키지에 15-state 확장 칼만 필터(EKF)를 직접 구현하였다.

i. EKF 상태 모델 및 예측 단계 (IMU Propagation)

본 팀이 설계한 kai_ekf_core의 EKF는 차량의 상태를 위치(3), 속도(3), 자세(3), 가속도계 바이어스(3), 자이로스코프 바이어스(3)의 총 15개 변수로 정의한다. EKF의 예측 단계는 3.2.5절에서 캘리브레이션된 IMU 데이터를 입력받아 구동된다. 고주파(100Hz 이상)의 IMU 데이터가 입력될 때마다 필터는 이를 적분하여 현재 상태를 예측한다. 이때, IMU 센서와 차량 기준점(base_link) 간의 물리적 거리(Lever Arm)로 인해 발생하는 원심가속도($\omega \times (\omega \times r)$)와 접선가속도($\alpha \times r$) 성분을 실시간으로 계산하여 측정된 가속도 값에서 보상하는 로직을 ekf_fusion_node.cpp에 구현하였다. 이를 통해 차체의 순수한 병진 운동을 더욱 정밀하게 추정한다.

ii. 보정 단계 및 핵심 전략: 다중 센서 융합과 동적 노이즈 조절

저주파(10Hz 내외)의 RTK-GPS 데이터가 수신되면 보정 단계가 수행된다. 필터는 IMU 예측치와 GNSS 측정치 간의 오차를 계산하고, 칼만 이득에 따라 상태 변수를 보정한다. 하지만 기존에 고려되었던 ZUPT(Zero-Velocity Update) 로직은 차량의 미세한 진동으로 인해 정차 상태를 오인하는 경우가 많았고, GNSS 신뢰도가 급격히 떨어지는 극저속 구간에서는 EKF의 발산 문제가 있었다.

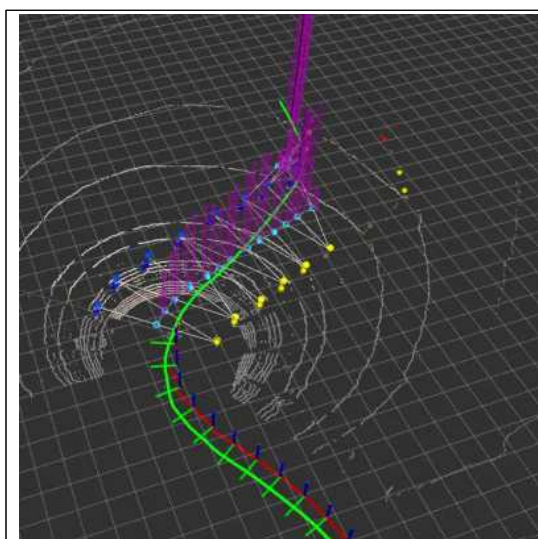
이 문제들을 해결하기 위해, 모터 엔코더 데이터를 추가적인 측정치로 도입하고 차량의 주행 상태에 따라 EKF의 신뢰도를 동적으로 조절하는 3-상태 적응형 시스템을 구축하였다.

정지 상태 (Stationary): 속도가 zupt_speed_threshold_low(0.08 m/s) 미만으로 일정 시간 유지되면, 시스템은 ZUPT를 적용한다. 이 상태에서는 신뢰할 수 없는 GNSS 방향각 정보를 완전히 무시하고(gps_heading_noise_low를 매우 큰 값으로 설정), IMU 업데이트 빈도를 stationary_imu_decimation_factor 만큼 줄이며, 프로세스 노이즈(Q)를 zupt_noise_scale_stationary 만큼 감소시켜 IMU 드리프트를 강력하게 억제한다.

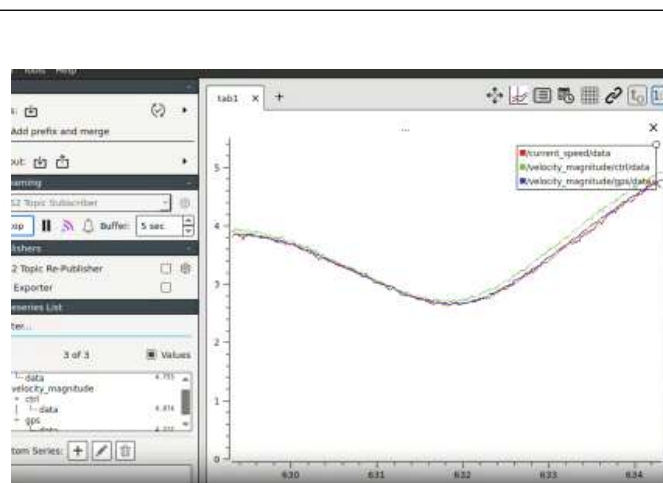
전환 상태 (Transition): 정지 후 출발하여 저속 구간에 진입하면, 시스템은 전환 상태가 된다. 이 상태에서는 이미 쌓인 자이로의 Yaw 방향 오차를 신속하게 제거하기 위해, RTK-GPS에서 취득한 Heading 데이터의 측정 노이즈(gps_heading_noise_mid)를 극단적으로 낮추어 필터가 GNSS 방향각을 거의 절대적으로 신뢰하게 만든다. 이를 통해 차량이 출발하자마자 지연 및 오차 없는 Heading을 확보하였다.

주행 상태 (Moving): 속도가 임계값을 넘어서면, 시스템은 일반 주행 상태로 전환하여 IMU의 빠른 반응성과 GNSS의 장기적인 정확성을 균형 있게 융합한다. 특히 이 구간에서는 구동 모터에 장착된 단일 엔코더 데이터를 20Hz 주기로 함께 융합한다. 차량 구동계가 슬립이 거의 없는 Open-Differential 구조라는 점에 착안하여, 중앙 모터 엔코더 속도를 양쪽 후륜의 평균 속도로 가정하였다. 이 엔코더 기반 속도는 GNSS 신뢰도가 낮은 저속 구간에서 EKF의 속도 상태 변수를 안정적으로 보정하는 핵심 측정치로 작용하며, 차량 속도에 따라 GNSS 속도와의 가중치를 동적으로 조절하는 적응형 로직을 구현하였다.

이러한 다중 센서 융합과 동적 상태 기반의 적응형 EKF 시스템을 통해, 우리는 다양한 주행 시나리오에서 발생할 수 있는 각각의 문제점(저속 드리프트, 정지 시 오인, 출발 시 헤딩 오차)을 효과적으로 해결하고, 일관되며 정밀한 Odometry 정보를 안정적으로 획득할 수 있었다.



GPS + IMU 퓨전을 통해 구한
오도메트리 시각화



GPS + IMU 퓨전을 통해 구한 오도메트리 속도

3.2.7. 센서 퓨전 - LiDAR, Camera

LiDAR와 카메라는 각각 3D 공간 정보와 2D 의미론적 정보에 강점을 가지는 상호 보완적인 센서이다. LiDAR만으로는 콘의 색상을 알 수 없으며, 카메라만으로는 정확한 3차원 위치를 측정하기 어렵다. 따라서 두 센서의 장점을 결합하여 '색상 정보가 포함된 정확한 3D 콘 객체'를 생성하는 센서 퓨전 과정은 우리 인지 시스템의 핵심적인 부분이다. 우리는 데이터 처리의 정밀도와 효율성을 극대화하기 위해, 포인트 단위로 색상을 입히는 Point-wise 엘리 퓨전과 객체 단위로 정보를 결합하는 오브젝트 레벨 퓨전을 순차적으로 적용하는 다단계 퓨전 전략을 채택하였다.

i. 센서 캘리브레이션

LiDAR와 카메라라는 두 종류의 센서 데이터를 하나의 좌표계에서 정합하는 것은 센서 퓨전의 성공을 위한 가장 중요한 선결 과제이다. 3차원 공간의 한 점이 2차원 이미지 평면의 어느 픽셀에 정확히 대응되는지를 알아야만 포인트클라우드 컬러링과 IoU 기반 센서 퓨전이 의미를 가지기 때문이다. 이를 위해 우리는 각 센서의 내외부 파라미터를 정밀하게 측정하는 다단계 캘리브레이션 프로세스를 수행하였다.

첫째로, 각 카메라의 내부 파라미터(Intrinsic Parameters)를 측정하였다. 이는 렌즈의 초점 거리, 주점, 렌즈 왜곡 계수 등을 포함하는 행렬이며, 이 행렬을 통해 3차원 세계 좌표계에 존재하는 어떤 점 벡터를 2차원 이미지 평면으로 사영시키는 선형 변환을 적용할 수 있다. 내부 파라미터 행렬을 얻기 위해 우리는 체커보드 패턴을 다양한 각도와 거리에서 촬영하여 OpenCV의 카메라 캘리브레이션 도구를 통해 정확한 값을 도출하였다.

이러한 캘리브레이션 과정의 자동화와 재현성을 위해 `get_intrinsic_camera_calibration.py` 노드를 구현하였다. 해당 노드는 실시간 이미지 스트림에서 체커보드 패턴을 감지하고, `cv2.findChessboardCorners` 및 `cv2.cornerSubPix` 함수를 통해 2D 이미지상의 코너 좌표를 서브픽셀 단위로 정밀하게 추출한다. 우리는 실제 세계에서 각 정사각형의 크기가 0.07m임을 이미 알고 있으므로, 이 3D 공간 정보와 추출된 2D 픽셀 정보의 대응 관계를 `cv2.calibrateCamera` 함수에 입력한다. 이 함수는 여러 이미지 프레임에 걸쳐 Perspective-n-Point(PnP) 문제를 풀어 카메라의 투영 행렬과 렌즈 왜곡 계수를 최적화한다. 렌즈 왜곡 계수는 렌즈 고유의 물리적 특성으로 인해 발생하는 이미지의 방사상, 접선 왜곡을 보정하여, 이후 프로젝션 과정에서 정확한 정합을 가능케 하는 핵심 파라미터이다. 캘리브레이션의 정밀도는 재투영 오차(RMS Reprojection Error)를 통해 정량적으로 평가하였으며, 우리는 이 값을 1.0 미만으로

최소화하기 위해 다양한 각도와 위치에서 100장 이상의 고품질 이미지를 캡처하는 노력을 기울였고, 이러한 노력의 결과는 캘리브레이션 결과 파일에서 정량적으로 계산된 rms_error 0.11, 0.38 이라는 낮은 오차 달성으로 확인할 수 있었다.

둘째로, 시스템의 기준 좌표계인 LiDAR 센서를 중심으로 각 카메라의 상대적인 위치와 방향, 즉 외부 파라미터(Extrinsic Parameters)를 측정하였다. 이 과정에서는 체커보드를 포함한 TV, 박스 등 기준점으로 삼을 형태가 명확한 물체를 LiDAR와 카메라 양쪽에서 동시에 인식할 수 있도록 배치하고, 각 센서 데이터에서 추출한 포인트들의 3D 위치 → 2D 픽셀좌표 대응을 분석하여 두 센서 간의 변환 행렬을 계산하였다.

외부 파라미터 획득을 위해, 먼저 동기화된 이미지와 포인트 클라우드 데이터 쌍을 저장하였다. 이후 스크립트의 시각화 도구를 통해 이미지와 포인트 클라우드에서 동일한 물리적 지점을 사용자가 직접 선택하여 대응점(correspondence)을 생성하였다. 이미지상에서 가까운 두 점이 실제 3D 공간에서는 깊이 축으로 멀리 떨어져 있을 수 있으므로, 다양한 각도와 거리에서 최소 4개 이상의 정확한 대응점 쌍을 신중하게 선택하였다. 이후 외부 파라미터 계산 스크립트는 이렇게 생성된 3D-2D 대응점 정보와 이전에 구한 내부 파라미터를 cv2.solvePnP 함수에 입력하여 외부 파라미터 행렬을 계산한다. 여러 개의 대응점을 구해 Overdetermined System을 구성하며, SOLVEPNP_ITERATIVE 플래그를 통해 반복적인 최적화 과정을 거쳐 Outlier의 영향을 최소화하고 가장 강건한 해를 도출할 수 있다.

이렇게 측정된 모든 캘리브레이션 값들은 prism 및 calico 패키지의 Calibration Manager 클래스를 통해 동적으로 로드되는 YAML 형식의 파일로 저장된다. 이를 통해 하드웨어의 미세한 변경이 있더라도 코드 수정 없이 캘리브레이션 파일만 교체하여 유연하게 대응할 수 있는 시스템을 구축하였다.

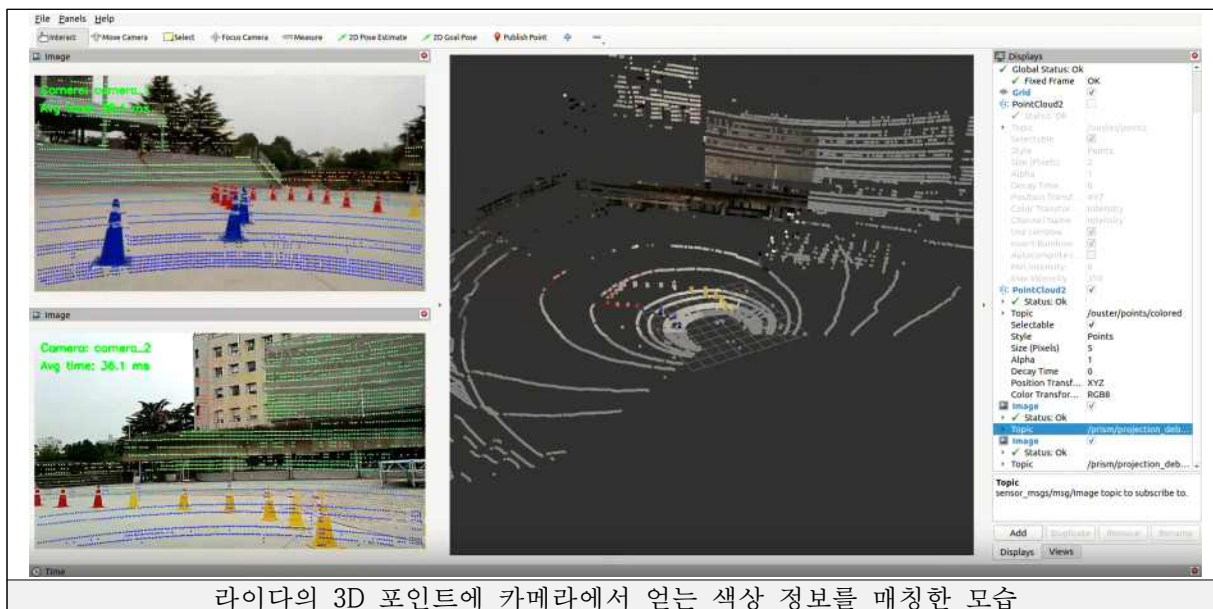


ii. Early-Fusion, 포인트 클라우드 컬러링

오브젝트 레벨 퓨전의 정확도를 높이고, 전체 주행 환경에 대한 풍부한 시각적 정보를 확보하기 위한 선행 단계로 포인트클라우드 컬러링을 수행하였다. 이는 LiDAR로부터 측정된 각 3차원 포인트에 카메라 이미지로부터 얻은 RGB 색상 정보를 부여하는 과정이다.

이 프로세스는 prism 패키지의 PrismFusionNode 내에서 핵심적으로 처리되며, 정밀하게 측정된 센서 캘리브레이션 파라미터를 기반으로 동작한다. 먼저 Projection Engine 클래스는 LiDAR 좌표계의 3D 포인트들을 transformPointsToCamera 함수를 통해 각 카메라의 좌표계로 변환한다. 그 후, projectPointsToImage 함수에서 카메라의 내부 파라미터와 왜곡 계수를 모두 고려하는 cv::projectPoints 함수를 호출하여, 변환된 3D 포인트를 2D 이미지 평면에 정확히 투영한다. 이 과정을 통해 각 3D 포인트에 해당하는 2D 픽셀 좌표를 계산할 수 있다. 마지막으로, ColorExtractor 클래스가 해당 픽셀 좌표의 RGB 값을 읽어와 원래의 3D 포인트에 색상 정보로 추가한다. 이때, 정수 단위의 픽셀 좌표가 아닌 서브픽셀 단위의 정밀도를 확보하기 위해, prism_params.yaml 설정 파일에 명시된 대로(interpolation: "bilinear") 투영된 좌표

주변 4개 픽셀의 색상 값을 가중 평균하는 Bilinear Interpolation을 적용하였다. 한 포인트가 여러 카메라에 투영될 경우, MultiCameraFusion 클래스는 신뢰도와 이미지 중심으로부터의 거리 등을 고려한 weighted_average 전략을 통해 최적의 색상을 결정한다. 이 기술을 통해 우리는 주변 환경 전체를 고밀도의 컬러 포인트클라우드로 복원할 수 있었으며, 이는 디버깅 및 시각화 과정에서 주행 상황을 직관적으로 이해하는 데 큰 도움을 주었다.



iii. Late-Fusion, IoU 기반 데이터 연관

포인트클라우드 컬러링이 환경 전체에 대한 정보를 생성하는 과정이라면, 오브젝트 레벨 퓨전은 LiDAR 기반 3D 콘 객체와 카메라 기반 2D 콘 객체를 정확하게 연결하여 최종적인 인지 결과를 도출하는 과정이다.

첫째로, 데이터 연관을 위해 LiDAR 인지 파이프라인에서 검출된 3D 콘 객체의 경계 상자를 각 카메라의 2D 이미지 평면으로 투영하였다. calico 패키지의 MultiIoUFusionNode는 project3DBBoxTo2D 함수를 통해 이 과정을 수행한다. 이 함수는 3D 바운딩 박스의 8개 꼭짓점을 이전에 획득한 외부 및 내부 파라미터를 이용해 카메라 좌표계로 변환한 후, 이미지 평면에 투영한다. 투영된 8개의 2D 점들을 모두 포함하는 최소 사각형을 계산하여 LiDAR 기반 3D 객체에 대한 2D 표현을 얻는다. 이 방식은 카메라 이미지에서 특정 객체가 3D 공간의 어디에 위치해야 하는지에 대한 강력한 기하학적 제약 조건을 제공하며, 복잡한 3D Frustum 연산을 2D 연산으로 대체하여 실시간성을 확보하는 효율적인 접근법이다.

둘째로, 이렇게 투영된 2D 박스와 카메라 이미지에서 YOLO로 검출된 2D 바운딩 박스 간의 데이터 연관을 수행하였다. 우리는 두 2D 박스 간의 중첩 정도를 정량적으로 평가하기 위해 IoU(Intersection over Union)를 계산하여 모든 LiDAR 콘과 모든 YOLO 감지 객체 간의 Cost Matrix를 생성한 후, HungarianMatcher 클래스에 구현된 헝가리안 알고리즘을 적용하여 총비용($1 - \text{IoU}$)을 최소화하는 최적의 매칭 쌍을 찾아냈다.

마지막으로, 매칭된 결과를 바탕으로 LiDAR 기반 3D 콘에 YOLO로부터 얻은 색상 클래스 정보를 최종적으로 부여하였다. 만약 두 대의 카메라가 동일한 LiDAR 콘에 대해 서로 다른 색상 정보를 할당하려 할 경우, mergeClassifications 함수 내에서 구현된 투표 기반의 충돌 해결 로직을 통해 가장 많은 표를 얻은 색상을 선택하도록 설계하였다. 이 과정을 통해 위치, 추적 ID, 그리고 정확한 색상 정보가 모두 포함된 3D 콘 배열이 완성되며, 이는 SLAM 파이프라인의 최종 입력으로 사용된다.



최종적으로 라이다의 콘 좌표와 카메라의 콘 클래스를 매칭하는 모습

3.2.8. SLAM(Simultaneous Localization and Mapping)

센서 퓨전을 통해 완성된 컬러 콘 정보는 차량이 피해야 할 장애물이자 주행 환경의 구조를 나타내는 핵심적인 랜드마크 역할을 수행한다. SLAM(Simultaneous Localization and Mapping) 파이프라인의 목표는 이러한 콘 랜드마크를 활용하여 트랙의 지도를 실시간으로 생성함과 동시에, 이 지도상에서 차량의 위치를 지속적으로 정밀하게 추정하는 것이다. 이를 통해 주행이 누적됨에 따라 발생하는 드리프트 오차를 효과적으로 보정하고, 전역적으로 일관된 주행 경로를 생성하고자 하였다. 우리는 높은 정확도와 유연성을 제공하는 그래프 기반 SLAM 방식을 채택하였다.

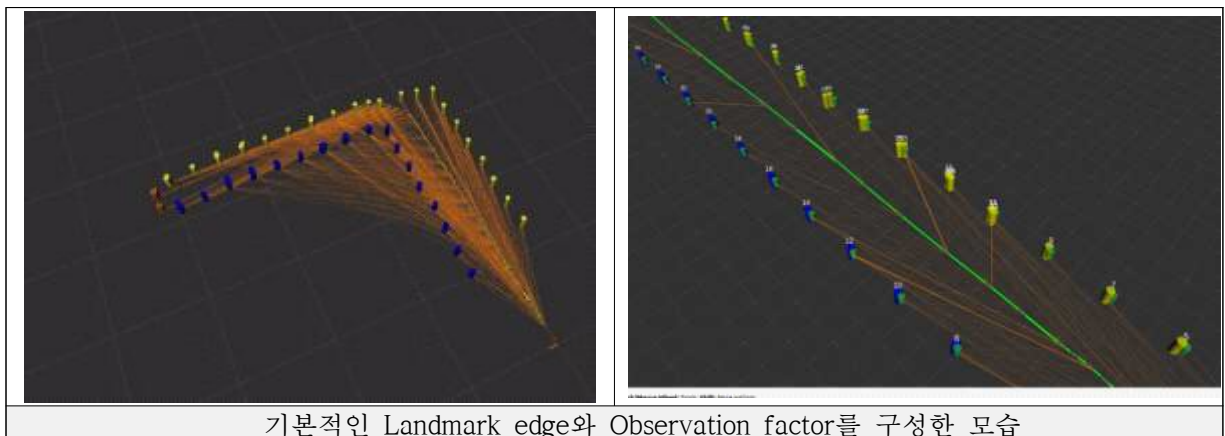
i. 팩터 그래프 기반 SLAM

우리 시스템은 최적화 라이브러리인 GTSAM을 기반으로 팩터 그래프를 구축하고 최적화하는 방식으로 동작한다. 팩터 그래프는 차량의 상태와 환경의 구조를 확률적인 네트워크로 모델링하는 기법으로, 다음과 같은 두 가지 핵심 요소로 구성된다.

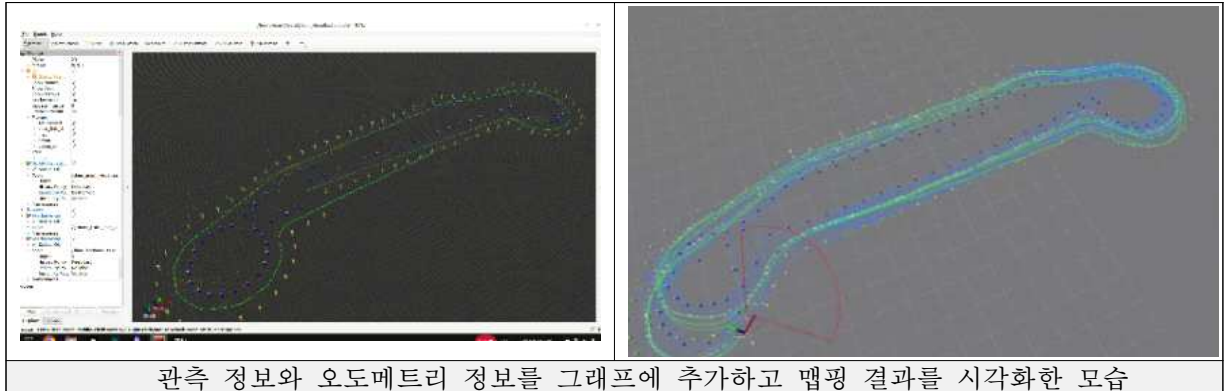
- 노드: 최적화하고자 하는 미지수들을 나타내며, 우리 시스템에서는 각 시간 스텝에서의 차량 위치(Pose)와 각 콘 랜드마크의 2D 지도상 위치(Point2)가 이에 해당한다.

- 팩터: 노드들 간의 관계, 즉 측정치로부터 얻어지는 제약조건을 나타낸다. 예를 들어, 차량의 오도메트리 정보는 연속된 차량 위치 노드 간의 관계를 정의하는 'Odometry Factor'로 표현되고, 특정 위치에서 콘을 관측한 정보는 차량 위치 노드와 콘 랜드마크 노드 간의 관계를 정의하는 'Cone-Observation Factor'로 표현된다.

새로운 관측 정보가 들어올 때마다 그래프에 새로운 노드와 팩터가 추가되며, GTSAM은 비선형 최소제곱법을 통해 모든 팩터의 제약조건을 가장 잘 만족시키는 노드 값들의 최적해를 계산한다. 이 과정을 통해 맵과 차량의 위치가 지속적으로 갱신되고 정제된다.



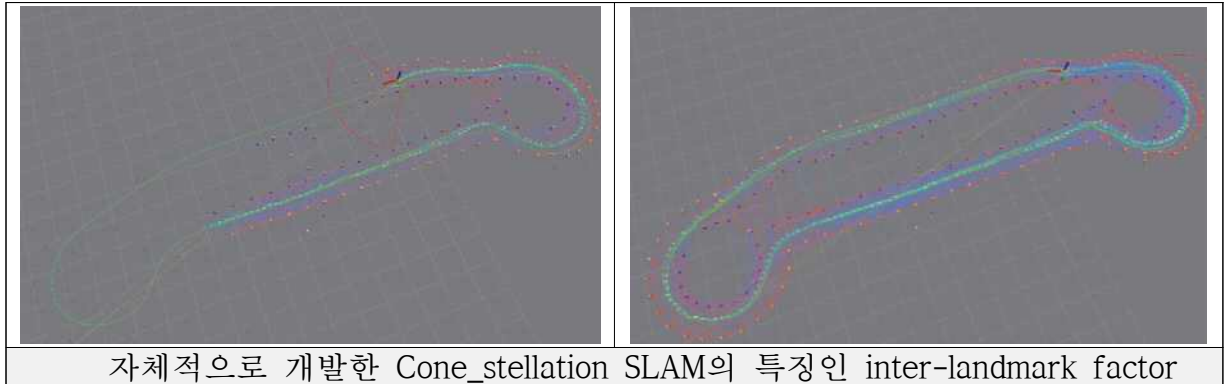
ii. 강건한 맵 생성을 위한 랜드마크 관리



실제 주행 환경에서는 False Positive나 일시적인 인지 실패가 발생할 수 있다. 부정확한 랜드마크 정보가 SLAM 그래프에 추가될 경우, 전체 맵의 정확성에 심각한 영향을 미칠 수 있다. 이를 방지하기 위해 우리는 'Tentative Landmark'라는 개념을 도입하여 랜드마크를 강건하게 관리하였다.

새로운 콘이 처음 관측되면 즉시 맵에 추가하지 않고, '임시(Tentative)' 상태로 등록하여 여러 프레임에 걸쳐 추적을 시작한다. 해당 랜드마크가 정식으로 맵에 추가되기 위해서는 다음과 같은 데이터 기반 Multi-criteria Promotion 조건을 모두 만족해야 한다. 첫째, 최소 관측 횟수(min_observations) 이상 안정적으로 관측되어야 한다. 둘째, 첫 관측 시점과 마지막 관측 시점 간의 시간 간격(min_time_span)이 일정 수준 이상이어야 한다. 이는 차량이 이동하는 동안에도 해당 랜드마크가 일관되게 관측됨을 보장한다. 셋째, 위치 추정치의 공분산(max_position_variance)이 일정 값 이하로 수렴하여 위치 정확도가 확보되어야 한다. 마지막으로, 여러 번 관측된 색상 정보의 일관성(min_color_confidence)이 높아야 한다. 이처럼 체계적인 검증 전략을 통해 일시적인 노이즈나 오탐지가 맵을 오염시키는 것을 효과적으로 방지하고, 높은 신뢰도를 가진 랜드마크들로만 맵을 구성할 수 있었다.

iii. 기하학적 제약조건 및 루프 클로저



정확하고 안정적인 SLAM 성능을 확보하기 위해, 우리는 기본적인 오도메트리와 관측 팩터 외에 추가적인 기하학적 제약조건을 적극적으로 활용하였다.

- Inter-Landmark Factor: 두 개 이상의 콘이 동일한 프레임에서 동시에 관측되었을 때, 그 콘들 사이의 상대적인 거리와 방향은 변하지 않는다는 강력한 기하학적 제약조건이 발생한다. 우리는 이 콘들 간의 상대 위치 관계를 'Inter-Landmark Factor'로 정의하여 그래프에 추가하였다. 이 팩터는 맵의 기하학적 강성을 크게 향상시켜 적은 수의 랜드마크만으로도 맵의 형태가 쉽게 무너지지 않도록 하는 핵심적인 역할을 수행했다.

- 루프 클로저: 차량이 이전에 방문했던 장소를 재방문했을 때 이를 인식하고, 과거와 현재의 관측 정보를 연결하여 누적된 오차를 한 번에 보정하는 과정이다.

우리는 라바콘의 배치에서 ‘별자리’라는 단어를 연상해 착안한 ‘Cone_stellation’라는 개념의 디스크립터를 개발하여 루프 클로저를 검출하였다. 이는 특정 위치에서 관측되는 주변 콘들의 기하학적 배치 패턴(거리, 각도, 색상 분포 등)을 하나의 고유한 특징 벡터로 표현한 것이다. 현재 프레임에서 생성된 디스크립터가 과거에 저장된 디스크립터와 일정 수준 이상 유사할 경우 루프 클로저가 발생했다고 판단하고, 두 시점의 맵을 정합하여 팩터 그래프에 강력한 제약조건을 추가하였다. 이를 통해 장거리 주행 시 발생하는 누적 오차를 효과적으로 해소하고, 전역적으로 일관된 맵을 구축할 수 있었다.

3.3. 판단

3.3.1. 경로 계획

경로 계획의 목표는 인지된 라바콘의 배치 정보로부터 차량이 따라가야 할 이상적인 주행선을 생성하는 것이다. 우리는 이를 장기적인 기준선을 제시하는 전역 경로와 실시간 환경 변화에 대응하는 지역 경로로 나누어 처리하였다.

i. 전역 경로 계획

전역 경로 계획 단계에서는, GPS-IMU 기반 기준 경로를 지도 좌표계(map) 상의 직교 좌표로 변환하고, 위치 추정 공분산을 색으로 표현한 경로 시각화를 통해 변환한 경로 및 신뢰도를 함께 제공한다. 파이프라인은 두 단계로 구성된다. 첫째, 실시간 GPS 고정(Position Fix)을 기준점(Ref Lat/Lon) 대비 로컬 평면 좌표계(ENU 근사)로 변환해 /local_xy를 퍼블리시한다. 둘째, 사전 취득한 경로 CSV의 위·경도와 위치 공분산을 동일 기준으로 투영해 정확도에 따라 색이 변하는 글로벌 경로를 생성한다. 운영 좌표계와 프레임은 map-base_link를 사용하며, 다운스트림 태스크(경로 추종·모션 제어)가 그대로 이용할 수 있도록 ROS 2 토픽 인터페이스로 표준화하였다.

좌표 변환은 지구 곡률이 작은 지역 범위라는 가정에서 구형 지구 반경 R 을 이용한 등각 보정 평면 근사로 처리하였다. 기준 위도·경도를 중심으로

$$x = R(\lambda - \lambda_0)\cos\phi_0, \quad y = R(\phi - \phi_0)$$

로 정의하고, 라디안 단위 변환을 거쳐 실시간으로 x, y 를 산출한다. 메시지 헤더는 원본 GPS의 타임스탬프를 유지해 다운스트림 파이프라인에서 발생할 수 있는 동기화 오차를 최소화하였다. 실측 환경에서는 기준점이 코스 중앙부 근처일 때 수 m 수준의 바이어스 없이 안정적으로 동작했고, 일반 주행 속도(3-7 m/s)에서 변환 계산 지연은 무시 가능한 수준(오차 0.1 ms 이하)로 관찰되었다.

기준이 될 전역 경로는 직접 주행한 데이터를 기반으로 구축한다. 이를 위해 차량을 수동으로 주행하거나, 첫 바퀴를 안정적인 속도로 자율주행하여 CSV 형식으로 기록한다. 이후, 과도한 점 밀집으로 인한 시각적·계산적 부담을 줄이기 위해 최소 간격 필터(기본 0.3 m)를 적용한다. 이후 공분산 임계값(기본 2×10^{-4})을 기준으로 임계 초과 구간은 적색(정확도 낮음), 이하 구간은 녹색(정확도 양호)으로 색을 부여한 LINE_STRIP 마커를 발행한다. 이때 색 배열을 점마다 개별 지정함으로써 동일 선분 내에서도 신뢰도 변화를 연속적으로 표현한다. 프레임은 map, 주기는 2Hz로 설정하여 RViz 상에서 경로·신뢰도 지도를 동시에 확인할 수 있다.

실 차량을 이용한 실험에서는 기준 경로 CSV 데이터(약 3.1 km, 12,842점)를 최소 간격 필터를 적용해 약 6,200점으로 줄였다. 이를 통해 RViz에서 지연 없이 부드럽게 시각화할 수 있었다. 위성 신호가 약한 구간(나무나 건물 근처)에서는 위치 공분산 값이 커져 경로가 적색으로 표시되었고, 탁 트인 개활지에서는 녹색으로 안정적으로 표현되었다. 실제 주행에서는 이러한 신뢰도 정보를 활용해 속도 계획을 보수적으로 조정할 수 있었으며, 그 결과 플래닝을 위한 콘 레이블링 성능이 소폭 개선되는 효과를 확인하였다.

ii. 지역 경로 계획

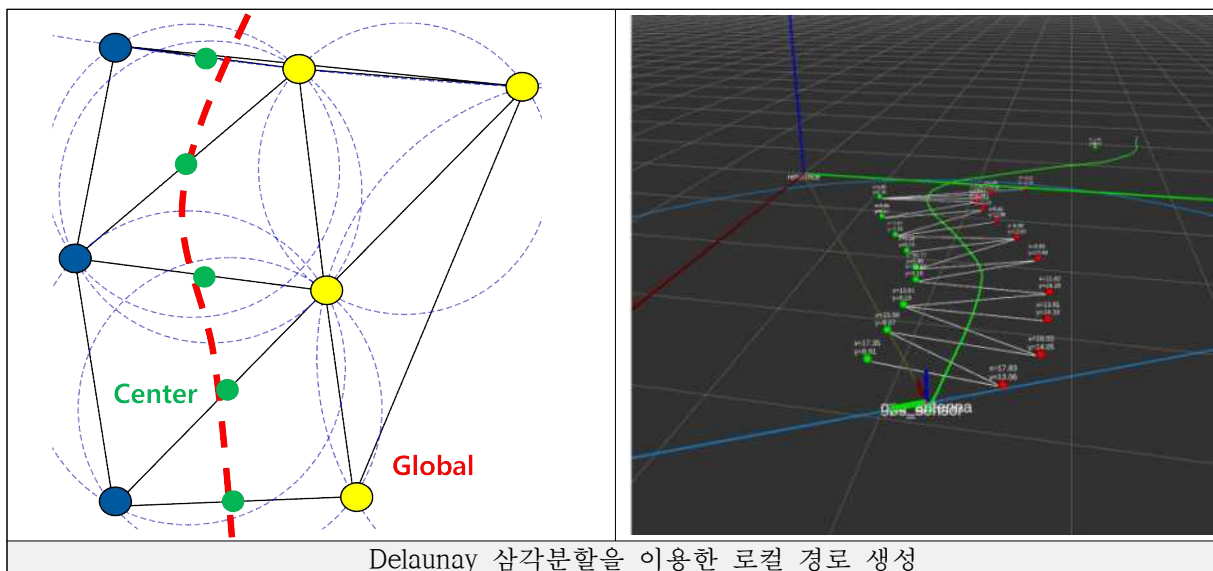
지역 경로 계획은 차량 좌표를 원점(0,0)으로 설정하여 실제 주행 경로를 생성하는 핵심 과정이다. 이는 앞서 설명한 전역 경로 계획 정보를 활용하는 방식과 활용하지 않는 방식으로 나눌 수 있으며, 공통적으로 좌우 콘을 안정적으로 회피할 수 있는 규칙 기반 경로 생성을 목표로 한다.

ii.a. 삼각 분할 및 전역 경로 활용

첫 번째 방식으로는 실시간으로 인지된 콘들을 Delaunay 삼각분할하여 그래프 모델링을 한 뒤, 좌우 구분하여 중점들을 경로로 사용하는 방식이다. 여기서 Delaunay 삼각분할이란 공간에 흩어져 있는 점들에 대해 각각의 점들을 연결해 삼각형을 만드는 것이 목적인데, 이때 만들어진 삼각형의 외접원 안에 다른 점이 포함되지 않도록 하여 점들을 연결하고 이를 통해 공간을 분할하는 방식이다. 이 방법을 사용하게 되면 각각의 점들에 대해 자연 이웃 관계를 얻을 수 있어 경로를 생성할 때 단순히 반대쪽 점들을 선택하는 것보다 더 많은 유효한 점들과 연결할 수 있다. 이를 통해 곡률이 큰 구간 혹은 콘 일부가 인식되지 않은 상황 등에서 더 강건하게 경로를 생성할 수 있다.

하지만 결국, 인식된 콘들에 대한 좌우 콘 구분을 진행해야 하는데, 여기서는 전역 경로를 참조선으로 활용하여 좌우 구분을 수행한다. 전역 경로를 중심 축으로 두고, 각 콘이 해당 참조선의 좌측에 위치하는지, 우측에 위치하는지를 판별함으로써 초기 좌·우 시드를 설정한다. 이후 전역 경로의 곡률과 진행 방향을 따라가며 인접한 콘들을 확장해 나가는데, 직선 구간에서는 종방향(x축) 임계값을 넓게 설정해 멀리 있는 콘까지 포함하고, 곡률이 큰 구간에서는 횡방향(y축) 임계값을 넓게 적용해 참조선 주변의 콘을 우선적으로 포함시킨다. 이를 통해 좌우 콘 구분은 전역 경로와 강하게 일관성을 유지하며, 실제 주행에서 안정적이고 신뢰도 높은 경로를 생성할 수 있다.

좌우 콘 구분을 진행한 이후, 삼각분할을 통해 생성된 삼각형들의 변들에 대해 좌측 콘과 우측 콘을 잇는 변이면서 사전에 정의한 트랙의 폭 이내의 변들만 유효한 변으로 추출한다. 이후 추출한 각 변의 중심점들을 통해 트랙 중심의 웨이포인트를 획득하고 다시 이 웨이포인트들을 차량으로부터의 거리에 따라 정렬한 후 B-Spline 보간을 통해 부드러운 경로를 생성하여 후속 행동 계획 및 제어 계층에서 사용될 수 있도록 한다.



ii.b. Potential Field Path Plan

두 번째 방식은 장애물을 중심으로 퍼텐셜 필드를 생성한뒤, 이를 활용하여 경로 생성을 수행하는 방식이다.

퍼텐셜 필드는 공간의 각 지점 $x \in \mathbb{R}^n$ 에 실수값을 대응시키는 스칼라장 $U: \mathbb{R}^n \rightarrow \mathbb{R}$ 으로, 주행 환경의 잠재량을 수치로 표현한 것이다. 이 필드에서의 기울기를 통해 가상의 힘을 정의할 수 있으며, 이를 이용하면 특정 위치 x 에서 필드의 국소적인 진행 방향과 세기 $F(x) = -\nabla U(x)$ 를 정의할 수 있다. (벡터 장)

본 경로 생성 방법에서는 퍼텐셜 필드의 성질을 효과적으로 활용하기 위해서 정의된 공간의 각 콘의 위치에 스칼라값 $U(x, y)$ 를 부여하되, 가우시안을 통해 Blurry 한 스칼라값을 부여한다. 즉, 인지된 콘 좌표를 $(x_i, y_i)_{i=1}^N$ 라고 할 때, 각 콘을 중심으로 한 가우시안의 합을 필드로 구성하여 3차원 공간을 형성한다.

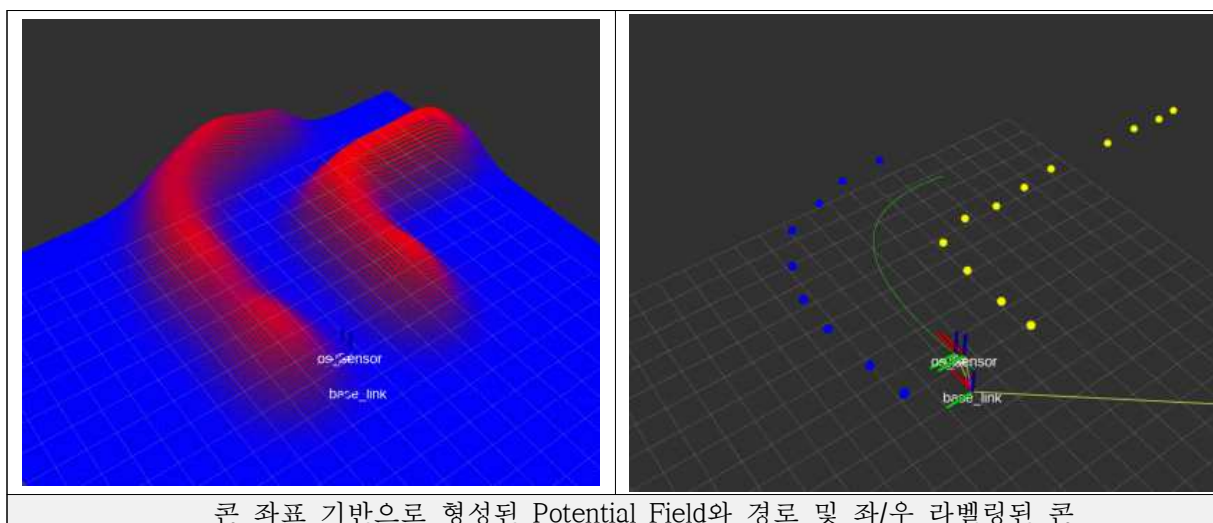
$$U(x, y) = \sum_{i=1}^N A \cdot \exp(-(x - x_i)^2 + (y - y_i)^2 / 2\sigma^2)$$

이때 A 는 진폭, σ 는 표준편차로, 정의된 격자 내 모든 셀 (x_j, y_k) 에 대해 위의 값을 누적 계산한다.

그다음 형성된 퍼텐셜 필드를 기반으로 경로를 추출하기 위해, 먼저 격자망을 설정하고 각 셀의 값을 주변 8방향 이웃과 비교한다. 이때 중심 값이 이웃 값보다 작은 경우가 여러 쌍에서 확인되면 해당 지점을 “골(trough) 후보”로 채택한다. 즉, 퍼텐셜 지형에서 국소적으로 낮은 골짜기 부분만을 이산적인 규칙으로 추려내는 것이다.

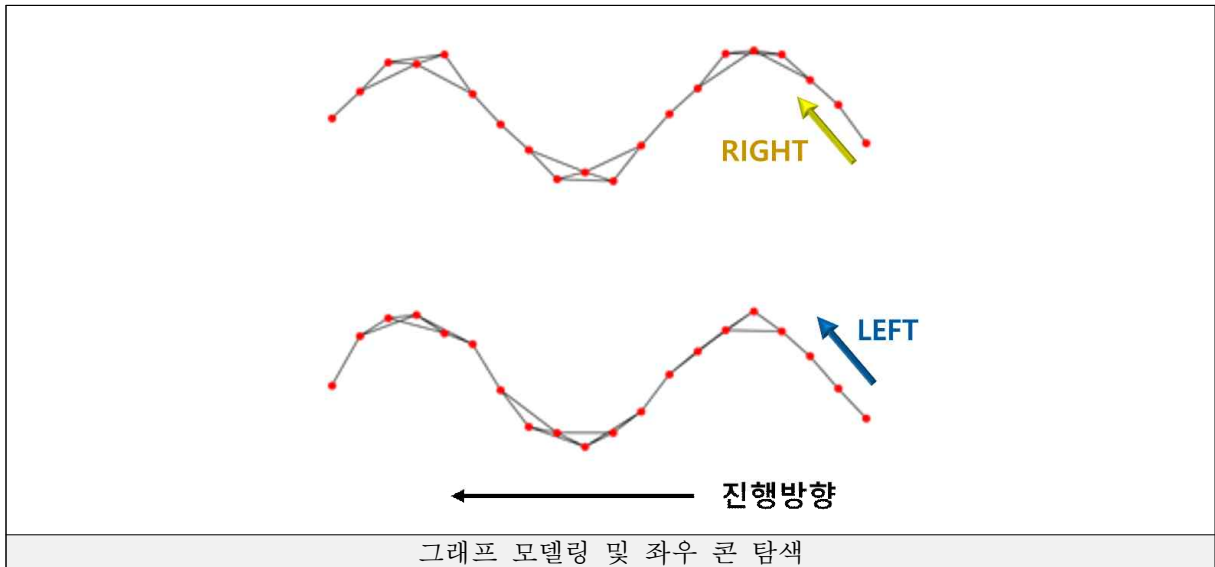
채택된 후보들 중에서 단순히 낮은 지점만 고르는 것이 아니라, 차량의 진행 방향과 일치하는 정도(전방 단위벡터와의 내적)를 평가하여 일정 임계값 이상인 지점만 남긴다. 이후 차량과 가장 가까운 후보를 시작점으로 선택하며, 만약 이 시작점이 차량에서 지나치게 멀리 떨어져 있다면 경로 생성을 중단한다.

경로 탐색은 “현재 점에서 가장 가깝고 전방으로 진행하는” 후보를 선택하는 그리디 탐색 방식으로 수행된다. 이 과정에서 점 간 최대 거리를 제한하여 급격한 점프나 후퇴를 방지한다. 이렇게 얻어진 점들은 B-스플라인으로 보간하여 부드러운 곡선 경로로 변환하며, 최종적으로는 호 길이 기준으로 등간격 리샘플링을 적용해 주행 제어기에 바로 활용 가능한 형태의 경로를 완성한다.



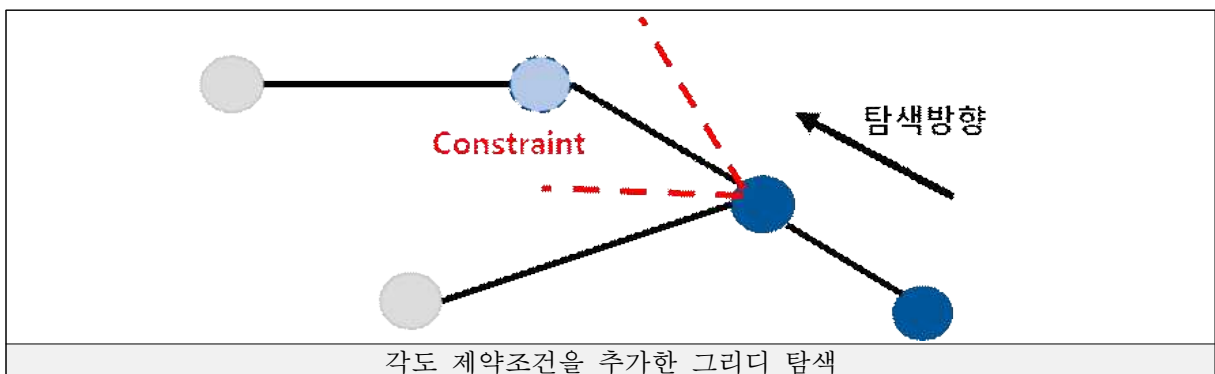
ii.c. 각도 제약 조건을 활용한 경로 생성

세 번째 경로 생성 방식으로는 부모 노드와의 각도 제약 조건을 활용한 경로 생성 방식을 적용하였다. 먼저, 미분류된 콘 좌표를 구독하여 좌·우로 구분하기 위해, 모든 콘의 (x, y) 좌표를 그래프 형태로 모델링한다. 이때 그래프 형태로 나타내기 위해서는 ii.a에서와 같이 Delaunay 삼각 분할 방식을 활용해도 되나, 복잡도를 낮추기 위해, 퍼텐셜 필드와 마찬가지로 8방향 그리드 기반의 그래프 모델링 방법론을 적용하였다. 이때, 도로 폭 값을 Threshold로 두어, Threshold보다 긴 간선은 배제하는 식으로 모델링을 수행했다.



좌우 구분을 위한 시작점으로써, 차량과 가까운 좌우 양쪽 점을 선택한다. 초기 탐색 지점을 선택할 때에는 1) 규칙 기반으로 좌우 각각 가까운 점을 우선으로 하거나, 2) 카메라 정보를 통해 알아낸 색 정보를 활용하여 초기값을 정하는 방법 모두 활용 가능하다.

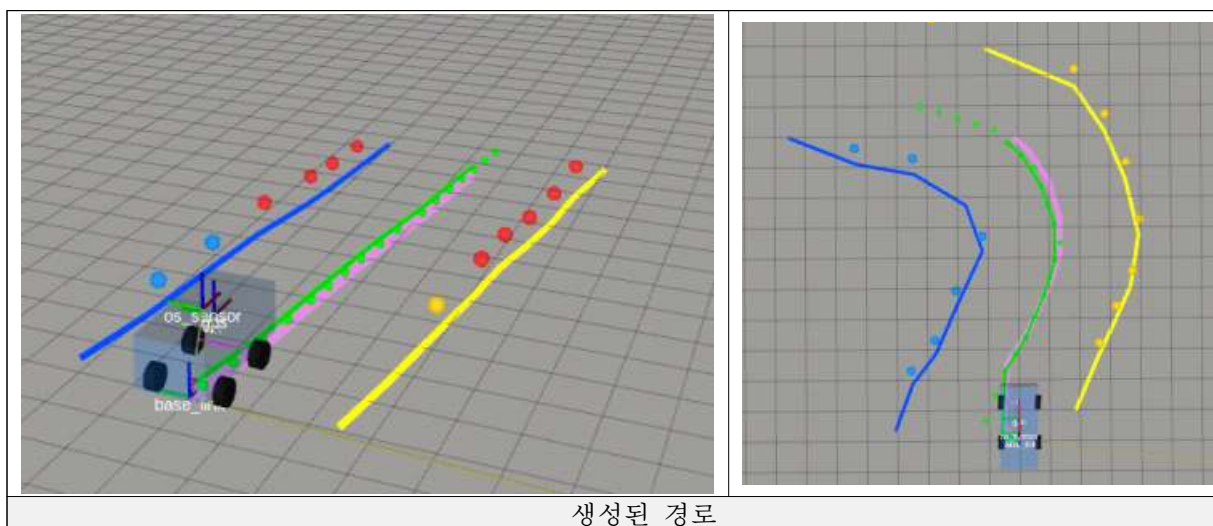
선택된 시작점에서 인접 노드로 전진하며 좌우벽을 구성한다. 전진 과정에서 부모 노드 방향 벡터 $\hat{p}$ (이전 점 - 현재 점 벡터), 조부모 노드 방향 벡터 $\hat{g}_p$ (두 단계 전 - 현재 점 벡터)를 이용하여, 최대 허용 각도 제약 조건을 걸며 그리디하게 탐색한다.



위 과정이 끝나고 나면, Vectorized 된 좌우 벽을 얻을 수 있으며, 연속된 선분들의 정보를 직접 활용하여 중앙 경로를 보다 단순하게 도출할 수 있다. 일반적인 콘 매칭 기반의 방식들은 좌우 콘을 개별적으로 매칭해야 하므로 헝가리안 알고리즘이나 투영(projection) 기반 매칭과 같은 추가 연산이 필요했다.

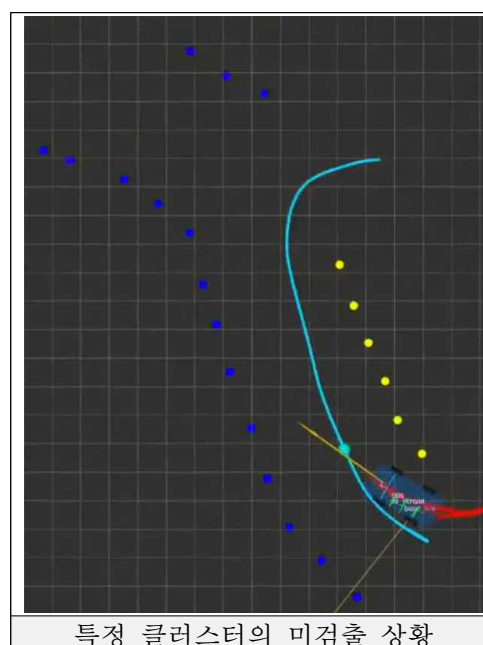
반면 벽을 선분 형태로 확보하면, 좌우 벽이 동일한 구간에서 대응될 수 있도록 좌우를 일정 개수로 리샘플링한 뒤 중점을 계산함으로써 중앙 경로를 얻을 수 있다.

이 접근법은 별도의 매칭 절차를 생략해 계산 효율을 높이고, 벽 라인이 연속적으로 유지되는 구간에서는 더욱 안정적이고 일관된 중앙 경로를 제공한다. 결과적으로 중점 기반 플래닝 과정을 단순화하면서도 실제 주행 환경에서 신뢰도 높은 경로 생성을 가능하게 한다.



ii.d. 경로 생성 방식의 비교

위 3개의 경로 생성 방식은 각각의 장단점이 뚜렷하게 나타났다. 첫 번째 방식(삼각 분할 및 전역 경로 활용)은 가장 성능이 우수했으나, 전역 경로 정보에 의존해야 하고, 차량의 위치나 자세 추정 정확도가 확보 되지 않으면 불안정하다는 한계가 있었다. 두 번째 방식(퍼텐셜 필드 방식)의 경우, 장애물을 자연스럽게 회피하도록 유도하는 좋은 품질의 경로를 만드는데 아주 효과적이었으나, 가려짐(occlusion)이나, 클러스터 미 검출(고속 환경) 등의 이유로 인해, 특정 콘이 감지 되지 않을 경우, 경로 이탈 현상이 간혹 발생했다.



3번째 방식은 전반적으로 안정적이고, 노이즈에 강건하다는 장점이 있었다. 다만 파라미터 튜닝이 까다롭고, 도로 폭 정보나, 그래프 민감도 설정(그리드 사이즈)에 성능이 크게 의존하는 단점이 있었다. 그럼에도 조건을 잘 맞췄을 때 가장 우수한 성능을 보였기에 최종적으로 ii.c 방식을 채택하였다.

방식	장점	단점	종합 평가
ii.a) 삼각분할 + 전역경로	- 성능 가장 우수 - 곡선/누락에도 강건	- 전역 경로 필요 - 위치/자세 추정 정확도 요구	전역 경로의 활용 전략을 잘 세우면 효과적일 것으로 예측 됨
ii.b) 퍼텐셜 필드	- 자연스러운 회피 경로 생성 - 구현 직관적	- Occlusion 및 클러스터 미검출 시 이탈	일반적으로 활용하기 어려움
ii.c) 각도 제약 그래프	- 노이즈에 강건 - 일반 상황에서 안정적	- 파라미터 튜닝 어려움 - 도로 폭/그리드 설정 민감	일반적으로 활용이 가능, 지속적인 튜닝 및 검증이 필요

3.3.2. 속도 계획

속도 플래닝은 자율주행 차량의 안전성과 주행 효율성을 동시에 확보하기 위해 꼭 필요하다. 단순히 경로만 따라가는 것이 아니라, 주행 환경의 곡률에 맞춰 적절한 속도를 유지해야 차량이 안정적으로 코너를 돌고, 제동 거리 안에서 멈출 수 있으며, 승차감도 보장된다. 곡률이 큰 구간에서는 속도를 줄여 차량이 미끄러지거나 전복되는 위험을 줄이고, 직선 구간에서는 속도를 높여 전체 주행 시간을 단축할 수 있다. 즉, 속도 플래닝은 경로 추종 성능과 차량 동역학적 한계 사이의 균형을 맞추는 핵심 요소로서, 실제 자율주행 시스템에서 경로 계획과 제어를 연결하는 다리 역할을 한다고 할 수 있다. 이에 따라 두 가지 방식의 속도 플래닝을 구현하였다.

i. 고·중·저속 삼분법 속도 플래너

본 단계의 속도 플래닝은 경로 곡률을 간단히 평가하여 고속(HIGH)-중속(MID)-저속(LOW) 세 구간 중 하나를 선택하고, 가감속 한계 안에서 목표 속도로 수렴시키는 삼분법(zone-based) 방식으로 구성하였다. 파이프라인은 (i) 경로 등간격 샘플링 및 곡률 추정-대표 곡률 산출, (ii) 구간 판정-램핑-속도/RPM 퍼블리시의 두 단계로 나뉜다. 구현은 20Hz 주기로 동작하며, RViz 마커를 통해 곡률 프로파일을 시각화하여 튜닝 가시성을 확보하였다.

(i) 경로 등간격 샘플링과 곡률 추정

로컬 경로에 대해 누적 호 길이 s 를 계산하고, 샘플 간격 Δs 로 균일 간격의 점을

선형 보간하여 생성한다. 각 샘플 P_i 에서 이웃점 (P_{i-1}, P_i, P_{i+1}) 로 이루는 삼각형의 외접원 곡률을 다음과 같이 근사한다.

$$\begin{aligned} a &= \|P_i - P_{i-1}\|, b = \|P_{i+1} - P_i\|, c = \|P_{i+1} - P_{i-1}\| \\ s_\Delta &= \frac{1}{2}(a + b + c), A = \sqrt{\max s_\Delta (s_\Delta - a)(s_\Delta - b)(s_\Delta - c)}, 0, \\ x_i &= \frac{4A}{abc} (\text{면적 } A \rightarrow 0 \text{이면 } x_i = 0) \end{aligned}$$

경로 시작부의 강제 연결에 따른 잡음을 줄이기 위해 선두 샘플 N_{ignore} 개에 대해서는 $x_i = 0$ 으로 처리하고, 이동 평균 윈도우 W 로 1차 평활화하여 $\tilde{x}_i$ 를 얻는다.

(ii) 대표 곡률(Preview) 산출

구간별 목표 속도를 $v^* \in v_H, v_M, v_L$ 로 정의하고, 샘플링 주기 Δt 에서 가속/감속 한계 a_{acc}, a_{dec} 로 선형 램핑하여 $v_{k+1} = v_k + \text{clip}(v^* - v_k, -a_{dec}\Delta t, a_{acc}\Delta t)$ 로 계산한다.

최종 v_{k+1} 는 /cmd/speed로 발행되면, 옵션으로 모터 RPM을 아래 변환으로 산출하여 /cmd/rpm으로 발행한다(감속비, 타이어 둘레 기반 상수 $c_v \approx 0.00535 \text{ m/s per RPM}$)

$$RPM_{motor} = \frac{v}{c_v} \approx \frac{v}{0.00535}$$

삼분법적 속도 플래닝은 구현이 단순하고 실시간성이 우수하며 튜닝이 직관적이라는 장점이 있으나, 속도가 이산 구간으로만 변해 연속적인 $v(x)$ 맵 대비 세밀한 조정이 어렵고, fixed-window에서 계산된 x_{rep} 이 정지거리 · 가시거리 · 제한속도 · 장애물 등 여러 제약을 체계적으로 반영하지 못하며, 차량 동역학(마찰 원, 하중 이동)과 중 · 횡 결합 제약(코너 진입 제동/가속 여유)의 보장에 한계가 있었다. 이러한 제약으로 인해, 다음 절에서 곡률과 동역학 제약 기반의 속도 계획 구현사항을 다룬다.

ii. 곡률 · 동역학 제약 기반 속도 플래너

i. 에서의 속도 계획의 한계를 극복하기 위해서는 해당 플래너는 로컬 경로와 현재 차량 속도를 입력으로 받아, 곡률에 따른 횡가속도 한계와 종방향 가감속 제약을 동시에 만족하는 속도 프로파일을 계산하고, 운용 시점마다 미리보기 기반의 목표 속도(m/s)와 이에 대응하는 모터 회전수(RPM)를 출력한다. 목표는 곡선부 감속과 직선부 가속이 자연스럽게 이어지는 안전 · 안정 주행이며, 노이즈에 강하고(EMA 기반) 명령 변화율이 과도하지 않도록(rate limit) 제어 신호를 평활화하는 것이다.

ii.a. 곡률 기반 속도 상한

경로는 필요 시 전방 가시거리로 제한하고, 세 점 A, B, C의 기하적 특징을 이용해 곡률을 근사한다. 삼각형 ABC의 두 배 면적을 $2A$, 세 변의 길이를 a, b, c 라 할 때, 수치 안정화 상수 $\epsilon (\epsilon > 0)$ 와 함께

$$\kappa \approx \frac{2A}{abc + \epsilon}$$

로 두고, 고주파 요동은 이동평균필터로 완화한다. 허용 횡가속도 $a_y^{\max}$ 가 주어지면 곡률 제한 속도 상한은 아래와 같다.

$$v_k = \sqrt{\frac{a_y^{\max}}{|\kappa| + \epsilon}}$$

최종 상한은 설정 최대속도 $v_{\max}$ 와의 최소값으로 정한다. 이렇게 얻은 상한을 기준으로 지난 경로에서 $v_i \leq \sqrt{v_{i-1}^2 + 2a_{acc}\Delta s_{i-1}}$ 를 적용해 가속 한계를 보장하고, 지날 경로에서 $v_i \leq \sqrt{v_{i+1}^2 + 2a_{brk}\Delta s_i}$ 를 적용해 제동 한계를 만족시킨다. 두 제약사항 적용이 끝난 배열이 곡률 · 가감속 제약을 모두 충족하는 속도 프로파일이며, 이는 시각화/디버깅을 위해 그대로 발행된다.

ii.b. 미리보기 기반 목표 속도 생성

실시간 목표 속도는 현재 필터링된 속도 $v_{current}$ 에 시간 미리보기 t_p 를 적용해 거리 영역으로 맵핑한 $s_{cmd} = clip(s_{\min} + t_p v_{current}, s_{\min}, s_{\max})$ 을 기준으로, 누적 호 길이 배열에서 선형 보간하여 원식 목표 속도 v_{raw} 를 얻는다. 이후 변화율 제한으로 $|v - v_{prev}| \leq \dot{v}_{\max} \Delta t$ 를 먼저 보장하고, 지수이동평균필터로

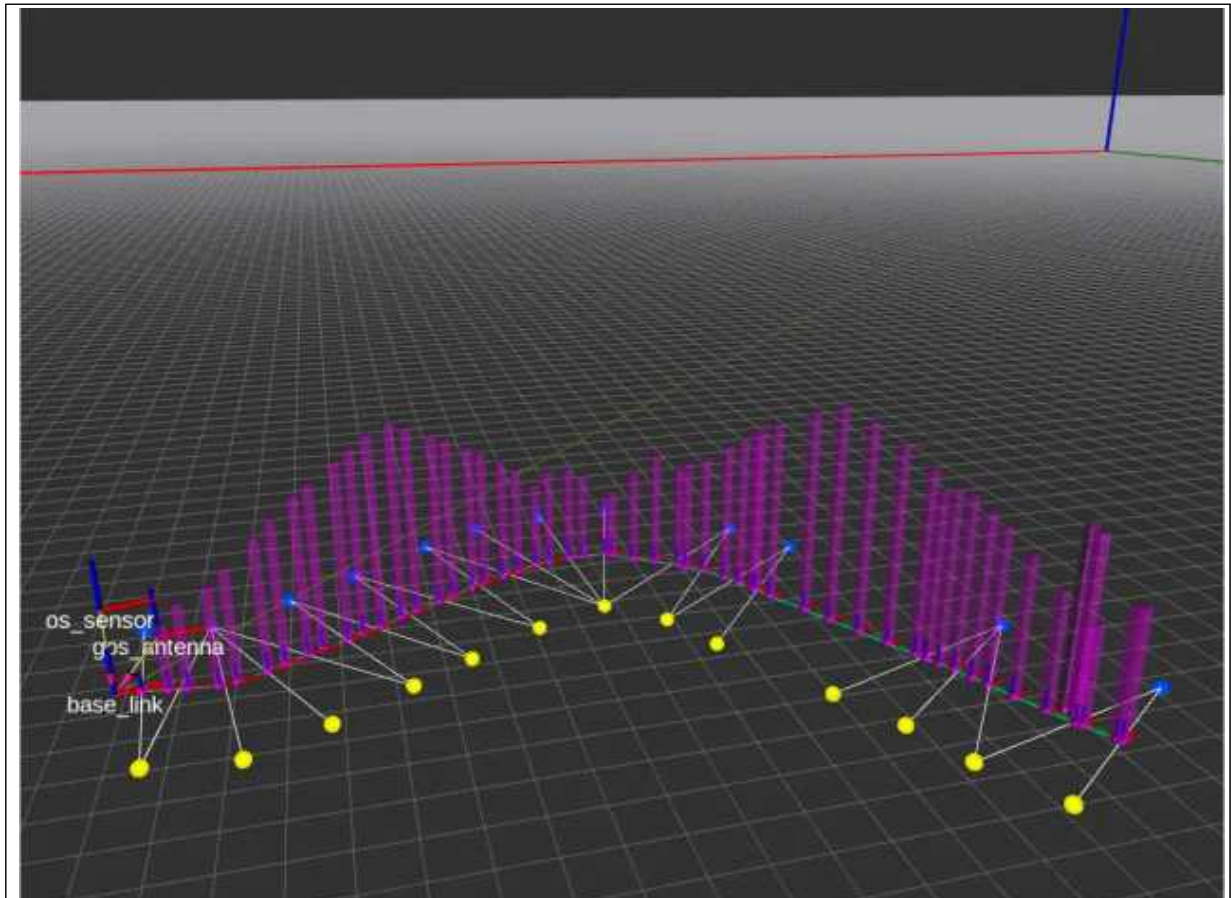
$$v_{cmd} = \alpha v_{prev} + (1 - \alpha) v_{limed}, \quad \alpha = e^{-\Delta t / \tau_{cmd}}$$

을 적용해 급격한 명령 변화를 억제한다. 최종 명령은 $[v_{\min}, v_{\max}]$ 범위로 클리핑되어 /cmd/speed로 출력된다. 실측 속도 자체도 별도의 EMA로 노이즈를 저감해 미리보기 거리의 불필요한 진동을 방지한다.

i, ii방식 모두 저수준 제어에게 속도를 넘겨주기 이전에 직접 모터를 구동하기 위해 RPM 값을 발행하기 위해 아래의 과정을 거친다.

최종 v_{k+1} 는 /cmd/speed로 발행되면, 옵션으로 모터 RPM을 아래 변환으로 산출하여 /cmd/rpm으로 발행한다(감속비, 타이어 둘레 기반 상수 $c_v \approx 0.00535m/s \text{ per RPM}$)

$$RPM_{motor} = \frac{v}{c_v} \approx \frac{v}{0.00535}$$



속도 플래닝된 속도 프로파일 시각화

3.4. 고수준 제어

모션 제어 계층은 상위 계층이 생성한 지역 경로와 현재 속도를 입력으로 받아 차량이 경로를 안정적으로 따르도록 최종 조향각을 산출한다. 우리는 전통적인 Pure Pursuit를 실제 주행 환경에 맞게 확장한 적응형 Pure Pursuit와 Model Predictive Control(MPC)을 적용하여 성능을 비교하였다. 비교의 기준선(baseline)으로는 속도에 따라 룩어헤드 거리를 가변화한 동적 Look Ahead Distance(이하 L_d) Pure Pursuit를 사용하였다.

3.4.1. 경로 추종

모션 제어 계층은 상위 계층에서 생성된 지역 경로와 현재 속도를 입력받아, 차량이 경로를 안정적으로 추종하기 위한 최종 조향각을 계산하는 역할을 한다. 우리는 전통적인 Pure Pursuit 알고리즘을 실제 주행 환경에 맞게 대폭 개선한 적응형 Pure Pursuit 알고리즘을 적용하였다.

i. Pure-Pursuit 알고리즘

본 단계에서는 차량 후륜축을 기준으로 경로상의 전방 기준점을 추종하는 기하학적 방식의 Pure Pursuit를 도입하였다. 핵심은 차량 속도에 비례해 룩어헤드 거리(L_d)를 조정하여 저속에서는 민첩하게, 고속에서는 조향 변동을 줄여 안정적으로 움직이도록 하는 것이다. 구현은 50 Hz 제어 루프에서 동작하며, 실측 속도와 로컬 경로를 입력으로 받아 먼저 L_d 를 계산하고, 차량 좌표계 기준 전방($x > 0$)에 위치한 경로점 가운데 L_d 이상 떨어진 최초 지점을 목표점으로 선택한다. 이후 기하학적 관계를 통해 조향각을 산출하고 물리 한계($\pm 30\text{deg}$)를 적용한 뒤, EMA 필터로 급격한 명령 변화를 완화하여 최종 명령을 출력한다. 룩어헤드 목표점은 RViz 마커로 실시간 표시하여 튜닝 과정의 가시성을 확보하였으며, 경로가 비거나 유효 목표점이 없을 때에는 안전한 정지 명령 또는 완만한 감속 명령으로 폴백하도록 했다.

동적 L_d 의 구현은 저속 구간의 기본적인 코너링 성능을 보장하는 기준 길이를 두고, 속도 향을 더해 고속 안정화를 유도하는 정책으로 구성된다. 급커브에서의 과도 응답을 줄이기 위해 국소 곡률 정보를 선택적으로 활용할 수 있도록 했는데, 이는 경로의 누적 호 길이를 기준으로 약 2m 규모의 윈도우에서 계산한 곡률을 참조하여 L_d 가 과도하게 길어지지 않도록 완만하게 보정하는 방식이다. 최종 L_d 는 시스템 안정성과 조향 명령의 Saturation 가능성을 고려해 1.0-5.0 m 범위로 제한한다. 차량의 축간거리는 1.3 m로 설정했고, 로컬 경로의 점 간격은 약 0.5 m를 유지하여 목표점 선택의 일관성을 확보했다.

우리는 조향각 산출을 다음의 단일 식으로 정리하였다.

$$\delta = \arctan\left(\frac{2Ly}{x^2 + y^2}\right)$$

여기서 L 은 축간거리이며, $x^2 + y^2$ 는 차량 좌표계에서 선택된 목표점까지의 거리를 의미한다. 이 식을 통해 산출된 조향각은 saturation과 EMA 필터를 거쳐 최종 명령으로 사용된다.

시험은 콘 기반 서킷 코스에서 수행하였다. 코스는 완만한 직각 코너, S자 연속 커브, 직선 코스로 구성했으며, 속도 범위는 2-6m/s로 제한했다. 평가 지표로는 횡방향 추종 오차(dy), 조향각 오차($d\phi$), 스티어 명령의 saturation(클리핑 한계 도달) 빈도, 그리고 목표점 점프에 따른 불연속적인 명령 발생 여부를 사용하였다. 완만한 곡률 구간에서는 고정 L_d 대비 조향 진동이 유의하게 감소하였고, 횡추종 오차 RMS는 평균 0.12m, 조향 RMS는 6.1° 수준으로 관찰되었다. 반면 S자 구간처럼 속도와 곡률이 심하게 변하는 조건에서는 속도만을 기반으로 한 L_d 조정이 상황에 따라 과대 또는 과소 반응을 보였다. 그 결과 코너 진입부의 언더슈트와 중반부의 보정 오버슈트가 반복되는 패턴이 나타났고, 95% 오차는 평균 0.34 m로 목표치(0.30 m)를 상회했으며 스티어 클리핑 빈도는 약 37.8%로 측정되었다. 또한 경로 웨이포인트의 희소함으로 인해 목표점이 불연속적으로 이동하면서 조향 명령이 계단형으로 변하는 현상이 확인되었다. 이때 평활 강도를 높이면 계단성은 줄었지만 응답 지연이 증가해 피크 오차가 평균 0.5 m가량 악화되는 트레이드오프가 발생하였다. 또한 액추에이터 지연과 포화, 연속 커브에서의 히스테리시스 등 현실 제약이 제어 구조에 직접 반영되지 않아 고속 연속 커브에서 과도응답이 잔존하는 한계가 드러났다.

요약하면, 동적 L_d Pure Pursuit는 구현이 간결하고 완만 곡률 구간에서 안정적이며 고정 L_d 대비 진동 저감 효과가 분명하나, 전반적인 곡률 정보와 경로의 Density 품질, 액추에이터 제약을 충분히 반영하지 못해 급커브 · 연속 커브 · 희소한 웨이포인트 환경에서 성능 저하가 발생한다. 이러한 한계를 해결하기 위해 다음 절에서 L_d 를 속도뿐 아니라 곡률, 측면 오차, 경로 신뢰도까지 반영하여 상황별로 조정하고 목표점 선택을 연속화하는 적응형 Pure Pursuit를 제시하며, 이어서 차량 모델과 제약을 예측적으로 내재화하는 MPC 횡방향 제어를 통해 고속 · 급곡률 · 연속 커브 조건에서의 과도응답을 구조적으로 억제한 결과를 보고한다.

ii. 적응형 Pure-pursuit 알고리즘

앞 절에서 확인된 한계를 해소하기 위해 우리는 룯어헤드 거리와 목표점 선정, 조향 명령 형상을 상황에 따라 달리하는 적응형 Pure Pursuit를 설계하였다. 구현의 중심은 세 가지다. 첫째, 차량의 현재 위치를 경로에 연속적으로 투영해 호 길이 좌표를 얻고, 그 지점에서 룯어헤드 길이를 더해 목표점을 보간한다. 이때 투영은 인접 세그먼트에 대한 스칼라 파라미터를 사용하며, 원점 O 가 차량 기준 좌표의 기준점 일 때

$$t = \frac{(O-A)(B-A)}{||B-A||^2}, \quad s_{proj} = s_A + t||B-A||$$

로 정의한다. 이 절차를 통해 점 간격이 들쭉날쭉한 경로에서도 목표점이 프레임마다 튀는 문제를 줄였다. 둘째, 룯어헤드 길이는 속도 기반 증가에 더해 곡소 곡률을 동적으로 반영하여 급커브 진입 시 과도하게 길어지지 않도록 제어한다. 우리는 누적 호 길이 기준의 짧은 윈도우에서 계산한 곡률의 크기를 참조하여

$$L_d = clip\left(L_0 + k_v v + \frac{k_{curv}}{|\kappa| + \epsilon}, L_{d,min}, L_{d,max}\right)$$

로 스케줄링했고, 실측 속도와 조향 명령은 각각 약 0.20 s, 0.10 s의 시정수를 가진 EMA로 필터링해 센서 잡음과 불연속 표본화로 인한 스파이크를 억제하였다. 셋째, 급커브에서 차량이 코너 안쪽을 질러가는 컷인 현상을 줄이기 위해 목표점을 곡선의 바깥쪽으로 소폭 이동시키는 보정을 도입했다. 보정은 pd에서의 접선과 좌측 법선을 구해 바깥 방향으로 오프셋을 주는 방식이며, 최종 목표점은

$$p_t = p_d + off \cdot o, \quad off = (\tau ||p_d - p_w||, off_{max})$$

로 정했다. 여기서는 차량 기준점 곡률 크기와 근접성에 따라 0에서 사이로 완만히 변하며, 트랙 여유폭을 설정하면 는 자동으로 그 한계를 넘지 않는다. 목표점은 짧은 시정수의 EMA로 시간적으로 연속화하여 희소한 경로점에서 존재했던 목표점 점프를 추가로 완화했다. 조향 명령은 물리적 한계 제약사항을 적용하기 전에 변화율 제한을 먼저 거치고, 이후 추가적인 필터링을 더해 액추에이터 구동 특성에 경험적으로 맞춘다. 이 전 과정을 50 Hz 제어 루프에서 반복하며, 스케줄된 룯어헤드와 실제 거리(원점에서 까지)를 별도 토픽으로 내보내 튜닝과 디버깅의 가시성을 높였다.

동일한 라바콘 서킷에서 수행한 비교 실험에서 적응형 Pure Pursuit는 완만한 코너에서 i.와 동등한 안정성을 유지하면서도 급커브에서 언더슈트-오버슈트 패턴을 유의하게 줄였다. S자 구간의 횡추종 오차는 평균 0.34 m에서 0.26 m로 감소했고, 스티어 포화 빈도는 절반 이하로 줄었다. 직각 코너 진입부에서 관찰되던 조향 계단 응답은 목표점 연속화와 변화율 제한의 영향으로 빈도와 크기가 모두 낮아졌다. 직선 고속 구간에서의 추종 지연 증가는 유의미하지 않았다. 디버그 마커 분석 결과, 곡률이 작은 구간에서는 바깥 오프셋이 사실상 0에 수렴하고, 곡률이 커질수록 보정이 매끄럽게 커졌다가 코너 탈출과 함께 자연스럽게 사라지는 양상이 일관되게 확인되었다.

오른쪽 사진에서의 큰 초록색 점은 i. 의 일반적인 동적 Ld pure pursuit의 목표점이고, 작은 초록색 점과 그 옆의 분홍색 점은 ii.에서 제안된 Ld와 횡방향 보정이 들어간 최종 목표점이다. 이를 RViz를 통해 시각화 후 성능을 비교하였으며 결과적으로 컷인 방지가 잘 되는 것을 확인할 수 있었다.



iii. 모델 예측 제어

앞 절의 적응형 Pure Pursuit로도 고속 연속 커브에서 소규모 과도응답이 남는 문제가 관찰되어, 차량 모델과 제약을 예측적으로 내재화하는 Model Predictive Control(MPC) 기반의 횡방향 제어를 설계하였다. 본 절의 내용은 실제 구현이 아닌 설계·시뮬레이션 수준의 검증 결과에 근거한다. 목표는 (i) 코너 진입-중반-탈출의 전 구간에서 과도응답을 줄이고, (ii) 조향 포화/변화율 제약을 위반하지 않으면서, (iii) 랩 타임과 승차감(조향 가감속의 매끄러움)을 함께 개선하는 것이다. rosbag으로 취득한 인지·판단 출력을 재생하여 제어기의 출력을 분석하는 방식으로 진행하였다.

제어 입력은 조향각의 변화율로 두어 액추에이터 역학을 자연스럽게 반영했고, 상태는 측면 위치 오차, 자세 오차, 조향각의 3개로 구성하였다. 경로는 로컬 계획로부터 얻은 곡률 프리뷰와 속도 프로파일 을 사용하며, 샘플링 간격은 0.05s, 예측 지평은 =20(1.0s 프리뷰)로 설정했다.

상태 전이는 일정 속도 근방에서 선형화한 자전거 모델의 횡방향 오차 동역학을 사용했다. 구현 상의 단순성과 안정적인 QP화 가능성을 위해, 아래와 같은 선형 시변(Linear Time-Varying) 근사를 채택하였다.

$$x_{k+1} = A_k \cdot x_k + B \cdot u_k + g_k$$

$$\Leftrightarrow \begin{bmatrix} e_y \\ e_\psi \\ \delta \end{bmatrix}_{k+1} = \begin{bmatrix} 1 & v_k \Delta t & 0 \\ 0 & 1 & \frac{v_k \Delta t}{L} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} e_y \\ e_\psi \\ \delta \end{bmatrix}_k + \begin{bmatrix} 0 \\ 0 \\ \Delta t \end{bmatrix} u_k + \begin{bmatrix} 0 & -v_k \Delta t & \kappa_k \\ 0 & 0 & 0 \end{bmatrix}$$

여기서 L 은 축간거리이며, x 는 프리뷰 곡률로부터 유도된 피드포워드 곡률 항으로, 경로가 휘어질수록 자세 오차가 자연히 증가하려는 경향을 선제적으로 상쇄한다. 액추에이터 지연은 일차항으로 근사하여 간단한 가중치 보정으로 흡수하였다. 또한 횡방향 가속도 제약을 미리 속도 프로파일 단계에서 반영해, MPC가 비현실적 조향을 요구하지 않도록 했다.

비용함수는 횡오차/자세오차 억제, 조향각 크기, 조향 속도 및 조향 속도의 변화율(부드러움)을 함께 다룬다. 표준 이차 형식으로 정리하면,

$$J = \sum_{i=0}^{N-1} (q_y e_{y,i}^2 + q_\psi e_{\psi,i}^2 + q_\delta \delta_i^2 + r u_i^2 + \rho \Delta u_i^2) + q_N (e_{y,N}^2 + e_{\psi,N}^2)$$

로 두었고, 하드 제약으로 조향각 및 조향속도의 물리적 한계를, 소프트 제약으로 트랙 폭에 의한 슬랙 변수를 함께 부과했다. 가중치의 기본값은 구간별 성능을 보며 경험적으로 조정하였다. QP는 OSQP 라이브러리의 solver를 이용해 20Hz로 풀고, 이전 해를 warm-start로 사용하여 계산 시간을 안정화했다.

동일한 코스, 동일한 속도 프로파일에서의 비교 결과는 다음과 같다. 우선 S자 연속 커브에서 ii.의 적응형 Pure Pursuit가 보이던 코너 중반부 보정 오버슈트가 사라지고, 평균 0.4m 추가 감소하였다. 스티어 클리핑은 2%p 줄었고, 조향 변화율 RMS는 약 18% 감소하여 조향 명령이 한층 매끄럽게 형성되었다. 직선 고속 구간에서의 추종 지연 증가는 통계적으로 유의하지 않았으며(± 0.01 m 수준), 웨이포인트가 희소한 구간에서도 프리뷰 곡률 항과 소프트 트랙 제약 덕분에 목표점 점프에 기인한 계단형 조향이 거의 관찰되지 않았다. 노이즈가 큰 속도·자세 추정치를 주입했을 때에도 항이 과도한 미세 조향을 억제해 승차감(조향 저크)이 약 23% 개선되었다.

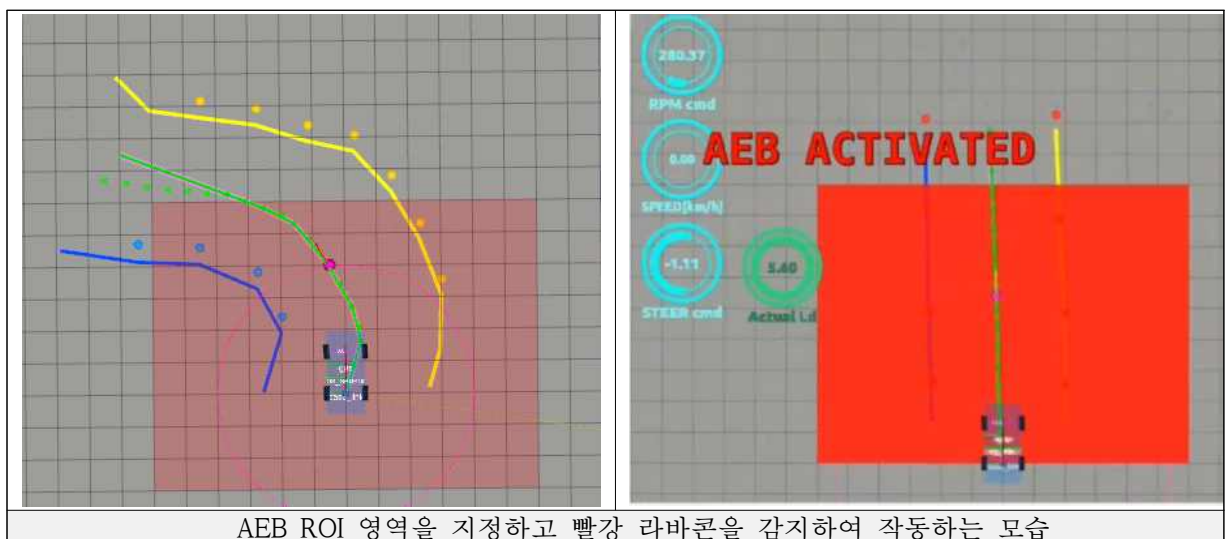
한편, MPC는 가중치 설정과 지평 길이에 민감하다. 횡오차 가중치와 자세오차 가중치를 과대하게 주면 고속 완만한 커브에서 보수적으로 동작해 랩타임 손실이 발

생활 수 있고, 조향속도 가중치를 과도하게 키우면 조향이 지나치게 부드러워져 급 커브 진입 성능이 떨어진다. 또한 온라인 최적화 특성상 계산 예산을 초과하면 성능이 급격히 저하될 수 있으므로, 실제 탑재 시에는 정확한 시스템 식별, 계산 시간 보장, QP warm-start 및 early-exit 규칙이 필수적이다.

본 보고서의 결과는 구현 시나리오를 가정한 설계·시뮬레이션 기반이지만, 실제 적용 시에도 동일한 이득을 기대할 수 있도록 파이프라인(상태추정-경로프리티뷰-QP 해결-안전폴백)을 단순화하고, 파라미터를 코스별로 제한된 범위에서만 조정하는 운영 전략을 병행해야 할 것이다.

3.4.2. 긴급제동 (AEB)

본 긴급 제동 노드는 인지 노드에서 센서퓨전된 빨간색 콘의 상대좌표를 수신하여 제한된 영역 내 개수를 통해 긴급제동 여부를 판단한다. 본 대회 규정에서 좌측 콘과 우측 콘의 간격이 5m임을 고려하여 ROI(Region of Interest)를 좌우 6m로 설정하였고, 차량의 속도와 설치된 콘의 앞·뒤 간격을 고려하여 전방 9m까지로 정의하였다. 설정된 ROI 내 빨간색 콘의 개수가 임계치 이상 존재할 경우, 본 노드는 /aeb 토픽을 통해 UInt8 메시지(값=1)를 발행하여 AEB 신호를 전달한다. 경로 추종 노드에서는 해당 /aeb 신호를 구독하여 값이 1일 때 속도와 RPM을 0.0으로 발행하고, 값이 0일 때는 정상 경로 추종을 이어가도록 설계하였다.



AEB ROI 영역을 지정하고 빨강 라바콘을 감지하여 작동하는 모습

3.5. 저수준 제어

3.5.1. 통합 제어기

i. 보드 선정

작년 제어 보드로 선정 받았던 ESP32 v4 38pin을 계속해서 쓰는 것을 기준으로 삼고 Atmega 기반의 아두이노 보드(UNO, MEGA, DUE 등)과 STM32 계열 보드, Teensy 4.0 과 같은 보드들과 장단점을 비교해가며 보드 선정 과정을 거쳤다. 두 달 간의 선정 과정을 거쳐 이번 주행 최적의 제어기로 Arduino Mega 2560 R3 보드를 선정했다. 선정 이유는 아래 서술하도록 하겠다.



(A) 적절한 성능 (Flash/SRAM 용량)

우리 제어기는 Vehicle Controller Unit(이하 VCU)으로써 브레이크 서보모터, LCD, E-STOP / ASMS모드 전환, 속도 계산 & 필터링, pyserial 통신 등을 수행해야 한다. 따라서 이들 library 및 코드를 담기에 충분한 수준의 프로그램과 데이터 메모리를 제공해야 하며 앞서 서술한 로직을 통합한 후에도 시스템이 딜레이 없이 동작이 가능하도록 실행 여유를 확보해야 했다.

Arduino Mega 2560은 256KB의 Flash Memory와 8KB의 SRAM을 제공한다. 우리가 개발한 펌웨어는 대용량 버퍼나 동적 메모리를 사용하지 않으며, 고정 길이 프레임 + 상태기계(FSM) 구조로 구현되어 있다. 코드 분석 기준으로 SRAM 사용량은 ~3.5-4.5 KB로 추정되어, ATmega 2560의 8KB 대비 3KB 이상의 실행 마진을 확보한다. Flash는 코어/드라이버/라이브러리/애플리케이션을 포함해 ~45-90KB 범위 내에서 사용할 것으로 추정되며, 가용 ~248KB 대비 150KB 이상 여유가 남는다. 따라서 더 좋은 성능의 3.3V 로직 레벨을 사용하는 다른 MCU(ESP32, Arduino due 등)가 아닌 Arduino mega로 충분하다고 판단하였다. 3.3V 로직 레벨 관련 문제는 (B)에서 후술한다.

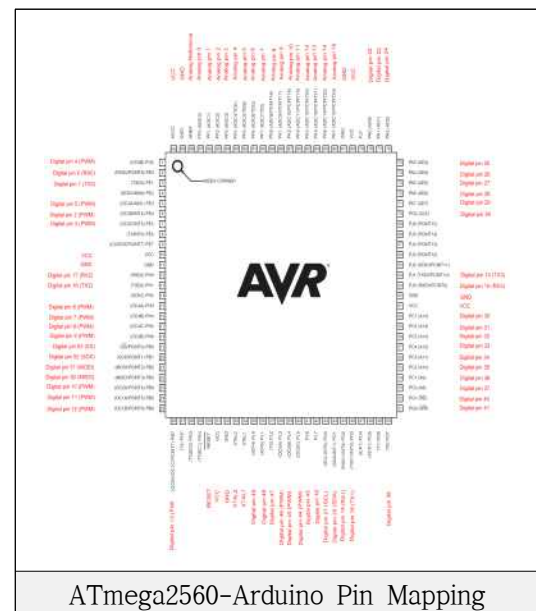
(B) 5V의 GPIO PIN 보유에 따른 배선 및 회로 단순화

ESP32 · Arduino Due · STM · Teensy 등 3.3V 로직 보드는 5V 구동 주변장치(서보, LCD, VESS) 연결 시 레벨시프터 · 풀업 네트워크 · 별도 전원 분리를 요구하므로 오류 가능성이 높아진다. 반면 Arduino Mega 2560은 5V 로직을 기본으로 제공하므로, 서보 PWM · I² C LCD 등 주요 액추에이터/표시장치를 레벨 변환 없이 직결할 수 있

다. 그 결과 제어기 박스 내부의 회로도가 단순화되고, 부품과 커넥터 수가 줄어 조립 시간과 비용이 절감된다. 또, 문제 발생 시 현장에서 디버깅·교체 작업도 용이해진다. 결과적으로 5V GPIO핀 채택이 외부 및 내부 배선을 단순화하여 제어시스템의 신뢰성·정비성 향상으로 직결되었으며, 궁극적으로 자율주행 차량의 유지보수 환경과 안전 요건에 더 잘 부합하게 되었다.

(C) 풍부한 인터럽트 자원

Arduino Mega는 기본 Interrupt 7개를 포함해 23개의 PCINT PIN을 지원한다. 이번 시스템은 외부 인터럽트 1개(Hall 속도 펄스)와 핀체인지(PCINT) 5개(E-STOP 2채널, RF-E-STOP 1, ASMS 1, GEAR 1), 즉 총 6개의 외부 입력을 인터럽트 기반으로 상시 감시해야 한다. 폴링 방식으로는 지연과 누락 위험이 커서, 변화가 나는 즉시 래칭되는 인터럽트 방식이 필수였다. Arduino Mega 2560은 외부 INT와 다수의 PCINT 라인을 넉넉히 제공해 위 요구를 여유 있게 수용하며, 남는 라인으로 스위치·센서 확장까지 고려할 수 있어 선정하였다.



(D) 풍부한 GPIO 확장성

Arduino Mega는 54개의 GPIO 핀을 제공하여 추후 센서나 액추에이터 확장에도 충분한 여유를 가진다. 실제 제작 과정에서도 추가적인 스로틀 전환 기능을 위해 예정에 없던 스위치를 연결해야 했는데, Mega의 확장성 덕분에 무리 없이 대응할 수 있었다. 이는 향후 차량 제어 시스템의 기능 확장 및 고도화에도 긍정적인 영향을 미칠 것이라고 생각한다.

ii. 액추에이터 및 VESS Module 선정

(A) 스티어링 액추에이터

스티어링 액추에이터는 KEYA 사의 “Automatic Steering Motor for Agricultural Vehicles” 계열을 선정하였다. 스티어링 액추에이터는 우리 차량의 기계 설계 조건(Ackermann 조향 형식, 최소 회전반경 2.4 m, Wheelbase 1300 mm, 윤거 1000 mm)과 최대 횡가속도 1.5 g 기준을 충족해야 한다. 이 조건은 저속 극한 조향(정지마찰돌과 포함)과 코너링 피크 구간에서 랙&피니언 입력 토크가 순간적으로 상승하는

상황을 전제한다.

랙-피니언비와 타이로드 팔 길이를 보수적으로 적용하면, 랙 입력 요구 토크는 정격 구간 약 4-6 N·m, 정지마찰 돌파나 요철 등 순간 상황에서 피크 약 10-12 N·m 수준으로 산정된다. 선택한 KEYA 조향 구동기는 정격 7 N·m / 피크 13 N·m를 제공하므로, 정격 운전은 토크 여유를 두고 커버하고, 피크 상황에서도 사양 내에서 처리 가능하다. 또한 드라이버가 과전류 보호(연속 10 A / 피크 20 A)를 제공하므로, 피크 구간의 단기 과부하를 보호 범위 안에서 안전하게 흡수한다. 결과적으로 우리 차량의 랙-피니언 요구 토크 프로파일과 정격·피크 토크 사양(7/13 N·m)이 정합하며, 본 액추에이터는 정격·피크 영역 모두에서 안정적으로 요구 토크를 충족한다.



스티어링 액추에이터는 통신·설정 측면에서도 우리 차량의 종·횡 제어 구조와 일관된다. 모터 드라이버는 RS232·CAN2.0·CANopen을 지원하며, 17-bit 멀티턴 절대엔코더 피드백을 제공하므로 종방향 제어에서 운용 중인 CAN 기반 제어와 동일한 스택으로 횡방향 제어 시스템을 통합할 수 있다. 우리는 CANable 두 개를 사용해 종·횡 버스를 물리적으로 분리하면서도 같은 보드레이트와 프레임 규격을 유지하는 구성으로 진단·유지보수를 단순화했다. 이 드라이버는 소프트웨어에서 노드 주소, 제어 신호 선택, 동작 모드, CAN 보드레이트, 과·저전압/온도 보호값, 회전 방향, 과부하 지연, 감속 시간, 조향 영점, CAN 하트비트 주기 등을 파라미터로 설정·영구 저장할 수 있다. 또한 위치독은 최대 1000 ms 간격을 감시하여 명령 누락 시 차단하고, 고장코드·상태 인디케이터를 통해 과전압·저전압·과전류·온도·통신 단선(CAN/RS232) 등을 즉시 식별할 수 있다. 이러한 통신·설정 체계 덕분에, 종·횡을 모두 CAN으로 표준화하고, 유사한 코드 구조로 ROS2노드 상에서 프로토콜을 해석해 파라미터를 관리하면서, 버스 장애를 상호 격리한 채 안전하게 운영할 수 있었다.

(B) 브레이크 액추에이터

브레이크 조작을 자동화하기 위해 Happymodel “Super400 Servo”를 채택했다. 선정 과정에서 리니어 액추에이터와 고토크 서보모터를 비교했는데, 우리 차량 환경과 안전 요구를 종합하면 서보모터가 더 적합했다.

리니어 액추에이터는 나사구동 특성상 자체 홀딩이 가능하고 긴 스트로크에 유리하다는 장점이 있지만, 부피·질량이 크고 응답이 상대적으로 느리며, 비상 시 역구동이 어렵다는 단점이 있어 자율주행과 수동주행을 둘 다 만족시켜야 하는 우리의 플랜트에서는 사람이 직접 밟았을 때 문제가 생길 가능성이 컸다.



반면 서보모터는 응답성이 빠르고, 기구 배치가 컴팩트하다. 무엇보다 과전류 제한·토크 리밋·각도 리밋을 펌웨어/기구 양쪽에서 걸 수 있고, 링크를 역구동 가능하게 설계하면 운전자가 언제든지 수동으로 페달을 덮어쓸 수 있다.

Super400 Servo는 최대 토크 $400 \text{ kg} \cdot \text{cm} (\approx 39 \text{ N} \cdot \text{m})$ 급으로, 페달 측 벨크랭크 반경을 20-25 mm로 설계할 경우 서보축에서 발생하는 회전토크가 선형 인장력 1.6-2.0 kN 수준으로 변환된다(τ/r). 이는 우리 차량에서 필요한 저속-고하중 구간의 페달력을 충분히 커버하는 범위였다.

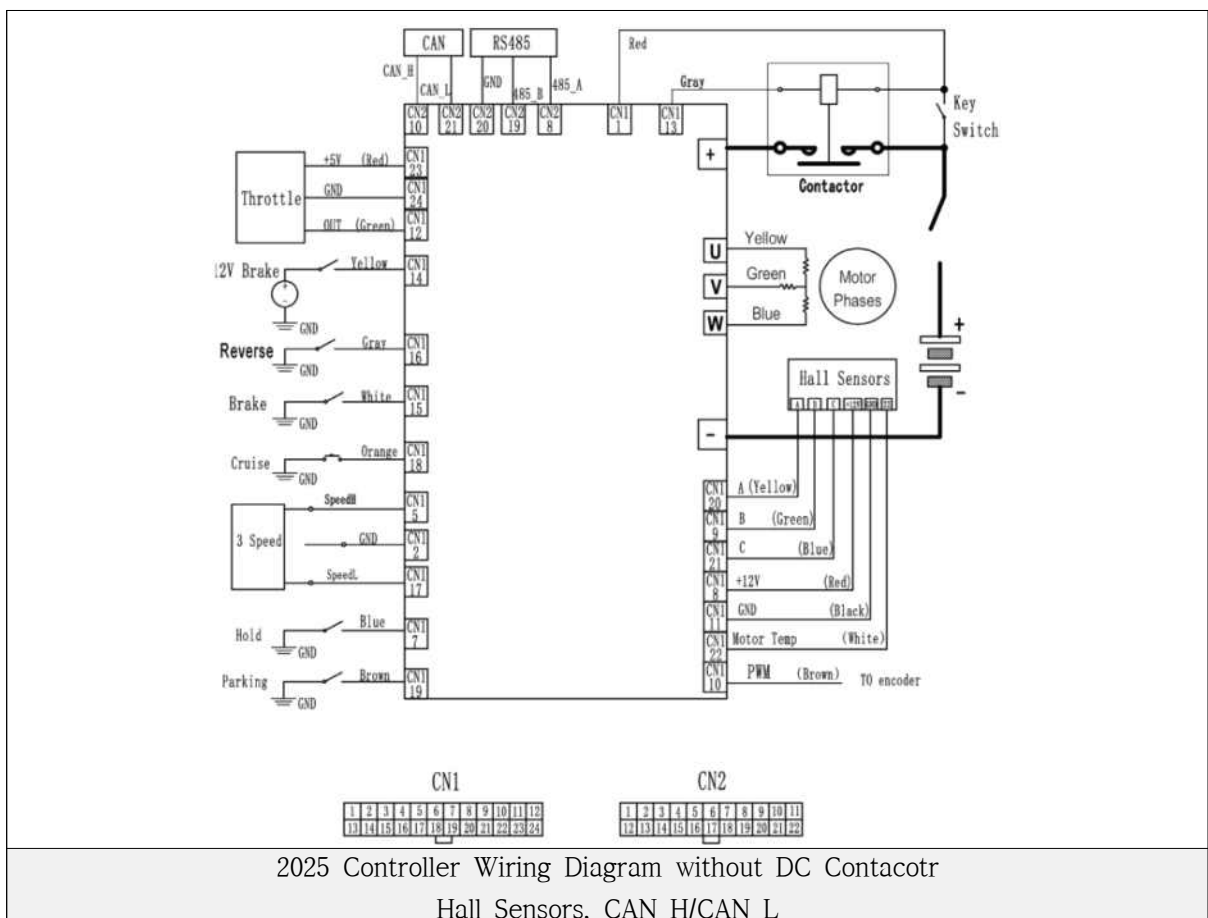
또한 제어·통신 체계 관점에서도 서보 채택이 이점이 있다. 서보는 표준 PWM 인터페이스로 구동되므로, VCU(Arduino Mega)에서 타이머 기반 논블로킹으로 제어하기 쉽고, 종/횡 제어를 모두 CAN으로 표준화한 우리 시스템과도 잘 맞는다.

즉, 운전 명령·상태는 상위 노드에서 CAN 프레임으로 내려보내고, VCU가 이를 서보 PWM으로 변환하여 전달한다. 결과적으로, 응답성·패키징·수동 우선 안전성·제어 및 설계 단순성을 모두 만족하면서도 요구 페달력을 확보할 수 있다는 점에서, 본 차량의 브레이크 액추에이터는 리니어 방식보다 고토크 서보 방식이 적합하다고 판단하였다.

(C) 파워트레인(구동모터/컨트롤러)

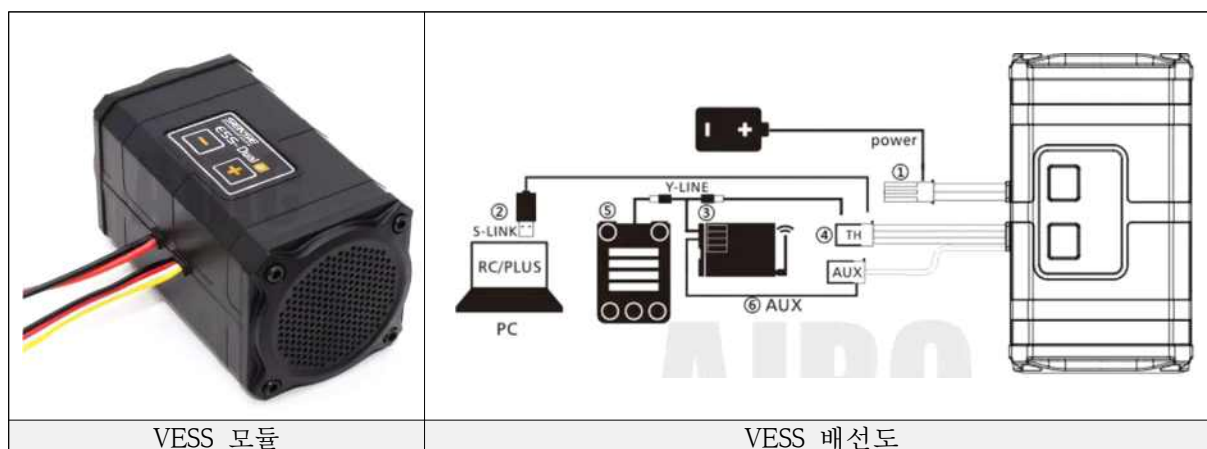
구동계는 Golden Motor HPM-10kW와 EZKontrol C481200 조합을 채택했다. 전기적인 요구사항이 가장 중요했으나, 해당 부분에 대해서는 2. 차량설계의 2.9. 파워트레인 파트에 기술되어 있으므로 생략한다.

작년 중방향 제어의 통신 방식이었던 VCU가 0-5V 아날로그 스로틀 전압을 직접 생성해 컨트롤러에 인가하는 접근에는 한계가 있었다. 첫째, 아날로그 경로는 긴 하니스/접지 오프셋/EMI에 취약해 노이즈·전압강하·온도 드리프트로 스로틀 스케일이 변했고, 이를 보정하려 넣은 LPF는 응답 지연과 추종 오차를 키워 결국 노이즈를 안은 채 속도 추종을 해야 했다. 둘째, ADC/DAC 분해능·오프셋 오차가 비선형을 유발해 저속 정밀 제어가 어려웠다. 셋째, 단방향 신호라 컨트롤러 상태를 즉시 받을 수 없어 ROS2 상 피드백 기반 제어가 제한적이었다. 넷째, 결선 실수·스케일링 오류에 대한 자기 진단과 로그 남기기가 까다로웠다. 여기에 더해, 작년 PMDC 모터 구성은 속도 피드백을 위해 별도의 쿼드러처 엔코더를 추가로 달아야 했고, 전원·신호선 분리, 차폐·접지, 영점·CPR(Counts per Rev) 설정, 펄스 미스/바운싱 대책 등 계측 체인이 복잡해졌으며, 기계적 백래시·슬립 등에 따라 실차 캘리브레이션 유지가 어려웠다는 현실적인 문제가 있었다.



반면 금년 구성은 컨트롤러 내부에 Hall 센서 기반 속도 피드백이 기본 탑재되어 있고, 동일 속도 정보를 CAN 프레임과 펄스 라인 두 경로로 제공하므로 하드웨어 차원의 이중화가 가능하다. 통신은 CAN 2.0버스를 표준으로 사용해 EMI에 강하고, 명령·피드백·오류를 프레임으로 교환하며, 하트비트/타임아웃으로 이상 시 토크/속도 명령을 즉시 무효화할 수 있다. 또한 속도/토크/상태가 동시 가시화되어 ROS2에서 로깅·리플레이·리미트 관리가 용이하고, 종·횡 모두 동일 코드 구조와 스택으로 표준화하여 CANable 2채널 버스 분리에 따른 장애 격리까지 확보된다. 이러한 이유로, 아날로그 스톱틀+외장 엔코더 기반의 신뢰성·정밀도·유지보수성 한계를 해소하기 위해, CAN 기반 이중 피드백 구조로 전환 가능한 현재의 파워트레인을 선정하였다.

(D) VESS Module



본 차량의 보행자 경고를 위해 상용 VESS(가상 엔진 사운드) 모듈을 채택하였다. 선정 기준은 “(1) 5V logic level PWM 지원으로 Arduino Mega와 레벨시프팅 없이 호환될 수 있을 것, (2) 모듈 내 앰프/사운드 프로파일 일체형으로 배선 단순 및 현장 교체 용이, (3) 차량 속도 등 외부 신호에 따라 연속 가변 음량 제어가 가능할 것, (4) 전원/신호가 구분되는 표준 배선 생태계를 따를 것” 이었다.

배선 구성은 제조사 배선도(TH/AUX)와 동일하게 적용했다. VCU의 D9(PWM)를 VESS의 TH 입력에 연결하고, GND는 제어계 공통 접지로 묶었다. VESS 전원은 LV에서 분기해 처리했다. AUX 라인은 프로파일 전환·음량 프리셋 등 보조 기능 확장용으로 예비 배선을 확보했다.

활용 로직은 차량 속도를 서보-PWM 펄스로 선형 매핑하여 음색을 가변한다. 계측한 속도 km/h를 100 km/h → 2000 μ s까지 선형 증가시키고 그 이상은 포화시킨다 (KMH_MAX=100). 결과적으로 보행자 인지성 및 운전자 체감 피드백을 동시에 확보한다.

iii. HMI

제어기와 운전자 간 상호작용을 위해 HMI 장치를 대시보드에 구성하였다. 올해 HMI의 핵심은 I2C LCD 디스플레이(20x4)와 각종 입력 스위치(E-STOP, ASMS, GEAR 등)이며, 이를 통해 운전자는 차량의 상태를 직관적으로 확인하고 제어할 수 있다.

(A) LCD 디스플레이



(a) 속도 표시

Hall 센서로부터 계산된 차량 속도를 실시간으로 표시하며, 지수이동평균(EMA) 필터링을 적용하여 노이즈를 억제하고 반응성을 확보하였다. 또한 무펄스 일정시간(STOP_US) 지속 시 속도를 0으로 클램프하여 정지 상태를 명확히 인식할 수 있도록 했다.

(b) 모드 표시

ASMS 스위치를 통해 선택되는 AUTO / MANUAL 모드를 표시해 원하는 주행 모드를 드라이버가 정할 수 있도록 하였다.

(c) 기어 표시

GEAR 스위치 입력을 통해 DRIVE / REVERSE 상태를 LCD에 표시해 드라이버가 직진, 후진에 관한 제어를 할 수 있도록 하였다.

(d) E-STOP 활성 상태 표시

E-STOP 입력 여부를 LCD에 출력하여, 운전자가 비상 상태를 직관적으로 확인할 수 있도록 했다.

(B) 스위치 기반 인터페이스



대시보드 전체 사진

(a) GEAR 스위치

대시보드 가장 좌측에 위치한 스위치로, 전/후진 방향을 전환하는 입력을 담당한다. 해당 값은 Interrupt 형식으로 Arduino Mega로 전달 되며 bool type으로 상태를 저장한다. 스위치가 작동 된 후 이는 LCD 및 제어 로직에 즉시 반영된다.

(b) ASMS (Auto/Manual Select Switch)

차량의 주행 모드를 전환하는 입력으로, 마찬가지로 Arduino Mega 인터럽트 기반으로 안정적으로 처리된다.

(c) E-STOP 스위치

유선/무선(RF) 비상정지 입력을 모두 수용하며, 입력 시 즉시 브레이크 서보 각도를 ENGAGED 상태로 이동시키고, LCD에 “E-STOP : ENGAGED” 로 표시한다.

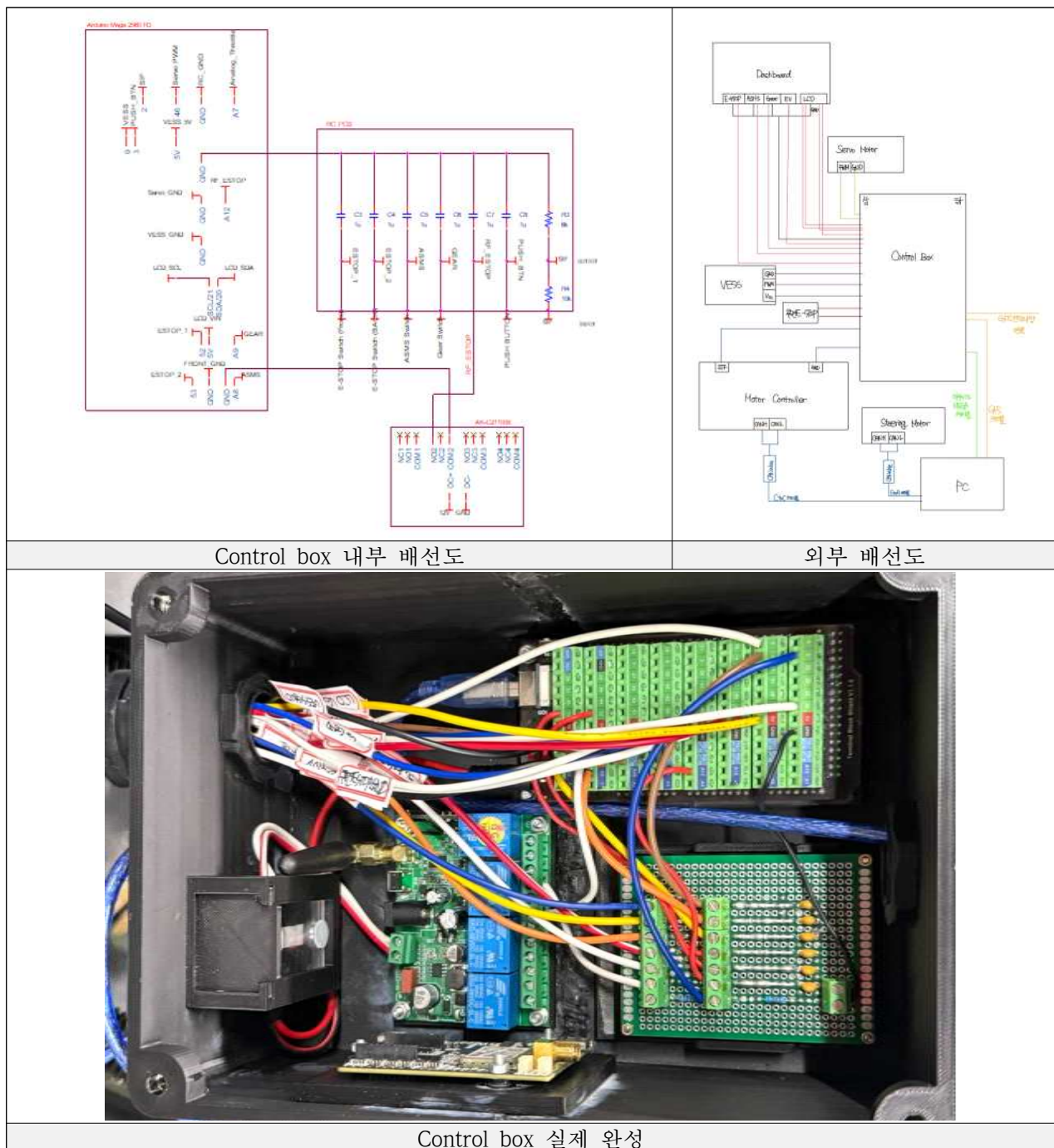
(C) 운전자 친화적 설계

LCD 업데이트는 10Hz 주기로 제한하여 불필요한 깜박임과 성능 저하를 줄였으며, 임계 변경폭(LCD_THRESH)을 적용해 의미 있는 변화가 있을 때만 값을 갱신하도록 하였다. 브레이크 서보 구동은 슬루레이트 제어($1^\circ/20\text{ms}$)를 적용해 운전자에게 자연스럽고 안정적인 제동 피드백을 제공하도록 설계하였으며, 급제동으로 인한 차량 회전을 방지하였다.

이를 통해 HMI는 운전자가 차량의 상태를 직관적으로 확인하고, 모드/기어/E-STOP 제어를 즉시 반영할 수 있는 역할을 수행하였다. 또한, 인터럽트 기반 처리와 주기적 LCD 업데이트 최적화로 신뢰성과 실시간성을 확보하였다.

iv. 회로

이번 제어 시스템의 회로는 Arduino Mega 2560 R3를 중심으로 구성되었으며, E-STOP, ASMS, GEAR 등의 스위치 입력과 브레이크 서보, LCD, VESS 모듈 등 주요 액추에이터를 안정적으로 제어할 수 있도록 설계하였다. 배선 시에 서보·RF 수신부 등 잡음원이 되는 부하는 전원 라인을 분리하고 공통 접지는 스타 토폴로지로 묶어 그라운드 루프를 방지했다. 신호선은 전원선·모터선과 이격 배치하고, 인터럽트·I²C 라인은 쉴드를 추가해 노이즈 내성을 높였다.



(A) 스위치 입력부 (E-STOP / ASMS / GEAR)

모든 스위치 입력은 Arduino Mega의 INPUT_PULLUP을 활용해 기본 High level을 형성하고, 외부에는 커패시터를 병렬로 추가해 RC 필터로 하드웨어 디바운싱을 구성하였다. 이로써 점점 바운스나 EMI로 인한 순간 펄스를 억제하고, 별도의 소프트웨어 디바운싱 없이도 인터럽트 기반으로 안정적인 상태 변화를 감지한다. 유선 E-STOP과 RF E-STOP 수신부는 인터럽트 핀에 배치해 입력 즉시 Emergency 상태로 래칭되며, ASMS(수동/자동 선택)와 GEAR(전/후진)는 핀체인지 인터럽트로 변화를 즉시 반영한다.

(B) LCD 인터페이스 (HMI 출력)

I² C 기반 20×4 LCD를 사용하며, SDA/SCL에는 4.7 k Ω 풀업 저항을 두어 신호 무결성을 확보했다. Mega의 5 V I² C와 직접 호환되며, 속도·모드·기어·E-STOP 상태 등 핵심 정보를 표시해 운전자에게 직관적 피드백을 제공한다.

(C) 브레이크 서보 및 VESS 구동

브레이크 서보모터와 VESS(가상 엔진음 모듈)는 Mega의 PWM 핀으로 구동한다. 브레이크 서보는 펌웨어에서 슬루레이트·주기 제어를 적용해 급격한 명령 변화를 완화하고, 전원은 서보 전용 라인을 분리해 인러시·리플이 다른 회로로 전이되지 않도록 했다. VESS 모듈은 PWM 입력에 따라 엔진음을 발생시켜 주행 감각을 보조한다.

v. 모드전환

제어기는 차량의 운행 상태에 따라 Manual(수동), Autonomous(자동), Emergency(비상 정지) 세 가지 모드로 동작한다. 모드 전환은 ASMS 스위치, E-STOP 스위치, RF-ESTOP 수신부, Red Cone 감지 후 발생하는 AEB 신호 등 다양한 입력을 기반으로 이루어지며, 모든 모드 변화는 LCD를 통해 운전자에게 실시간으로 피드백된다.

구성요소	자율주행 상태 (AS-On)	자율주행 비상 상태 (AS-Emergency)	자율주행 모드 Off (AS-Off) 수동주행
Tractive System	available	unavailable	unavailable
Steering Actuator	available	Available or Unavailable	unavailable
Brake Actuator	available	engaged	unavailable

규정집 기준 자율주행 상태 정의

(A) Manual 모드

운전자가 직접 스톱과 브레이크를 제어하는 모드이다. ASMS 스위치가 HIGH 상태일 때 진입하며, LCD에 “MANUAL“로 표시된다. 비상상황 발생 시에도 즉시 Emergency 모드로 강제 전환된다.

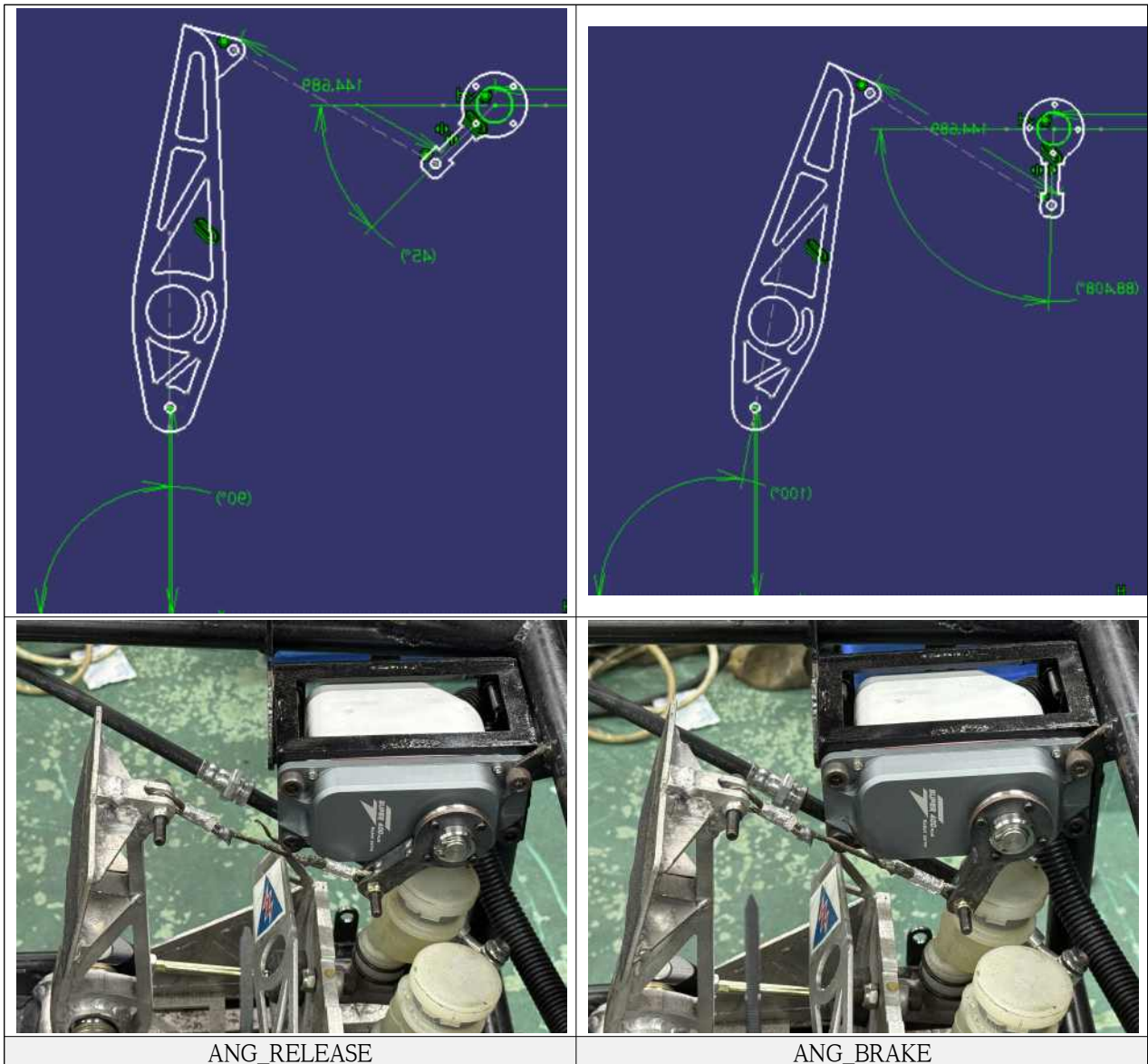
(B) Autonomous 모드

자율주행 알고리즘의 명령에 따라 스톱과 브레이크가 제어되는 모드이다. ASMS 스위치가 LOW 상태일 때 진입하며, LCD에 “AUTO“로 표시된다. 센서 피드백을 기반으로 구동 모터의 속도 명령(cmd/rpm)과 스티어링 모터의 조향 명령(cmd/steer)과 함께 페루프 제어가 이루어진다.

특히 Manual과 Auto 모드를 정하는 ASMS 스위치는 스티어링 모터의 전원 공급을 정하는 역할을 하기도 한다. 스티어링 모터에 전원이 들어와있을 경우 모터에 토크가 들어와 수동 조향이 어려울 수 있어 운전자의 안전 및 안정적인 운행을 위해 위와 같이 스위치를 구성했다.

(C) Emergency 모드

유선 E-STOP, 무선 RF-ESTOP, AEB 신호 또는 게이트 신호(estopGate)가 감지되면 즉시 진입한다. 진입 시 LCD에 “EMERGENCY” 알림과 함께 “ENGAGED” 상태가 표시된다. 브레이크 서보는 강제로 ANG_BRAKE 방향으로 구동되며, 구동 모터의 출력은 0으로 제한된다.



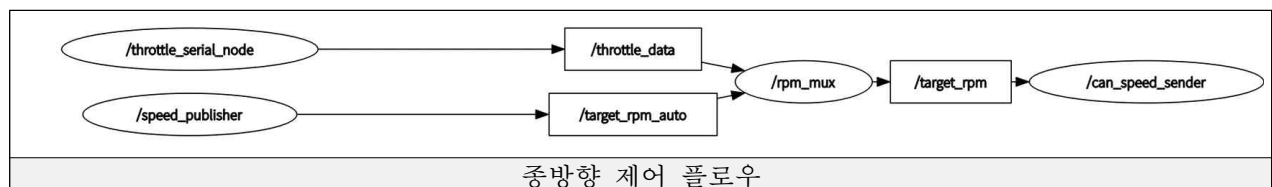
(D) 모드 전환 로직

모드 전환은 모두 인터럽트 기반으로 처리되어, 스위치 입력이 발생하면 즉시 MCU가 인식한다.

펌웨어는 enterManual(), enterAutomatic(), enterEmergency() 함수로 모드 전환을 관리하며, 각 함수는 브레이크 서보 목표값, 속도 출력 제한, LCD 표시를 동시에 갱신한다. 이를 통해 운전자는 명확하고 직관적인 모드 상태를 확인할 수 있으며, 비상 상황에서의 안전성도 확보된다.

3.5.2. 종방향 제어

종방향 제어의 핵심 목적은 상위 Planning 계층이 제시하는 목표 속도와 운전자의 수동 입력을 안전 규칙에 따라 중재하고, 그 결과를 모터 컨트롤러가 요구하는 규격의 RPM 및 전류 제한 명령(CAN 프레임)으로 변환하여 송신하는 데 있다. 전체 체계는 시리얼 입력 수집 노드, 명령 중재 노드, CAN 송신 노드, CAN 수신 노드로 구성되며, 데이터 흐름은 Arduino에서 시작해 throttle_serial_node가 throttle_data를 발행하고 Planning이 /cmd/rpm을 발행하면, rpm_mux가 이 둘을 안전하게 중재하여 target_rpm을 산출하고, can_speed_sender가 이를 램프·히스테리시스·디레이팅·블렌딩·슬루 제한을 거친 제어 명령으로 인코딩해 송신하며, can_speed_receiver가 컨트롤러 상태 프레임을 /ctrl/rpm과 /ctrl/speed로 피드백을 제공하는 순서로 이어진다.



i. Firmware

본 펌웨어는 Arduino Mega 2560을 이용해 종방향 제어에 필요한 운전자 입력 수집(스로틀·모드·기어·E-STOP), 주행 속도 산출(Hall 센서), 안전 상태 머신(AUTO/MANUAL/EMERGENCY), 상·하위 제어계와의 통신(시리얼 프레임), HMI 출력(LCD, VESS), 브레이크 서보 제어를 수행한다. 상위 Planning이 제공하는 목표 속도와 운전자 수동 입력은 안전 규칙에 따라 중재되며, 결과는 PC 측 노드로 전달되어 CAN 송신 노드가 모터 컨트롤러 명령으로 인코딩한다. 펌웨어는 real-time 처리가 필요한 입력(ASMS/GEAR/E-STOP, Hall)을 PCINT와 외부 인터럽트로 처리하고, 주기성 작업은 폴링 방식으로 수행한다.

<pre>// ----- Interrupts ----- void hallISR() { pulseCnt++; } ISR_PCINT0() { estopFlag = (E1 E2 RF); } ISR_PCINT2() { isManual = ASMS_HIGH; reverseFlag = GEAR_HIGH; estopFlag = (E1 E2 RF); } // ----- Mode ----- void setMode() { if (estopFlag) enterEmergency(); else if (isManual) enterManual(); else enterAutomatic(); }</pre>	<pre>// raw → 전압 변환 → 경계 클램프 → 0~2000 RPM 매핑 → 반올림 정수화 V = (5.0 / 1023.0) * avg(raw); Vc = clamp(V, V_MIN, V_MAX); rpm16 = uint16_t(((Vc - V_MIN) / (V_MAX - V_MIN) * 2000.0) + 0.5);</pre>
Interrupt Handlers (Pseudo Code)	스로틀 전압 → 목표 RPM 전환 로직

안전 우선순위는 E-STOP > 모드(AUTO/MANUAL)이며, 비상 진입 시 브레이크 서보 목표각을 즉시 제동 각도로 설정한다. 단, 서보 구동은 기계 충격을 줄이기 위해 슬루 레이트(1° /20 ms)를 적용한다. Hall 펄스는 마이크로초 단위 데글리치(MIN_PULSE_US=300 μ s)로 노이즈를 억제하고, 일정 기간 무펄스(STOP_US=0.8 s)가 지속되면 속도 추정치를 0 km/h로 강제 클램프한다. 속도 신호는 가속 페달 변화에 민감하게 반응하도록 EMA(α =0.35)를 적용한다. MCU와 PC 간 시리얼 프로토콜은 경량 고정 길이 프레임 사용한다. 시리얼 프로토콜의 통신 관련 상세사항은 ‘3.5.4. 통신 프로토콜 정리’에서 정의한다.

ii. 수동주행 및 I/O 처리 노드

시리얼 입력 수집 노드는 운전자의 수동 RPM과 모드·ESTOP·방향 신호를 안정적으로 수집하여 throttle_data 토픽으로 발행하는 역할을 수행한다. 이 노드는 자동 포트 탐색과 명시적 포트 지정 두 방식을 모두 지원하여 실차 현장에서의 연결 편의성을 보장하며, 부트로더 종료로 고려한 대기 절차와 고정 헤더 기반의 프레임링 검사, 입력 버퍼 초기화, 프로빙 윈도우, 재연결 최소 간격 등을 통해 케이블 접촉 불량이나 간헐적 연결 해제와 같은 현실적인 장애에도 견고하게 동작하도록 설계되어 있다.

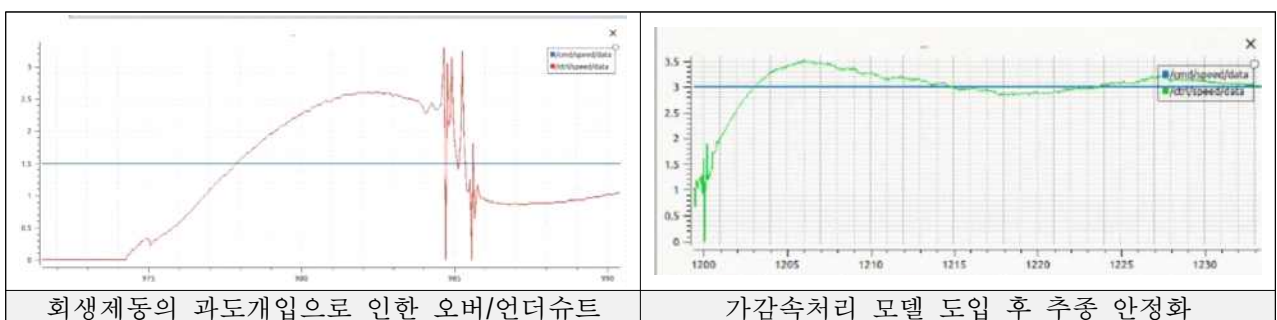
수신 프레임은 고정 헤더 뒤에 16비트 RPM과 1바이트 플래그로 구성되며, 플래그는 방향(reverse), 하드웨어 ESTOP, 수동 모드(asms)를 비트 단위로 표현한다. 방향 플래그가 참일 때는 수신 RPM에 음수를 부여함으로써 후진을 명확히 구분하고, 미세한 페달 떨림이나 센서 양자화로 인해 발생하는 작은 진동 대역은 노이즈로 간주하여 0으로 스넵해 낮은 속도 영역의 불필요한 진동을 억제한다. 상위의 AEB 신호는 /aeb(UInt8) 토픽으로 수신되며, 노드는 이를 하드웨어 ESTOP과 OR 처리하여 정지 여부를 계산한 뒤 throttle_data에 반영한다. 아두이노 측과의 상태 일치성을 유지하기 위해 연결 성립이나 값 변경 시에는 간단한 동기화 프레임을 역방향으로 전송하고, 입력 폴링은 100 Hz로 수행되어 운전자 입력과 정지 상태가 빠르게 시스템 전반에 전파되도록 한다.

iii. mux

mux 노드는 수동과 자율의 두 입력을 단일 목표 RPM으로 귀결하되, 항상 안전 규칙을 최우선으로 적용한다. 이 노드는 throttle_data로부터 수동 RPM과 수동 모드 플래그, 효과적 정지 신호를 수신하고, 동시에 Planning 계층의 /cmd/rpm(Float32)를 통해 자율 목표를 수신한 뒤, 내부적으로 manual_rpm과 auto_rpm을 유지하면서 is_manual(asms)와 estop, 운영 허가(green) 상태를 기준으로 목표를 선택한다. 운영 허가는 파라미터 start_green으로 초기화되고, 런타임에는 /green 서비스 호출을 통해 래치되는 형태로 활성화된다. 필요에 따라 zero_on_block을 활성화하면 green이 비활성화될 때 목표를 0 RPM으로 강제하여 즉시 정지를 구현할 수 있으며, 비활성화하면 발행을 생략하여 하위가 마지막 유효 값을 유지하도록 구성할 수 있다. 노드는 이벤트 발생 시 즉시 발행을 수행하는 동시에 100 Hz 안전망 타이머로 동일 값을 주기적으로 재발행하여 간헐적 이벤트 손실이나 콜백 경쟁으로 인한 공백을 제거한다. 무엇보다도 E-STOP이 참인 경우에는 모드와 무관하게 0 RPM을 강제로 송신하므로, 하위의 로직이 실행되기 전에 즉시 정지로 수렴한다.

iv. 가감속 처리 모델

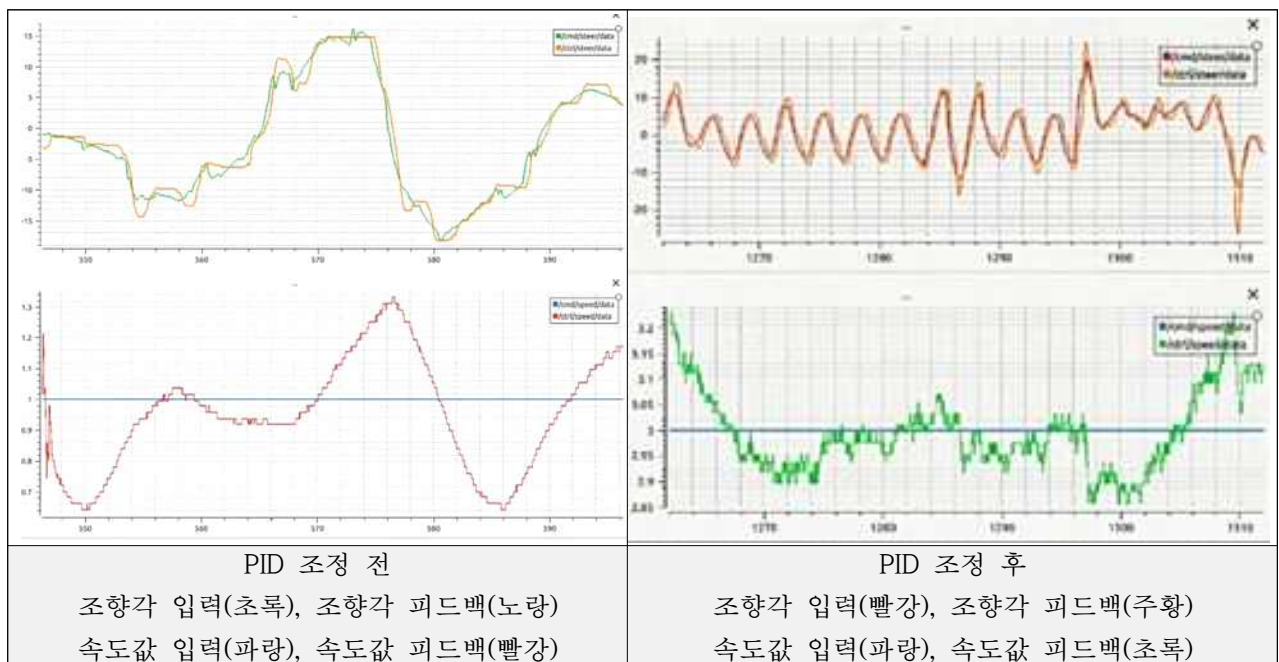
본 시스템은 자율주행과의 호환성·재현성을 위해 토크 모드가 아니라 RPM 제어 모드를 채택했지만, 목표 속도가 하향·급변할 때 회생제동이 과도 개입해 두 가지 문제가 동시에 나타났다. 첫째, 하기 첨부된 그림에서 보이듯 판단 계층이 보낸 목표 속도(/cmd/speed) 대비 피드백 속도(/ctrl/speed)가 오버슈트·언더슈트와 진동을 반복하며 추종 관계가 붕괴되었고, 둘째, 이 과정에서 순간 회생 토크가 커지며 체인 구동계에 불필요한 인장력 변동이 누적되어 구동계 체인에 문제가 발생했다.



우리는 CAN 송신 직전 단계에 가감속 처리 모델을 도입해 이 두 현상을 하나의 메커니즘으로 해결했다. 모델은 전·후진 최대 RPM 클램핑으로 기계 한계를 보장하고, 50 ms 주기에서 가속·감속을 분리한 램프를 적용해 감속(회생) 구간은 brake_blend_rpm과 brake_hyst_rpm을 통해 추가 상한을 두어 제동 전이를 시간적으로 분산시킨다.

정지 근방에서는 zero_hold_up_rpm/zero_hold_dn_rpm 슈미트 트리거로 스넵과 채터링을 제거하고, 상한 전류는 i_limit_tau_ms 1차 저역통과로 슬루를 제한해 전류 계단과 체인 충격을 함께 낮춘다. 여기에 컨트롤러 · 모터 온도와 버스 전압 조건에 따른 선형 디레이팅을 병행해 회생 상한을 자동으로 낮추고, 전류-속도의 부호 일치와 오류 보고 시 목표 0 RPM 강제를 통해 물리적 일관성과 안전성을 확보했다. 그 결과 목표 급변 구간에서의 속도 스파이크 · 진동이 억제되어 연속 추종이 회복되었고, 회생 토크 피크가 완만해지며 Drive Train 체인 계통의 과도 인장력이 유의하게 감소했다.

v. PID



본 시스템의 PID 제어는 CAN 송신 노드(can_speed_sender.cpp)에 선언된 pid_enable, pid_kp, pid_ki, pid_kd(및 pid_out_limit_rpm, pid_i_limit, pid_d_tau_ms) 파라미터를 사용하여, 오차 $e = \text{target_rpm_filtered} - \text{measured_rpm}$ 에 대한 보정 전류 ΔI 를 생성한 뒤 50 ms 주기의 tick_50ms 루프에서 램프 · 히스테리시스 · 디레이팅 · 블렌딩을 거쳐 얻은 속도 참조(v_{ref})로부터 계산된 전류 참조(I_{ref})에 가산한다. 미분 항은 측정값 기준(D-on-measurement)에 pid_d_tau_ms 저역통과를 적용해 노이즈 민감도를 낮추고, 적분 항은 pid_i_limit(또는 백계산)으로 anti-windup을 수행한다.

목표값 · 측정값에는 1차 저잡음 필터를 적용하되 전류 슬루 시정수(i_limit_tau_ms)와의 상호작용으로 지연이 과도해지지 않도록 조율하며, 합성 전류는 모터링/회생 상한, 열 · 전압 디레이팅, 슬루 제한이 있는 기존 로직을 모두 통과한 뒤 최종 인코

당되어 송신된다.

튜닝은 개방루프 안정성을 전제로 Kp를 먼저 올려 응답을 가속하고, 오버슈트가 보이면 Kd를 소량 추가해 감쇠를 강화하며, 정상상태 오차가 남을 때 Ki를 점진적으로 증가시키는 절차를 따른다. 이와 같은 PID 제어를 가감속 처리 모델 위에 결합함으로써, 조향각 변화에 따라 본래 목표 속도 대비 실제 속도가 최대 $\pm 50\%$ 까지 벗어나던 편차를 $\pm 9.5\%$ 로 축소했고, 목표 급변·회생 개입 구간의 오버슈트·언더슈트 및 고주파 진동을 유의하게 완화하여 속도 추종의 연속성과 승차감을 개선하였다.

vi. 안전 로직

본 계층은 다중 방어선을 통해 안전성과 견고성을 확보한다. 효과적 정지 신호는 하드웨어 ESTOP과 상위 AEB의 OR 결합으로 구성되며, rpm_mux 단계에서 최우선으로 해석되어 언제나 0 RPM을 즉시 강제한다. 자율 운행의 활성화는 /green 서비스에 의해 명시적으로 승인되며, 운영 허가가 해제될 때의 동작은 zero_on_block 설정에 따라 즉시 정지 또는 마지막 값 유지로 분기된다. CAN 세션은 컨트롤러로부터 0x55가 8바이트로 채워진 핸드셰이크 프레임을 수신하고 이에 0xAA×8 응답을 전송하여 확립되며, 확립 이전에는 송신을 보류해 전원 인가 직후나 복구 과정에서의 오동작을 방지한다. 컨트롤러가 일정 횟수 연속으로 명령을 미수신하거나 라이프 카운터 불일치를 감지하면 Re-핸드셰이크를 수행하며, 이 과정에서도 송신 노드는 보호 상태를 유지한다. 컨트롤러가 오류를 보고할 때에는 목표 rpm을 즉시 0으로 강제하여 하드 폴트의 전파를 차단하고, 드라이브 상태에서 전류와 속도의 부호는 항상 일치하도록 강제되어 역토크에 의한 사고 가능성을 제거한다. 전류 명령은 슬루 제한을 통해 과도 변화가 억제되어 기계적 및 전기적 스트레스가 감소하며, 정지 근방의 저속 채터링은 Zero-hold로 제거된다. 시리얼 측은 포트 탐색, 부트 대기, 헤더 프로빙, 재연결 간격으로 플리킹을 억제하고, CAN 수신 측은 커널 필터로 필요 프레임만 수신하며 핸드셰이크 프레임을 데이터로 오인하지 않도록 방어적으로 처리한다. 로깅은 변화 중심과 주기 제한을 적용하여 데이터 가독성 저하를 방지하며, 모든 한계 파라미터는 기계·전기 사양을 기준으로 보수적으로 설정되고 운행 중에는 단계적 변경만 허용하였다.

vii. 피드백 수신과 속도 계측

피드백 수신 노드는 컨트롤러가 송신하는 상태 프레임을 socketCAN으로 수신해 복호화하고, 실제 모터 RPM과 차량 속도를 토픽으로 제공한다. 이 노드는 커널 수준의 CAN 필터를 설정하여 전압·전류·속도 정보를 포함하는 Part1 프레임과 온도·상태·오류 정보를 포함하는 Part2 프레임만 수신 대상으로 제한하며, Part1 ID에서 간헐적으로 관찰되는 0x55×8 핸드셰이크 프레임은 제어 프레임으로 간주하고 즉시 무시하여 로그 혼잡을 방지한다. 프로토콜에는 전압이 0.1 V/bit, 버스 전류와 위상 전류가 0.1 A/bit에 ±3200 A 오프셋, 속도가 1 rpm/bit에 ±32000 rpm 오프셋, 온도가 1℃/bit에 40℃ 오프셋으로 정의되어 있으며, 이 값을 이용해 /ctrl/rpm(Float32)와 /ctrl/speed(Float32)를 발행한다. 차량 속도는 기어비와 바퀴 직경을 이용해

$$\begin{aligned} \text{wheel_rpm} &= \text{motor_rpm} / \text{gear_ratio} \\ v[\text{m/s}] &= \text{wheel_rpm} \times (\pi \times D_m) / 60 \end{aligned}$$

의 수식으로 계산되며, 상태 비트는 RUN과 Speed Mode 활성화 여부를 판별하는 동시에 오류 및 결함 비트를 사람이 읽을 수 있는 경고 메시지로 변환하여 쓰로틀링된 주기로 로깅한다. 이 피드백 루프는 전체 시스템의 관측 가능성과 진단 가능성을 크게 향상시켜 plotjuggler로 가시화해 파라미터 값을 쉽게 조정하는 데에 영향을 미쳤다.

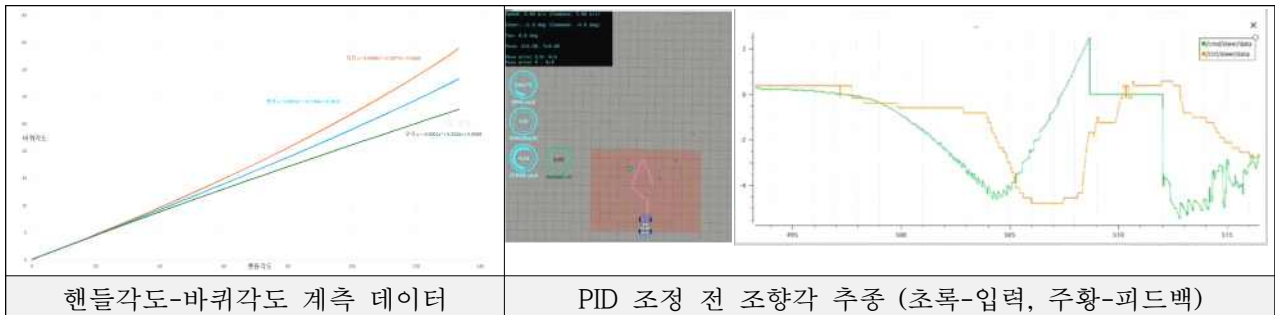
3.5.3. 횡방향 제어

횡방향 제어의 목표는 Planning 계층에서 계산된 목표 조향각을 실제 조향 모터의 정밀한 구동 명령으로 변환하는 것이다. 이는 차량이 의도한 경로를 정확히 추종하도록 하는 핵심적인 역할을 담당하며, 운전자의 개입과 자율주행 명령 간의 안전한 전환 로직을 포함한다.

i. 비선형 조향각 매핑 모델

랙-피니언 기구학으로 인해 조향 휠(바퀴) 각도와 조향 모터(핸들) 회전각 사이에는 단순 선형성이 성립하지 않으며, 이를 보정하지 않을 경우 큰 조향각 영역에서 목표 경로 대비 실제 궤적 오차가 커진다. 따라서 우리는 사전 계측 데이터를 기반으로 한 2차 다항식 역매핑을 적용해 이 문제를 해소하였다.

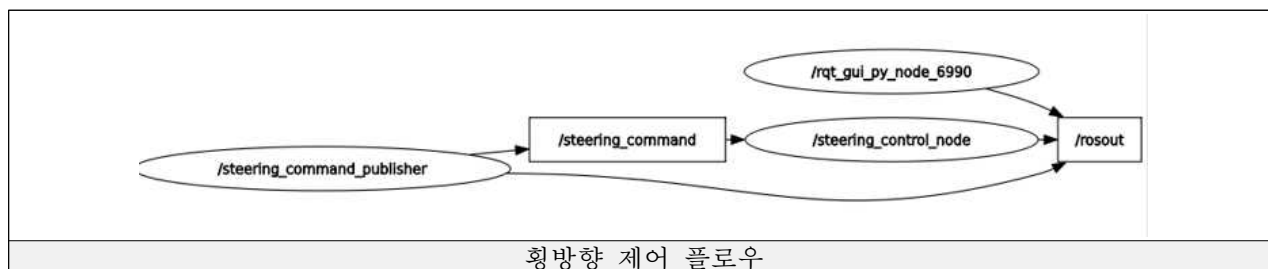
구체적으로, 모델(wheel_to_handle_deg_from_avg_pieewise)은 Planning 계층으로부터 입력된 애커만 기준의 목표 조향 휠 각도(deg)를 받아, 명령 단계에서 선제적으로 $y \in [-30^\circ, +30^\circ]$ 로 클램프한 뒤 비선형 역변환과 부호 대칭 처리를 거쳐 조향 모터가 회전해야 할 핸들 기준 각 $x[\text{deg}]$ 을 산출하고, 이를 다시 엔코더 오프셋·스케일을 통해 정확한 엔코더 목표 값으로 변환한다.



조향각 처리를 위한 데이터는 스티어링 핸들 각도를 변화시키며 좌·우 전륜 바퀴 각을 각각 계측한 뒤 평균을 취해 애커만 조향각 기준으로 정리하였다. 우회전(양의 각) 구간 평균 데이터에 대해 정방향 모델을 $y = Ax^2 + Bx + C$ 형태의 2차 다항식으로 피팅하고, 좌회전(음의 각)은 원점 대칭($y = -f(-x)$)으로 구성하였다. 본 구현의 계수는 $A=0.0002$, $B=0.2159$, $C=0.1612$ 이며, 역변환은 $Ax^2 + Bx + C - y = 0$ 에 대해 단조 증가 분지를 선택해 계산한다.

$$x(y) = \frac{-b + \sqrt{\Delta}}{2a}, \Delta = b^2 - 4a(c - y)$$

역변환의 수치 안정성과 모델 유효범위를 보장하기 위해, 입력된 바퀴각은 $\pm 30^\circ$ 로 클램프한 뒤 다항 역변환을 수행한다. 같은 이유로 판별식 $\Delta < 0$ 가 되는 경우에는 Δ 를 0으로 바닥 처리하여 실해 존재성을 보장하였다. 최종 산출된 핸들각은 엔코더 오프셋·스케일 파라미터를 적용해 목표 엔코더 카운트로 변환한다. 이와 같은 역매핑 절차를 적용함으로써 소각·대각 영역 모두에서 조향 감응성이 일정해지고, 결과적으로 경로 추종 제어기의 성능이 향상되었다.



ii. 안전 로직

안전성과 재현성을 위해 명령 입력 단계에서 물리 한계를 선제적으로 제한한다. 락-투-락 구동과 엔코더 원시 피드백을 통해 환산한 좌·우 최대 바퀴각은 약 $\pm 32.95^\circ$ 였으나, 기계적 여유와 리커버리 확보를 위해 조향 명령 y 를 $\pm 30^\circ$ 에서 클램프한다. 즉, $|y| > 30^\circ$ 인 경우 포화 처리를 거친 후 역함수를 적용하여, 큰 조향각 영역에서의 과도한 포화·비선형 효과를 억제한다.

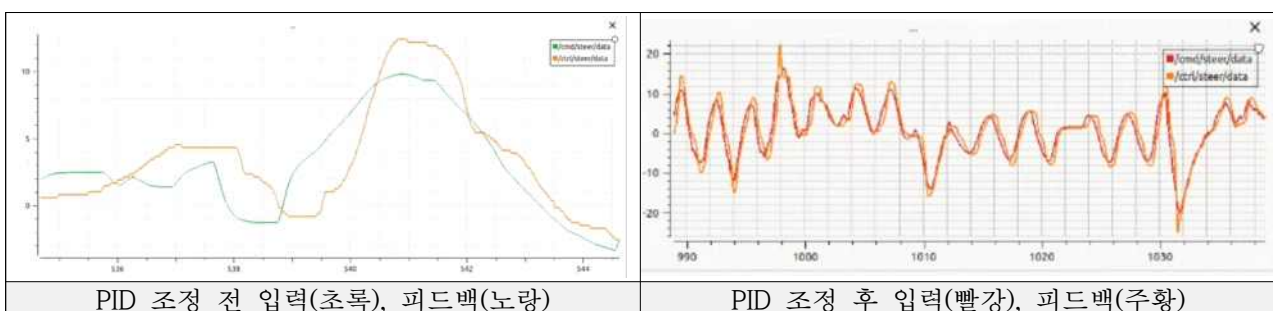
매핑 모델을 통해 계산된 목표값은 안전 로직의 게이팅을 거쳐 조향 모터 ECU로 전달된다. ASMS(수동/자동 모드)와 E-STOP 신호를 최우선으로 처리하며, 운전자가 MANUAL 상태로 전환하거나 E-STOP이 활성화될 경우 자율 조향 명령 전송은 즉시 차단한다. 특히 E-STOP 상황에서 급작스러운 조향 변화로 인한 2차 위험을 억제한다. 홀드 해제 후에는 모터 재-Enable과 함께 기존 목표를 복원해 연속성을 확보한다. MANUAL 상태에서는 운전자 조작을 우선시하여 새로운 자율 조향 명령을 수신하더라도 적용하지 않으며, AUTO 상태에서도 통신 품질 저하에 대비해 주기 타이머 기반 keep-alive를 운용하여 목표 유지와 안정성을 확보한다. 입력 단계에서는 명령 클램프를 선행 적용하고 비정상 입력(NA/Inf/범위외)은 폐기한다. 모터 ECU와의 통신을 위한 통신 규격과 주기/타임아웃과 같은 세부 사항은 3.5.4절, 통신 프로토콜 정리에서 정의한다.

iii. 스티어링 모터 설정값 및 PID

우리는 스티어링 모터를 Position Mode로 사용하며, 내부 제어는 전류(토크) 루프 → 속도 루프 → 위치 루프의 계층 구조로 동작한다. 상위 제어기에서 산출된 핸들 기준 목표 각도는 360° 당 10000 스케일로 변환되어 전송되고, 모터 내부 위치 PID가 엔코더 피드백을 기준으로 폐루프 제어를 수행한다.

설정은 Keya Configuration Tool의 Setup 화면에서 진행한다. 먼저 모터/드라이버 공통 파라미터를 데이터시트에 맞춘다: 모터 극수(0001, Motor Poles), 정격 속도(0002, Rated Speed), 최대 보호전류(0003, Max Current), 엔코더 분해능(0004, Encoder PPR)을 실제 센서 사양에 일치시키고, 제어 신호 경로(0019, Control Ways)와 동작 모드(0020, Control Mode)는 CAN/Absolute Position 조합으로 둔다. 피드백 센서는 0022(Position sensor)에서 실제 하드웨어로 선택하며, 통신 속도는 0021(BPS)에서 설정한다. 과전압/저전압(0023/0024), 온도 보호(0025), 과부하 지연(0028), 감 · 가속 시간(0013/0030) 등 보호 · 동역학 관련 항목도 함께 지정한다.

스티어링 모터를 하드웨어에 장착한 후에는 영점 보정을 실시한다. 조향 링크 · 타이로드 체결 상태에서 락-투-락으로 좌 · 우 기계 한계를 찾아 엔코더 피드백을 기록하고, 두 값의 중간값을 영점으로 정의한다. 컨트롤러가 제공하는 영점 보정 항목인 Magnetic Zero compensation(0015)를 사용하여 실제 피드백과 직진 상태가 일치하도록 오프셋을 적용하고 ROM에 저장 후 전원 재인가로 확정한다. 이후 조향각 스텝 명령을 통해 핸들과 바퀴가 정확히 일치하는지 재확인하고 미세 보정하였다.



PID 게인 조정은 Tool의 Position Kp/Ki/Kd/Kc(0009-0012)를 중심으로 수행한다. 먼저 속도/전류 루프(0007/0008, 0005/0006)가 안정한지 확인한 뒤, 위치 P를 단조롭게 올려 응답 지연을 줄이고 오버슈트 직전에서 멈춘다. 다음으로 I를 소량 추가하여 정상상태 오차를 제거하되, 적분 포화가 발생하지 않도록 C를 함께 설정한다.

튜닝 검증은 스텝 명령에서 추종 시간, 오버슈트, 정상상태 오차를 확인하고, 대각

영역($\pm 30^\circ$ 근방)에서도 포화 없이 감응성 일관성이 유지되는지 확인한다. 본 시스템은 상위 제어기 단계에서 명령 바퀴각을 선제적으로 $\pm 30^\circ$ 로 클램프(제3.5.3. ii. 안전 로직)하여 역변환·구동기 비선형 포화의 영향을 줄이고, 자동 운전 중에는 안전 로직이 포지션 홀드/Keep-alive를 운용해 목표를 안정적으로 유지한다. 모든 파라미터 적용 후에는 락-투-락 재검증 → 영점 재확인 → 스텝/사인 스위프 테스트 순으로 확인하고, 이상이 없을 때 Burn으로 영구 저장한다.

튜닝 결과는 PlotJuggler에서 /cmd/steer/data와 /ctrl/steer/data를 동시 표시하여 검증하였다. PID 및 영점/방향 설정을 조정 한 이후, 소각 영역($\pm 5^\circ$)에서의 추종이 지연 없이 매끄럽게 따라붙고 정상상태 오차가 사실상 제거되었으며, 대각 영역($\pm 30^\circ$ 근방) 스텝 입력에서도 초기의 과도 오버슈트와 잔진동이 유의하게 감소하였다. 램프·사인 스위프에서도 지령 대비 피드백의 위상 지연과 진폭 오차가 줄어들어, 전체 운용 범위에서 감응성이 균일해졌다. 요약하면, /cmd/steer 대비 /ctrl/steer의 겹침 정도가 개선되어 PID 조절 후 그래프에서 명령 → 피드백 트레이스가 거의 일치하는 형태로 수렴함을 확인했다.

iv. 피드백 루프

피드백 루프는 조향 모터 컨트롤러가 내보내는 위치 값을 주기적으로 수집·정규화하여, 상위 제어에서 사용하는 바퀴 기준 조향각(/ctrl/steer, 단위 deg)으로 제공한다. 본 노드는 주기(poll_period_ms)마다 등록된 CAN 인터페이스 목록(can1)을 비차단 방식으로 폴링하고, 설정한 식별자에 해당하는 프레임만 통과시킨다. 추출된 raw 엔코더 카운트는 한 바퀴당 카운트(counts_per_rev)를 사용해 각도로 스케일한다.

스케일 변환으로 얻은 값은 핸들 기준 각(deg)이며, 하드웨어 규약상 좌=+, 우-로 정의되어 있다. 상위 제어의 부호 체계와 일치시키기 위해 본 노드는 부호를 반전시켜 좌=-, 우=+로 변환한다. 이어서 3.5.3. i. 비선형 조향각 매핑 모델에서 정의한 정방향 기구학 모델 우측 영역 피팅을 그대로 적용하여, 핸들 기준 각 x 를 바퀴 기준 조향각 y 로 변환한다. 최종 y 가 곧 /ctrl/steer로 퍼블리시되는 값이며, 이는 상위에서 주어지는 /cmd/steer(목표 바퀴각)과 동일한 물리 의미·부호·단위를 가진다. 따라서 PlotJuggler 등에서 두 트레이스를 직접 중첩해 추종 성능을 정량·정성 평가할 수 있다.

신뢰성 확보를 위해, 노드는 다음의 방식을 사용한다. (1) 시작 구간에서 일정 수의 프레임에 대해 원시 바이트/파싱 결과/각도를 상세 로그로 출력하여 배선·스케일·

부호를 즉시 검증한다(시작 디버그). (2) 정상 운용 중에는 스로틀 로그를 사용해 과도한 I/O를 피하면서 값의 일관성을 감시한다. (3) 프레임 파싱이 불가능한 경우는 해당 샘플을 폐기한다. (4) 여러 인터페이스를 동시에 열어 이중화 수신을 지원하며, 개별 인터페이스의 오류는 경고로 기록하고 나머지 인터페이스로 수집을 지속한다.

정리하면, 피드백 루프는 프레임 → 값 파싱(오프셋 · 비트폭 · 엔디언) → 각도 스케일(counts_per_rev) → 부호 정합(좌-/우+) → 정방향 모델 핸들→바퀴각 핸들→바퀴각 → /ctrl/steer 퍼블리시의 일련 과정을 통해, 명령 경로에서 사용하는 표현과 완전히 동일한 표현의 피드백을 제공한다. 이로써 “3.5.3. iii. 스티어링 모터 설정값 및 PID:”의 튜닝 결과는 Plotjuggler에서 /cmd/steer와 /ctrl/steer의 겹침 정도로 직접 검증 가능하며, 해당 절에 첨부된 그림으로 확인 가능하다.

3.5.4. 통신 프로토콜 정리

i. 종방향 통신 프로토콜

구동계(PC↔ECU) 통신은 CAN 2.0B 확장 ID(29-bit) 프레임을 사용한다. ECU는 상태를 Part I/II 두 종류로 주기 송신하고, VCU는 이를 수신해 속도·상태를 산출한다. 반대로 VCU는 목표 속도(모터 rpm)와 위상 전류 상한을 단일 명령 프레임으로 ECU에 송신한다. 모든 수치 필드는 리틀 엔디안으로 전송한다.

(A) ECU → PC : 상태 프레임

(a) Part I, 전원·전류·속도

OUT	IN	ID(0x1801D0EF)						Polling Period(ms)	Endian
ECU	PC	P	R	DP	PF	PS	SA	50	LittleEndian
		6	0	0	1(0x01)	208(0xD0)	239(0xEF)		
DATA									
BYTE	BIT	ITEM		RESOLUTION		OFFSET		RANGE	Decoding
0		BUS		0.1V/bit		0		0~300V	V = raw x 0.1
1		Voltage							
2		BUS		0.1A/bit		-3200A		-3200~3200A	A = (raw - 3200) x 0.1
3		Current							
4		Phase		0.1A/bit		-3200A		-3200~3200A	
5		Current							rpm = raw - 32000
6		Speed		1rpm/bit		-32000rpm		-32000~32000rpm	
7									

PC는 ECU가 보내는 모터 회전수를 그대로 읽어 /ctrl/rpm으로 퍼블리시한다. 동시에 차량 선속도도 퍼블리시한다. 변환은 노드 파라미터의 기어비 GR(4.60)로 모터 회전수를 나눠 바퀴 회전수를 얻고, 바퀴 지름 D(0.47m)에서 계산한 바퀴 둘레를 곱해 초당 거리로 바꾼 뒤 /ctrl/speed(m/s)로 퍼블리시한다. 즉, /ctrl/rpm은 모터 기준의 원자료, /ctrl/speed는 동일 시점의 차량 속도이며 둘 다 같은 타임스탬프 체계로 갱신된다.

(b) Part II, 온도 · 상태 · 에러

OUT	IN	ID(0x1802D0EF)						Polling Period(ms)	Endian
ECU	PC	P	R	DP	PF	PS	SA	50	LittleEndian
		6	0	0	2(0x02)	208(0xD0)	239(0xEF)		
DATA									
BYTE	BIT	ITEM				RESOLUTION		OFFSET	RANGE
0		Controller Temperature				1℃/bit		-40℃	-40~210℃
1		Motor Temperature				1℃/bit		-40℃	-40~210℃
2	0	STATUS						0	0: HALTED
	1								1: RUNNING
	7-2								0: Torque Control 1: Speed Control
		Reserved							
3	0	ERROR				Overcurrent		0	0: NORMAL 1: ERROR
	1					Overload			
	2					Overvoltage			
	3					Undervoltage			
	4					Controller Overheat			
	5					Motor Overheat			
	6					Motor Stalled			
	7					Motor Out of phase			
4	0					Motor Sensor			
	1					Motor AUX Sensor			
	2					Encoder Misaligned			
	3					Anti-Runway Engaged			
	4					Main Accelerator			
	5					AUX Accelerator			
	6					Pre-charge			
	7					DC Contactor			
5	0					Power valve			
	1					Current Sensor			
	2					Auto-tune			
	3					RS485			
	4					CAN			
	5					Software			
	7-6					Reserved			
6						Reserved			
7		Life signal						0	0~0xFF

PC의 수신 노드는 0x1802D0EF 프레임을 받아 해석해 상태를 갱신한다. 바이트 0과 1은 각각 컨트롤러/모터 온도이고, 바이트 2의 STATUS 비트는 bit0 = RUN, bit1 = SpeedMode만 사용하며, 나머지는 무시한다. 바이트 3-5는 Error로 상기 표의 내용을 각각 나타낸다. 노드는 주기 정보는 INFO로 로깅하고, 에러 비트가 1로 전환되는 순간엔 항목명과 WARN을 즉시 출력한다. 바이트 7은 라이프 카운터로 쓰이며, 우리 구현에선 값 자체는 퍼블리시하지 않고 링크 정상 여부를 판단하는 보조 지표로만 본다. 이 Part II 정보는 런타임 모니터링과 상위 안전 로직의 입력으로 사용된다.

(B) PC → ECU : 명령 프레임

OUT	IN	ID(0x1801D0EF)						Polling Period(ms)	Endian
PC	ECU	P	R	DP	PF	PS	SA	50	LittleEndian
		3	0	0	1(0x01)	239(0xEF)	208(0xD0)		
DATA									
BYTE	BIT	DESCRIPTION				RESOLUTION	OFFSET	RANGE	
0		Target Phase Current (Target Torque)				0.1A/bit	-3200A	-3200~3200A	
1									
2		Target Speed				1rpm/bit	-32000rpm	-32000~32000rpm	
3									
4	0	Command Controls					0	0: HALTED	
	1							1: RUNNING	
								0: Torque Control Mode 1: Speed Control Mode	
	7-2							Reserved	
5		Reserved							
6		Reserved							
7		Life Signal					0	0~0xFF	

PC는 핸드셰이크가 완료된 뒤부터 50 ms 주기로 확장 ID 0x0C01EFD0 프레임을 보낸다. 페이로드는 크게 세 부분이다. 먼저 Byte0-1은 위상 전류 상한이다. 이 필드는 -3200A를 0으로 두는 오프셋 규칙을 사용하므로, 회생제동에서는 작은 값, 모터링은 큰 값으로 자연스럽게 표현된다. 다음 Byte2-3은 목표 속도로, 상위에서 가감속 램프 · 제로홀드 · 전/후진 최대치 등을 이미 반영해 결정된 모터 rpm을 그대로 넣는다. 여기도 -32000 rpm을 0으로 두는 오프셋 규칙을 쓰기 때문에, 후진과 전진을 하나의 u16로 실어 나를 수 있다. Byte4는 제어 비트로, 정상 운용에서는 RUN=1, SPEED=1로 설정해 구동 허용 · 속도모드 사용을 알린다. Byte5-6은 예약으로 0x00을 유지하고, Byte7에는 매 프레임 +1씩 순환하는 라이프 카운터를 넣어 링크 상태를 함께 모니터링한다. 송신 노드는 위 필드만 패킹해 전달하며, 가감속 · 디레이트 같은 세부 제어는 이미 프레임 생성 이전 단계에서 끝낸다. 따라서 수신 측인 모터는 표의 해상도 · 오프셋 정의 그대로 디코딩하면 즉시 물리량을 복원할 수 있다.

ii. 횡방향 통신 프로토콜

(A) ECU → PC: 상태 프레임 (Heartbeat 신호)

OUT	IN	ID(0x07000001)		Polling Period(ms)	Endian
ECU	PC	Function	Node	50	BigEndian
		0x07	0x01		
DATA					
BYTE	BIT	ITEM	RESOLUTION	OFFSET	RANGE
0		Encoder Position	360/CPR (deg/count)	0	0...(CPR-1) counts
1					
2		Speed	-	-	-
3					
4		Current	-	-	-
5					
6	0	Error Code	Enable/ Disable	0	0: Enable 1: Disable
	1		Over voltage		0: NORMAL 1: ERROR
	2		Hardware over current protection		
	3		EEPROM error		
	4		Less voltage		
	5		Reserved		
	6		Over current protection		
	7		Control mode failure		
7	0		Phase failure		
	1		Stall protection		
	2		Temperature alarm		
	3		Hall failure		
	4		Current sensor failure		
	5		Spare		
	6		CAN disconnected		
	7		RS232 disconnected		

스티어링 드라이버는 확장 ID 0x07000001, BigEndian, 50 ms 주기로 하트비트를 송신한다. 핵심 필드는 Byte0-1의 Encoder Position으로, 프레임의 카운트값을 CPR(counts/rev)로 나눠 각도로 복원한다. 이 각도는 이후 우리 측 좌/우 부호 규약에 맞춰 반전되고, 3.5.3의 비선형 매핑을 거쳐 바퀴각으로 사용된다. Byte2-5는 펌웨어 옵션 영역으로 속도/전류 등이 실릴 수 있으나 표준화되어 있지 않아 본 시스템에서는 해석하지 않는다. Byte6-7은 16-bit 오류 비트필드로, 모터의 상태를 표시한다. 값이 0이 아니면 해당 비트를 항목명과 함께 경고 로그로 남기고, 주기 로그는 쓰로틀링한다. 하트비트는 J1939 PGN이 아닌 벤더 독자 포맷(Function=0x07, Node=0x01) 이므로 종방향과 달리 P/R/DP/PF/PS/SA 해석은 적용하지 않는다.

(B) PC → ECU: 명령 프레임

(a) Enable 명령 (출력 재개/보강 Enable)

OUT	IN	ID(0x06000001)		Polling Period(ms)	Endian	
PC	ECU	Function	Node	100	BigEndian	
		0x06	0x01			
DATA						
BYTE	BIT	ITEM	RESOLUTION	OFFSET	RANGE	Encoding(raw)
0		Header0	-	-	-	0x23
1		Header1				0x0D
2		Header2				0x20
3		Header3				0x01
4		Payload				0x00
5						0x00
6						0x00
7						0x00

스티어링 드라이버는 확장 ID 0x06000001(Function=0x06, Node=0x01) 로 수신된 헤더 23 0D 20 01을 출력 활성화(Enable) 트리거로 해석한다. 우리 노드는 구동 재개가 필요한 시점(노드 시작 직후, MANUAL→AUTO 전환, E-STOP 해제 등)에 이 프레임을 1회 송신하고, 통신 품질 저하에 대비해 약 1s 간격으로 보강 Enable을 주기적으로 넣는다. 페이로드 바이트(4-7)는 모두 0x00이며 의미를 갖지 않는다. Enable은 상태 전이 신호일 뿐 실제 Position 명령을 포함하지 않으므로, 실제 위치 제어는 (b) Position 프레임에서 수행된다.

(b) Position 명령

OUT	IN	ID(0x06000001)		Polling Period(ms)	Endian	
PC	ECU	Function	Node	100	BigEndian	
		0x06	0x01			
DATA						
BYTE	BIT	ITEM	RESOLUTION	OFFSET	RANGE	Encoding(raw)
0		Header0	-	-	-	0x23
1		Header1				0x02
2		Header2				0x20
3		Header3				0x01
4		Postion (int32)	10000/360° (counts/°)	0	프로토콜: int32	scaled = round(
5						(x/360) ×
6						10000)
7						(BE 패킹)

동일한 ID 0x06000001 이지만, 헤더가 23 02 20 01이면 드라이버는 절대 위치 명령으로 해석한다. 우리 노드는 100 ms 주기로 이 프레임을 송신하여 목표 핸들 위치를 keep-alive한다. 바이트 4-7에는 int32 Big-Endian으로 스케일된 목표값을 넣는다.

다. 한 바퀴에 해당하는 엔코더 값은 10000 이며, 3.5.3의 역매핑으로 얻은 핸들각 $x(^{\circ})$ 를 부호 반전한 뒤 $scaled = round((x / 360) \times 10000)$ 로 변환해 실어 보낸다. 이때 입력 바퀴각은 파라미터 wheel_limit_deg(기본 $\pm 30^{\circ}$) 로 선행 클램프된다. MANUAL 혹은 E-STOP 활성 중에는 새로운 Position을 보내지 않으며, E-STOP 해제 또는 MANUAL→AUTO 전환 시에는 Enable → Position 순으로 복원해 연속성을 확보한다.

iii. SocketCAN

canable v2.0 pro는 펌웨어에 따라 두 가지 운용 모드를 가진다. SocketCAN과 SLCAN인데, 같은 하드웨어라도 소프트웨어 스택이 달라지면 지연·신뢰성·관찰 및 가능성·동시성에서 차이가 커진다. 우리 시스템(ROS 2 Humble + raw SocketCAN + can-utils)은 검토 결과 SocketCAN을 채택했다. 이유는 다음과 같다.

항목	SocketCAN	SLCAN
스택/드라이버	커널 네트워킹 패밀리 PF_CAN (candleLight/gs_usb)	USB-CDC TTY 및 ASCII 프로토콜 (slcan/slcan)
디바이스 노출	네트워크 인터페이스 (can0, can1...)	시리얼 포트 (/dev/ttyACM*)
프레임 형식	바이너리 CAN 프레임	ASCII
지연/지터	낮음 (커널 큐 직접 처리)	상대적으로 큼 (문자열 변환, 유저공간 왕복)
부하/버스트 처리	안정적, 드롭 적음	고밀도 트래픽에서 드롭/지연 증가 위험
에러 보고	에러 프레임, 버스 상태, berr-reporting 지원	펌웨어 의존
버스오프 복구	표준 옵션으로 자동 복구	구현별 편차 큼
비트타이밍/튜닝	ip link set canX ~ 로 세밀 설정	펌웨어 의존
타임스탬프	커널 타임스탬프 제공	제공 제한/불일치 가능
동시성	다중 프로세스 동시 구독 용이	기본 TTY 모델이므로 관리 필요
도구 생태계	can-utils 바로 사용 가능	slcand로 가상 canX 생성 후 일부 동작
ROS2 통합	C++에서 <linux/can.h> raw 소켓 직접 사용	TTY 읽기/쓰기 혹은 가상 canX 경유
설정/운영	간단함 (ip link set canX ...)	데몬 실행 및 추가 관리 필요
자동화/스크립팅	시스템 표준(netlink)으로 일괄 처리 용이	관리 스크립트 필요
유리한 환경	실차 제어/로깅/리플레이	간단한 실험/데모

우리 시스템은 canable v2.0 pro 두 대를 장착해 can0(종방향) 과 can1(횡방향) 을 동시에 운용하고, 각 버스에서 제어 노드·로거·진단이 병렬로 붙는다. SocketCAN 은 인터페이스가 네트워크 장치(can0, can1)로 올라오고, 커널 송수신 큐·하드웨어 타임스탬프·에러프레임·버스오프 자동복구(restart-ms)·에러리포팅(berr-reporting)

을 표준 옵션으로 제공한다. 덕분에 버스트 명령(가감속 램프, 포지션 홀드 keep-alive) 를 내보내는 동안에도 지연과 지터가 작고, 간헐 노이즈나 접촉 불량으로 버스오프가 발생해도 현장 복귀 절차가 단순하다. 또한 동일 인터페이스를 여러 프로세스가 동시에 구독할 수 있어, 예를 들어 can1에서는 횡방향 제어 노드가 송신하는 동안 candump 기반 로거와 진단 노드가 동시에 데이터를 잃지 않고 받아본다. SLCAN은 TTY+ASCII 구조라 단일 소비자 모델에 가깝고, 문자열 인/디코딩·데몬 왕복으로 지연/드롭과 경합 이슈가 더 쉽게 발생했다.

ROS 2 Humble 통합에서도 SocketCAN이 자연스러웠다. 제어 노드들은 `<linux/can.h>` / `<linux/can/raw.h>`로 raw 소켓을 직접 열어 프레임을 주고받고, 표준 ip link로 비트레이트/샘플포인트/에러리포팅/자동재시작을 런타임에 일관되게 설정한다. 실험·로그·재현 사이클을 자주 돌리는 우리 워크플로우(can0 = 구동계, can1=스티어링 동시 제어)에서 커널 타임스탬프와 can-utils(candump, cansend, canplayer, cansniffer)는 관찰 가능성과 실시간성을 높였다. 반면 SLCAN 경로는 slcand 데몬 관리, 포트 비트레이트/옵션 동기화, ASCII 파싱 오버헤드 때문에 운용 복잡도와 리스크가 커졌다.

정리하면, 이중 버스 동시 운용(can0/can1), 다중 노드 병렬 구독, 낮은 지연과 확실한 에러 복구, 표준 도구·스크립트와의 결합성이라는 우리 실차 요건에 SocketCAN이 정확히 부합했다. SLCAN은 간단한 데모에는 유용하지만, 우리의 실차 제어·로깅·진단을 동시에 수행하는 환경에서는 SocketCAN이 더 안전하고 생산적이었다.

iv. VCU - PC 간 시리얼 통신

(A) 파이시리얼 (throttle_serrial.py) 처리

본 링크의 PC 측 소프트웨어는 PySerial을 사용해 USB-CDC 가상 직렬포트 (/dev/ttyACM*)를 연다. 포트는 기본적으로 자동 탐지 되고, 사용자가 port 파라미터로 특정 포트를 명시하면 해당 포트를 연다. 포트를 열 때는 baud=115200, timeout=0.05s로 설정하고, 보드의 자동 리셋/부트로더 구간을 감안해 boot_wait만큼 대기한 뒤 수신 버퍼를 비우고 동기화 절차를 시작한다. 동기화는 헤더 0xAA 탐색으로 수행한다. 연결 후 probe_window 동안 0xAA가 관찰되면 유효 링크로 간주하고, 첫 패킷을 정렬하기 위해 이어지는 3바이트를 읽어 버린다. 연결 재시도 간격은 과도한 리셋을 막기 위해 최소 6초로 제한한다.

오류 처리와 복구는 보수적으로 설계돼 있다. 읽기/쓰기 도중 SerialException 또는 OS 오류가 발생하면 해당 포트를 안전하게 닫고 연결 상태를 None으로 전환한 뒤, 다음 주기부터 자동 재연결을 시도한다. 재연결 이후에는 AEB 동기화 패킷을 우선 전송한다. 수신 측은 스트림이 손상되었거나 헤더가 맞지 않는 경우 해당 바이트를 버리고 다음 헤더를 기다리는 헤더 기반 동기화 전략을 사용하므로, coalescing이나 분할 상황에서도 프레임 경계가 회복된다.

(B) VCU → PC: 상태/입력 참조값

OUT	IN	Transport	baud rate	Data bits	Parity	Stop bits	Polling Period(ms)	Endian
VCU	PC	USB-CDC	115200	8	N	1	10	BigEndian
DATA								
Byte		Bit	ITEM			형식		해석
0			Header			0xAA		고정
1			RPM_H			UInt8		0~400rpm
2			RPM_L			UInt8		
3		0	ASMS			UInt8		0: Autonomus 1: Maunual
		1	E-STOP					0: Normal 1: E-STOP
		2	Gear					0: Forward 1: Reverse
		3~7	Reserved					

수신 루프는 10 ms 주기로 동작한다. 연결이 성립되면 먼저 1바이트를 읽어 헤더 0xAA인지 확인하고, 일치할 때만 뒤이어 3바이트를 더 읽어 총 4바이트 프레임을 완성한다. 두 번째와 세 번째 바이트는 RPM이다. 네 번째 바이트의 비트0은 ASMS, 비트1은 하드웨어 E-STOP, 비트2는 회전 방향으로 해석한다. 소프트웨어는 방향 비트를 참조해 RPM 부호를 결정하되, ± 40 rpm 미만은 노이즈로 간주해 0으로 클램핑한다. 또한 하드웨어 E-STOP과 소프트웨어 AEB 명령((c)의 송신 절차) 중 어느 하나라도 활성화되면 효과적 E-STOP으로 판단하여 중방향과 횡방향 제어를 할 수 있도록 한다. 파싱된 값은 ROS2 throttle_data 토픽으로 퍼블리시되며, 필드는 rpm, estop, asms이다. 값이 변하지 않으면 불필요한 로그 출력을 억제하도록 구성하였다.

(C) PC → VCU: 제어/게이트, 비동기

OUT	IN	Transport	baud rate	Data bits	Parity	Stop bits	Polling Period(ms)	Endian
PC	VCU	USB-CDC	115200	8	N	1	필요 시 송신	BigEndian
DATA								
Byte		Bit	ITEM		형식		해석	
0			Header		0xAB		고정	
1			Reserved		UInt8			
2					UInt8			
3		0	AEB		UInt8		0: Normal 1: Autonomus E-STOP (AEB cmd가 있을 때)	
		2-7	Reserved					

송신 방향은 필요시에만 사용한다. 자율 긴급제동(AEB) 신호를 외부 토픽(/aeb, std_msgs/UInt8, 0/1)으로부터 구독해, 직렬 링크가 열려 있고 직전 송신값과 다를 때 헤더 0xAB 로 시작하는 4바이트 프레임을 MCU로 전송한다. 프레임의 마지막 바이트의 비트0에 AEB를 실어 보낸다. 새로 연결이 성립되면 직전 송신상태를 초기화해 현재 AEB 상태를 즉시 동기화한다. 송신 시에는 write() 후 flush() 로 버퍼링 지연을 최소화한다.

4. 피지컬 디버깅 · 모니터링

4.1. Dashboard 제작

본 Dashboard는 자율주행 차량의 센서 데이터와 제어 상태를 실시간으로 모니터링 하고, 이상 발생 시 신속하게 대응할 수 있도록 제작되었다. 초기 개발 단계에서는 각 센서 토픽을 개별적으로 터미널에서 확인했으나, 데이터 간 상관관계를 파악하기 어렵고 다양한 시각화 툴을 동시에 사용해야 하는 불편함이 있었다. 이러한 한계를 해결하기 위해 Dashboard는 카메라, LiDAR, GPS, IMU, Local Path, Fusion 등 주요 센서 데이터와 차량 속도, 스티어링 각도, RPM, Encoder Error, E-STOP 상태 등을 하나의 화면에서 통합 관리할 수 있도록 설계되었다.

각 센서의 정상 동작 여부는 메시지 수신 주파수(Hz)와 사전에 정의된 값의 범위를 기준으로 평가되며, 결과는 색상과 상태 텍스트를 통해 직관적으로 표현된다. 또한 GPS 정확도 저하, 센서 실패, 제어 이상과 같은 문제 상황을 자동으로 감지하여 Alert 형태로 표시함으로써 사용자가 즉각적으로 대응할 수 있도록 지원한다.

4.1.1. 시스템 구조

Dashboard 시스템은 ROS2 Node -> WebSocket Server -> Frontend의 3계층 구조로 설계되었다. 먼저 ROS2 Node는 자율주행 차량의 카메라, LiDAR, GPS, IMU, Local Path 등 다양한 센서 및 제어 토픽을 구독한다. 수집된 데이터는 내부에서 최근 20개의 타임스탬프를 관리하여 센서별 수신 주파수(Hz)를 계산하고, 유효성 검사 및 Alert 조건 판정을 수행한다. 그 결과는 모니터링 데이터와 차트 데이터의 두 가지 형태로 구분되어 JSON 포맷으로 변환된다.

이후 데이터는 WebSocket Server로 전송된다. WebSocket Server는 ROS2 Node와 Frontend 사이의 중계 계층으로, ROS2에서 전달된 JSON 데이터를 그대로 유지하며 모든 클라이언트에 broadcast 방식으로 배포한다.

마지막으로 Frontend는 WebSocket으로부터 모니터링 데이터와 차트 데이터를 실시간으로 수신한다. 모니터링 데이터는 센서와 제어 상태를 색상 및 상태 텍스트로 표시하며, 차트 데이터는 속도, 스티어링, 가속도, Yaw 등의 항목을 시각화한다. Alert는 CRITICAL과 WARNING 등급으로 구분되어 표시되며, 모든 UI 요소는 WebSocket을 통해 수신된 최신 데이터에 따라 즉시 갱신된다.

4.1.2. 데이터 처리

ROS2 Node에서는 각 센서별로 최근 20개의 메시지 타임스탬프를 기반으로 센서 주파수(Hz)를 계산한다. 이를 통해 센서 데이터의 정상 동작 여부를 평가하며, 값은 사전에 정의된 최소/최대 기준에 따라 유효성을 검사한다. 검사 결과는 GO 또는 NO-GO로 판별되며, 특정 조건을 만족하지 않을 경우 Alert를 발생시킨다. Alert는 CRITICAL과 WARNING 두 단계로 구분되며, 센서 실패, 제어 이상, GPS 정확도 저하와 같은 상황을 구체적으로 구분하여 사용자에게 알린다. 또한 속도, 스티어링, Yaw, 가속도(g_longitudinal, g_lateral)와 같은 데이터는 실시간으로 차트에 반영되어 차량의 동작을 추적할 수 있도록 설계하였다.

4.1.3. 데이터 전송

데이터 전송은 Python의 websockets 라이브러리를 사용하여 WebSocket 기반으로 구현하였다. 초기에는 단순 HTTP 통신으로 데이터를 송신하는 방식을 검토하였으나, 요청-응답 지연으로 인해 실시간 모니터링이 불가능하였다. 이에 따라 양방향 통신이 가능하고 지연 시간이 짧은 WebSocket 프로토콜을 채택하였다.

시스템은 모니터링 데이터와 차트 데이터를 구분하여 전송하도록 설계되었으며, 클라이언트가 연결 해제되거나 전송 오류가 발생하는 상황을 고려하여 자동 재접속 메커니즘을 추가하였다. 초기 버전에서는 단순히 연결이 끊어지면 데이터 송신이 중단되는 문제가 있었으나, 개선 후에는 연결 오류 발생 시 예외 처리를 통해 연결을 초기화하고 재시도하도록 변경하였다.

또한 Hearbeat(Ping/Pong) 메커니즘을 적용하여 비활성 클라이언트나 네트워크 단절 상황을 주기적으로 점검한다. 초기 구현에서는 클라이언트 종료를 감지하지 못해 송신 대기열이 불필요하여 쌓이는 문제가 발생하였다. 이를 해결하기 위해 Ping/Pong 메시지를 활용하여 클라이언트의 생존 여부를 확인하고, 응답이 없을 경우 연결을 종료 후 재연결하도록 수정하였다.

데이터 전송 방식은 연결된 모든 클라이언트에 broadcast 방식으로 이루어지며, 하나의 클라이언트에서 송신 실패가 발생하더라도 다른 클라이언트로의 전송에 영향을 주지 않도록 설계되었다. 이러한 구조를 통해 실시간성, 안정성, 장애 복구 능력을 동시에 확보하였다.

4.1.4. Dashboard UI 설계

초기에는 센서 상태와 차량 동작 데이터를 단순 텍스트 로그 또는 개별 그래프 도구로 확인하였다. 그러나 이 방식은 센서 간의 상태를 직관적으로 비교하기 어렵고, 문제 발생 시 전체 시스템의 영향을 한눈에 파악하기 힘들다는 한계가 있었다.

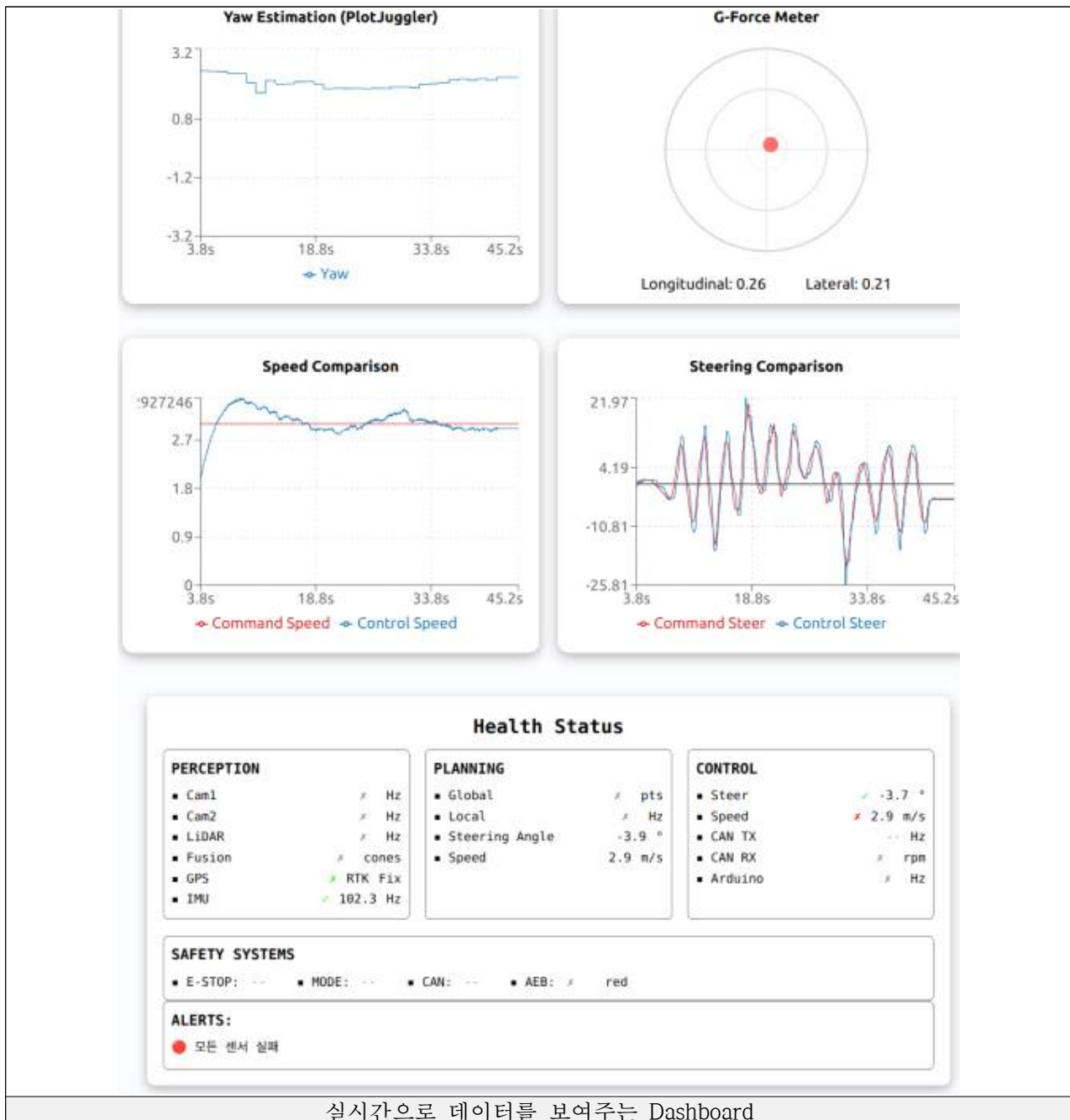
이를 해결하기 위해 Dashboard UI는 React.js 기반으로 통합 설계되었다. Recharts 라이브러리를 활용하여 속도, 스티어링, Yaw, 가속도($g_{longitudinal}$, $g_{lateral}$) 등 주요 데이터를 실시간 차트로 시각화하였으며, 센서 상태는 GO(라임색)와 NO-GO(빨간색)로 구분하여 직관적으로 확인할 수 있도록 하였다. 또한 Alert는 CRITICAL과 WARNING 단계로 분류되어 우선순위에 따라 강조 표시되며, 사용자가 즉각적으로 대응할 수 있도록 설계되었다.

UI 갱신은 WebSocket을 통해 실시간으로 전송되는 데이터를 기반으로 즉시 반영되며, 별도의 새로고침이나 요청이 필요하지 않다. 이러한 구조를 통해 사용자는 복잡한 로그 해석 과정 없이도 센서 및 제어 상태를 직관적으로 파악하고, 문제 발생 시 빠르게 원인을 진단할 수 있다.

4.1.5. 결과 및 활용 예시

본 Dashboard는 실제 테스트워크 상황에 적용한 결과 실시간 모니터링을 통해 디버깅과 성능 검증에 효과적으로 활용될 수 있음이 확인되었다. 센서별 상태, 제어 응답, 차량 동역학 지표가 하나의 화면에서 동시에 갱신되기 때문에, 특정 시점에 발생한 이상 현상을 빠르게 원인별로 추적할 수 있었다. 예를 들어, GPS RTK Fix가 끊어지거나 CAN RX 신호가 수신되지 않을 경우, Health Status 패널과 Alerts 영역이 즉시 이를 감지해 CRITICAL 경고를 발생시켰으며, 사용자는 로그 확인 없이도 문제의 위치를 직관적으로 파악할 수 있었다.

또한 속도·조향 비교 차트를 통해 명령 대비 제어 추종 오차를 실시간으로 검증할 수 있었고, 곡선 구간에서 속도 플레너의 단계적 속도 전환(고·중·저속 구간)이 제어에 반영되는지 직접 확인할 수 있었다. Yaw Estimation과 G-Force Meter는 차량의 자세 변화와 횡가속도를 시각적으로 표현해, 동역학적 안정성을 판단하는 근거 자료로 활용되었다. 이를 통해 단순히 센서 데이터 유무를 확인하는 수준을 넘어, 실제 차량의 응답 특성과 주행 안전성을 분석하는 도구로 발전시킬 수 있었다.



첨부된 사진은 이러한 결과를 잘 보여준다. 상단의 Yaw Estimation 그래프는 차량의 요각 변화가 시간에 따라 안정적으로 유지되는 모습을 나타내며, 오른쪽의 G-Force Meter는 주행 중 발생한 종·횡가속도를 원형 차트로 직관적으로 표현한다. 하단의 Speed Comparison과 Steering Comparison 그래프는 명령 값(빨간색)과 실제 제어 값(파란색)을 비교해 추종 성능을 보여주고 있으며, Health Status 패널은 센서·플래닝·제어 상태를 한눈에 요약한다. 마지막으로 Alerts 영역은 “모든 센서 실패”와 같은 심각한 상황을 즉시 강조해 사용자가 빠르게 대응할 수 있도록 설계되었다.